

Egyptian E-Learning University

Faculty of Computers & Information Technology

Infrastructure Reporting System (I.R.S)

By

Abdul Rahman Safwan Ibrahim Ahmed	2202108
Abdul Rahman Samir Abdel Samie Saeed	2202082
Ammar Hamada Attia Mahmoud	2202066
Ashraf Moawad Ramadan Moawad	2202090
Islam Saad Abdel -Tawab Ragab	2202039
Mahmoud Mustafa Fikry Othman	2202131
Mostafa Ahmed Mohamed Ibrahim	2200107
Omar Rabee Muhammad Khalil	2202119
Yomna Nasser Mustafa Abdul Latif	2202607

Supervised by

Dr. Samar Hisham

Assistant

Eng. Sara Galal

[Al-Fayoum]-2026

Abstract

The **Infrastructure Reporting System** addresses the critical challenge of inefficient urban infrastructure management by providing an integrated, AI-powered platform that connects citizens, government authorities, field workers, and administrators. Traditional reporting methods suffer from fragmentation, slow response times, and poor resource allocation, leading to delayed repairs and reduced public trust. Our solution implements a responsive web application that enables citizens to report infrastructure issues via photo uploads with automatic GPS location tagging. The system employs a **CNN-based AI model** to analyze uploaded images, automatically categorizing issues, and generating descriptive summaries, thereby reducing manual processing time. Key features include **real-time dashboards** for authorities to monitor and assign tasks, **mobile-optimized interfaces** for field workers to receive assignments and update statuses, and **comprehensive admin controls** for system oversight. The platform is built using **Angular** for the frontend, **.Net** for the backend, and integrates **Microsoft SQL Server** for data management. Our project demonstrates how modern web technologies and artificial intelligence can streamline public service workflows, improve response times, and foster greater civic engagement. Initial testing shows promising results in issue of classification accuracy and user satisfaction.

Acknowledgments

We extend our sincere gratitude to our project supervisors, **Dr. Samar** and **Assistant Engineer Sara Galal**, for their invaluable guidance, continuous support, and insightful feedback throughout the development of this project. Their expertise and encouragement have been instrumental in shaping our work.

We thank the **Faculty of Computers & Information Technology at the Egyptian E-Learning University** for providing the resources and academic environment that made this research possible. Special appreciation goes to our colleagues and peers for their constructive discussions and moral support.

Contents

Abstract	1
Acknowledgments	2
Chapter 1	6
Introduction	7
1.1 Introduction	7
1.2 Background and Motivation	7
1.3 Importance of the Problem	7
1.4 Problem Statement	8
1.5 Objectives	8
1.6 Brief Overview of the Proposed Solution	8
Chapter 2	9
Literature Review / Related Work	10
2.1 Introduction	10
2.2 Related Work and Critical Gap Analysis	10
2.2.1 Advanced AI in Infrastructure Health Monitoring	10
Gaps in AI-Only Systems	11
2.2.2 Existing Citizen Reporting Platforms	12
2.3 Conclusion and Project Contribution	13
Chapter 3	14
Proposed system	14
3.1 Introduction	14
3.2 Approach Used to Solve the Problem	14
3.3 System Architecture:	15

3.3.1 Use Case Diagram:	16
3.3.1.1 Citizens	17
3.3.1.2 For Government Authorities	18
3.3.1.3 Worker.....	18
3.3.1.4 Administrator Role.....	18
3.3.2 Use Case Scenarios:.....	18
3.3.3 Sequence Diagram:.....	22
3.3.3.1 Public User Sequence diagram:	23
3.3.3.1.1 Public User Epic: Authentication & Account Management.....	23
3.3.3.1.2 Public User Epic: Issue Reporting	24
3.3.3.1.3 Public User Epic: Tracking & Resolution	24
3.3.3.1.4 Public User Epic: Profile Management.....	24
3.3.3.2 Authority User Sequence diagram:	24
3.3.3.2.1 Government Authority Epic: Authentication & Profile Management.....	24
3.3.3.2.2 Government Authority Epic: Report Management & Assignment	24
3.3.3.3 Worker User Sequence diagram:.....	24
3.3.3.3.1 Worker Epic: Authentication & Profile Management	24
3.3.3.3.2 Worker Epic: Task Management	24
3.3.5 Data Flow Diagram:	24
3.3.5.1 context Diagram (Level 0 DFD):.....	24
3.3.5.2 Level 1 DFD:	24
Chapter 4.....	24
Implementation	24
4.1 Front-End Implementation.....	24
4.1.1 Front-End Architecture Overview	24

4.1.2 Front-End Project Structure.....	24
4.1.3 Technologies and Development Tools	24
Angular:	24
TypeScript:	24
HTML5 and CSS3:	24
Angular Router:	24
Angular HTTP Client:.....	24
Git and GitHub:.....	24
4.1.4 Routing and Navigation System	24
4.1.5 Front-End Modules.....	24
4.1.5.1 Authentication Module.....	24
4.1.5.2 Public User Module	24
4.1.5.3 Authority Module.....	24
4.1.5.4 Worker Module	24
4.1.5.5 Administrator Module.....	24
4.1.5.6 Profile Management Module	24
4.1.5.7 Shared Components and Services	24
4.1.6 API Integration	24
4.1.7 Challenges Faced and Solutions	24
Challenge 1: Managing Multiple User Roles.....	24
Solution:	24
Challenge 2: Integration with the Back-End API	24
Solution:	24
Challenge 3: Form Validation and Data Accuracy	24
Solution:	24

Challenge 4: Responsive User Interface	24
Solution:	24
Challenge 5: Navigation and Session Management.....	24
Solution:	24
4.1.8 User Interface Implementation	24
The landing page:.....	24
Smart Features Section:	24
How It Works Section:	24
Partners in Change & Success Section:	24
Empowering Community Action Section:	24
Authentication: Login Portal.....	24
Authentication: User Registration Portal	24
This component provides a streamlined interface for new users to register within the system, ensuring secure data capture and integration with the backend authentication services to establish new user roles.	
Authentication: OTP Verification Module	24
Authentication: Password Recovery Module	24
Authentication: Password Reset Verification.....	24
Authentication: Password Reset Completion.....	24
Public User Dashboard: Issue Reporting Interface	24
AI-Powered Reporting: Automated Issue Analysis	24
User Dashboard: Report Tracking Interface	24
User Profile: Personal Information Management	24
Authority Dashboard:.....	24
Authority Dashboard: Live Monitoring Interface	24
Authority Dashboard: Incoming Report Details	24

Authority Dashboard: Geographical Report Mapping	24
Worker Dashboard:	24
Worker Portal: Task Management Interface	24
Worker Portal: Active Task Management	24
Worker Portal: Task History Archive	24
Administrator Dashboard:.....	24
Administrator Dashboard: System Status Overview	24
Administrator Dashboard: Operational Oversight & Audit	24
Administrator Dashboard: Users Management.....	24
Administrator Dashboard: User Creation & Provisioning.....	24
Administrator Dashboard: System Audit Logs	24
Administrator Dashboard: Detailed System Audit Trail	24
4.1.9 State Management and Data Flow	24
4.1.10 Security Implementation.....	24
4.1.11 Error Handling and User Feedback	24
4.1.12 Reusable Components and Code Reusability	24
4.1.13 Front-End Performance Considerations	24
4.1.14 Summary.....	24
4.2 Back-End Implementation	24
4.2.1 Architecture Overview.....	24
4.2.2 Project Structure	24
4.2.3 Domain Layer	24
4.2.4 Database Design and Persistence Layer	24
4.2.5 Feature Organization Pattern	24
4.2.6 Core Business Features: Public User.....	24

Submit Report	24
Nearby Reports	24
Also Suffer	24
Map and Marker Popup	24
My Reports.....	24
4.2.7 Core Business Features: Authority	24
Incoming Reports.....	24
Assign Worker.....	24
Workers Management	24
Map and Marker Popup	24
4.2.8 Core Business Features: Worker.....	24
Tasks Screen.....	24
Current Task Screen	24
Tasks History Screen.....	24
4.2.9 Core Business Features: Admin.....	24
Dashboard Screen	24
User Provisioning.....	24
Users Management.....	24
Audit Logs	24
4.2.10 Authentication and Authorization System	24
Registration	24
OTP-Based Email Confirmation.....	24
Login and Token Issuance.....	24
Token Refresh	24
Password Recovery	24

Logout	24
4.2.11 AI Service Integration	24
4.2.12 External Services: Email and Image Management	24
Email Service	24
Image Service.....	24
4.2.13 Configuration and Secrets Management.....	24
4.2.14 API Documentation with Swagger	24
4.2.15 CI/CD Pipeline	24
4.2.16 API Testing with Postman	24
4.2.17 Deployment	24
Chapter 5	24
Testing & Evaluation	24
Chapter 6	24
Results & Discussion	24
6.1 Introduction:.....	24
6.2 Summary of Findings:.....	24
6.3 Interpretation of Results:.....	24
Objective 1: Providing a Multi-Role Infrastructure Reporting Platform	24
Objective 2: Improving Issue Management and Coordination.....	24
Objective 3: Enhancing Security and Reliability	24
6.4 Limitations of the Proposed Solution:	24
Manual Reporting Dependency	24
AI Dataset Limitations.....	24
Large-Scale Deployment Evaluation	24
Lack of Mobile Applications	24

External System Integration	24
Chapter 7	24
Conclusion & Future Work	24
7.1 Summary of Contributions.....	24
7.2 Strengths of the Proposed System:	24
7.3 Challenges and Limitations:	24
7.4 Future Work	24
Summary:.....	24
References	24

LIST OF FIGURES

Figure 2.1: Example of automated pothole detection using the YOLOv5 algorithm

Figure 2.2: A professional smart city monitoring dashboard

Figure 3.1: Use Case Diagram of the Infrastructure Reporting System

Figure 3.2: Public User Sequence Diagram: Authentication and Account Management (Part 1)

Figure 3.3: Public User Sequence Diagram: Authentication and Account Management (Part 2)

Figure 3.4: Public User Sequence Diagram: Authentication and Account Management (Part 3)

Figure 3.5: Public User Sequence Diagram: Authentication and Account Management (Part 4)

Figure 3.6: Public User Sequence Diagram: Issue Reporting

Figure 3.7: Public User Sequence Diagram: Tracking and Resolution

Figure 3.8: Public User Sequence Diagram: Profile Management (Part 1)

Figure 3.9: Public User Sequence Diagram: Profile Management (Part 2)

Figure 3.10: Public User Sequence Diagram: Profile Management (Part 3)

Figure 3.11: Public User Sequence Diagram: Profile Management (Part 4)

Figure 3.12: Public User Sequence Diagram: Profile Management (Part 5)

Figure 3.13: Authority Sequence Diagram: Authentication and Profile Management (Part 1)

Figure 3.14: Authority Sequence Diagram: Authentication and Profile Management (Part 2)

Figure 3.15: Authority Sequence Diagram: Authentication and Profile Management (Part 3)

Figure 3.16: Authority Sequence Diagram: Report Management and Assignment (Part 1)

Figure 3.17: Authority Sequence Diagram: Report Management and Assignment (Part 2)

Figure 3.18: Worker Sequence Diagram: Authentication and Profile Management (Part 1)

Figure 3.19: Worker Sequence Diagram: Authentication and Profile Management (Part 2)

Figure 3.20: Worker Sequence Diagram: Authentication and Profile Management (Part 3)

Figure 3.21: Worker Sequence Diagram: Authentication and Profile Management (Part 4)

Figure 3.22: Worker Sequence Diagram: Task Management (Part 1)

Figure 3.23: Worker Sequence Diagram: Task Management (Part 2)

Figure 3.24: Entity Relationship Diagram (ERD)

Figure 3.25: Context Diagram (Level 0 DFD)

Figure 3.26: Level 1 Data Flow Diagram (DFD)

Chapter 1

Introduction

1.1 Introduction

The rapid urbanization of modern cities has led to increasing challenges in maintaining public infrastructure. Issues such as potholes, broken streetlights, water leaks, and waste accumulation affect citizens' daily lives and pose safety risks. Traditional reporting methods are often slow, fragmented, and inefficient, resulting in delayed responses and wasted municipal resources. This project proposes the **Smart Infrastructure Reporting System**—an AI integrated, web-based platform that connects citizens, government authorities, and field workers to streamline the reporting, management, and resolution of urban infrastructure issues.

1.2 Background and Motivation

Currently, citizens report infrastructure problems through multiple channels—phone hotlines or municipal websites—which are often disconnected from authorities' workflows. Studies indicate that only a small percentage of issues are reported due to cumbersome processes, and response times can be excessively long. Moreover, authorities struggle with manual task prioritization and resource allocation. The motivation behind this project is to leverage modern web technologies and artificial intelligence to create a unified system that improves responsiveness, transparency, and civic engagement.

1.3 Importance of the Problem

Inefficient infrastructure management leads to:

- Increased public safety risks
- Higher maintenance costs due to poor resource allocation
- Low citizen trust in municipal services
- Delayed economic development

Solving this problem can enhance urban livability, optimize government expenditure, and foster a collaborative relationship between citizens and local authorities.

1.4 Problem Statement

Citizens lack a simple, unified platform to report infrastructure issues and track their resolution. Authorities lack real-time tools to prioritize, assign, and monitor repairs efficiently. Field workers often receive incomplete task information and lack direct communication channels with reporters. This project addresses these gaps by developing an **integrated web-based reporting system** that uses AI for automatic issue categorization and description generation, provides real-time connection through all stakeholders, and ensures accessibility through responsive design for both desktop and mobile devices.

Justification: The problem is worth solving because it impacts daily life, public safety, and municipal efficiency. A well-designed system can significantly reduce response times and operational costs while increasing citizen participation.

1.5 Objectives

Main Objective:

To design and develop a web-based Smart Infrastructure Reporting System that enables efficient reporting, structured task management, and seamless coordination among citizens, authorities, and field workers.

Specific Objectives:

1. To develop a multi-role web platform (Public User, Authority, Worker, Admin) with secure authentication and role-based dashboards.
2. To implement an AI module for automatic report **categorization and description generation** based on image analysis.
3. To create a real-time interactive map for visualizing reported issues and assigned tasks.
4. To design a notification and tracking system that keeps all stakeholders informed throughout the report lifecycle.
5. To ensure system scalability, security, and usability through responsive design and rigorous testing.

1.6 Brief Overview of the Proposed Solution

The system will be a **responsive website** accessible on both desktop and mobile browsers. It will include:

- A **Public User interface** for reporting issues with photos, GPS, and AI-assisted categorization/description.
- An **Authority dashboard** with live maps, report management, and manual assignment features.
- A **Worker Profile** for viewing tasks, updating statuses, and uploading completion evidence.
- An **Admin panel** for user management and system oversight.

Chapter 2

Literature Review / Related Work

2.1 Introduction

This chapter establishes the practical and academic context for the Smart Infrastructure Reporting System (SIRS) project by conducting a comprehensive review of related work and existing solutions. The primary objective is to critically analyze the current state of urban infrastructure reporting mechanisms, identify their systemic deficiencies, and thereby justify the necessity and define the unique contribution of the proposed SIRS framework. The chapter concludes with a synthesis of the findings, positioning SIRS as a novel, integrated solution designed to address these identified gaps.

2.2 Related Work and Critical Gap Analysis

The current landscape of infrastructure reporting can be broadly categorized into two areas: advanced AI applications for infrastructure health monitoring (research focus) and established citizen reporting platforms (system focus). A critical analysis of each area reveals specific limitations that the SIRS project is designed to overcome.

2.2.1 Advanced AI in Infrastructure Health Monitoring

Recent academic literature highlights the significant performance gains achieved through the application of AI in infrastructure management, primarily focusing on predictive maintenance and automated fault detection [1] [2]. These systems demonstrate high accuracy in fault prediction and traffic forecasting, leading to substantial operational improvements.

Technology Domain	Key Functionality	Performance Metrics (Research Findings)
-------------------	-------------------	---

Predictive Maintenance	Failure anticipation and system reliability enhancement.	Up to 92% accuracy in traffic forecasting; 30% faster response times; 25-30% productivity gains [1].
Resource Optimization	AI-based balancing of supply/demand and maintenance scheduling.	15% energy savings; 25-30% cost reductions; 25% congestion reduction [1].



Figure 2.1: Example of automated pothole detection using the YOLOv5 algorithm, demonstrating the use of bounding boxes for real-time infrastructure issue identification [8].

Gaps in AI-Only Systems

While powerful, these AI-centric solutions often operate in isolation from the public interface, leading to two critical gaps:

Lack of Citizen Integration: These systems typically rely on fixed sensors or dedicated inspection vehicles, failing to leverage the vast, real-time, and geographically distributed data source provided by citizens. They are not designed to process the unstructured, multimedia data (photos, text descriptions) inherent in citizen reports.

Siloed Data and Fragmentation: The focus remains on technical data rather than integrating with municipal workflow data, preventing a unified operational view [3].

2.2.2 Existing Citizen Reporting Platforms

Platforms such as the NYC 311 system and various commercial solutions (e.g., Hexagon, GovPilot) have successfully digitalized the reporting process, serving as the primary interface for civic engagement [4] [5].

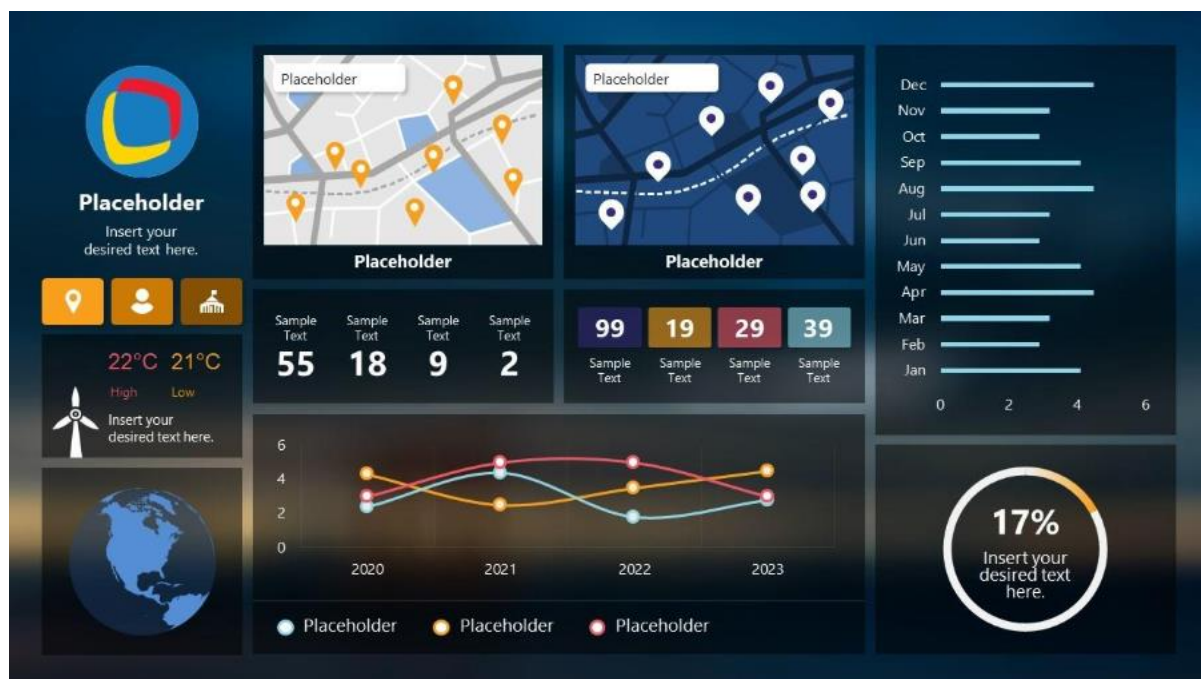


Figure 2.2: A professional smart city monitoring dashboard, illustrating the integration of various urban data streams but highlighting the complexity of manual prioritization in current systems [3].

Gaps in Existing Reporting Platforms

A comparative analysis reveals that these platforms suffer from systemic deficiencies:

Deficiency in Intelligent Prioritization: These systems primarily function as data loggers, relying on manual or rule-based prioritization. They lack the advanced, real-time, AI-driven algorithms necessary to prioritize reports based on urgency and impact.

Low Civic Engagement and Verification Challenges: Current systems struggle with low citizen participation due to complicated processes and a lack of transparency [3]. Furthermore, the reliance on unverified, single-source reports introduces a vulnerability to data integrity issues.

2.3 Conclusion and Project Contribution

This literature review and critical gap analysis confirm that while the foundational technologies for smart infrastructure management exist, a critical need remains for a fully integrated, intelligent, and citizen-centric reporting system.

The Smart Infrastructure Reporting System (SIRS) is proposed as a direct solution to the identified gaps. Its unique contribution lies in the integration of three novel components:

Unified Platform: Eliminating fragmentation by connecting all stakeholders in a single, real-time ecosystem.

AI-Driven Decision Engine: Utilizing CNNs and Reinforcement Learning for automated and resource optimization, supported by the Urban Issues Dataset for robust training.

Community Verification System: A novel mechanism that leverages collective intelligence to validate reports, significantly enhancing data accuracy and fostering greater civic participation.

The subsequent chapters will detail the architectural design and implementation of the SIRS framework, demonstrating how it overcomes the limitations of current systems to deliver a truly smart, responsive, and sustainable urban management solution.

Chapter 3

Proposed system

3.1 Introduction

This chapter presents the proposed Smart Infrastructure Reporting System, which is designed to enhance the process of reporting and managing urban infrastructure issues. The proposed system aims to overcome the limitations of traditional methods, including slow response times, inefficient coordination, and poor resource utilization.

The system provides an integrated platform that connects citizens, government authorities, and field crews. Citizens can submit infrastructure reports through a simple mobile application, while authorities receive real-time information to support issue prioritization and decision-making. Field crews are assigned tasks efficiently and can update the status of issues directly from the field.

The proposed system incorporates artificial intelligence techniques to classify reported issues. In addition, a community verification mechanism is used to improve report accuracy and transparency. Overall, the system seeks to improve response time, optimize maintenance operations, and support effective urban infrastructure management.

3.2 Approach Used to Solve the Problem

Several infrastructure reporting systems have been developed with the aim of improving communication between citizens and government authorities. While these systems contribute to

issue reporting, many existing approaches suffer from limitations such as fragmented reporting channels, manual or rule-based prioritization, limited real-time coordination between stakeholders, and insufficient use of intelligent data analysis. As a result, response times remain high, resources are often inefficiently allocated, and citizen engagement is relatively low.

To address these limitations, the proposed solution adopts a **web-based, intelligent approach** that integrates all stakeholders within a single unified platform. The system enables citizens to report infrastructure issues easily through a responsive web interface by submitting images, descriptions, and location information. Unlike traditional systems, the proposed approach applies artificial intelligence techniques to automatically classify reported issues and determine their priority based on urgency and potential impact.

Government authorities interact with the system through a centralized web dashboard that provides real-time visibility of reported issues. Field crews are connected to the same platform through a mobile-optimized web interface, allowing them to receive tasks, update progress, and submit repair information directly from the field.

By combining web technologies with AI-based analysis and centralized communication, the proposed approach overcomes the limitations of existing systems by reducing response time, improving resource utilization, enhancing transparency, and increasing citizen participation in urban infrastructure management

3.3 System Architecture:

The proposed Smart Infrastructure Reporting System follows a web-based, multi-layer architecture designed to ensure scalability, maintainability, and efficient communication between system components. The architecture integrates frontend interfaces, backend services, artificial intelligence modules, and data storage within a unified platform.

The presentation layer consists of a responsive web application that provides different interfaces for citizens, government authorities, and field crews. This layer handles user interaction, report submission, dashboards, and task management, and communicates with the backend through secure RESTful APIs.

The application layer represents the backend server, which is responsible for handling business logic, user authentication, data validation, and workflow management. This layer coordinates communication between the frontend, the AI processing modules, and the database, ensuring that reports are processed and routed correctly.

The intelligence layer includes AI and machine learning components used for image analysis, issue classification, and priority assessment. These components analyze incoming reports to support automated decision-making while allowing human oversight by authorities.

The data layer manages the storage of all system data, including user information, infrastructure reports, task assignments, and historical records. This layer supports both operational processes and data analysis for system improvement.

Through this layered architecture, the proposed system enables real-time data exchange, intelligent processing, and seamless collaboration among all stakeholders, providing an efficient solution for urban infrastructure reporting and management.

3.3.1 Use Case Diagram:

A Use Case diagram helps identify the main components and actions that make up a App. In this diagram, the main components are known as "actors," and the actions are referred to as "use cases." It illustrates how actors interact with each use case. The Use Case diagram focuses on the functional aspects of a App, specifically capturing the business processes it performs.

Additionally, these diagrams define the requirements of the modelled App, serving as a basis for creating test scripts for the App being modelled.

Use case

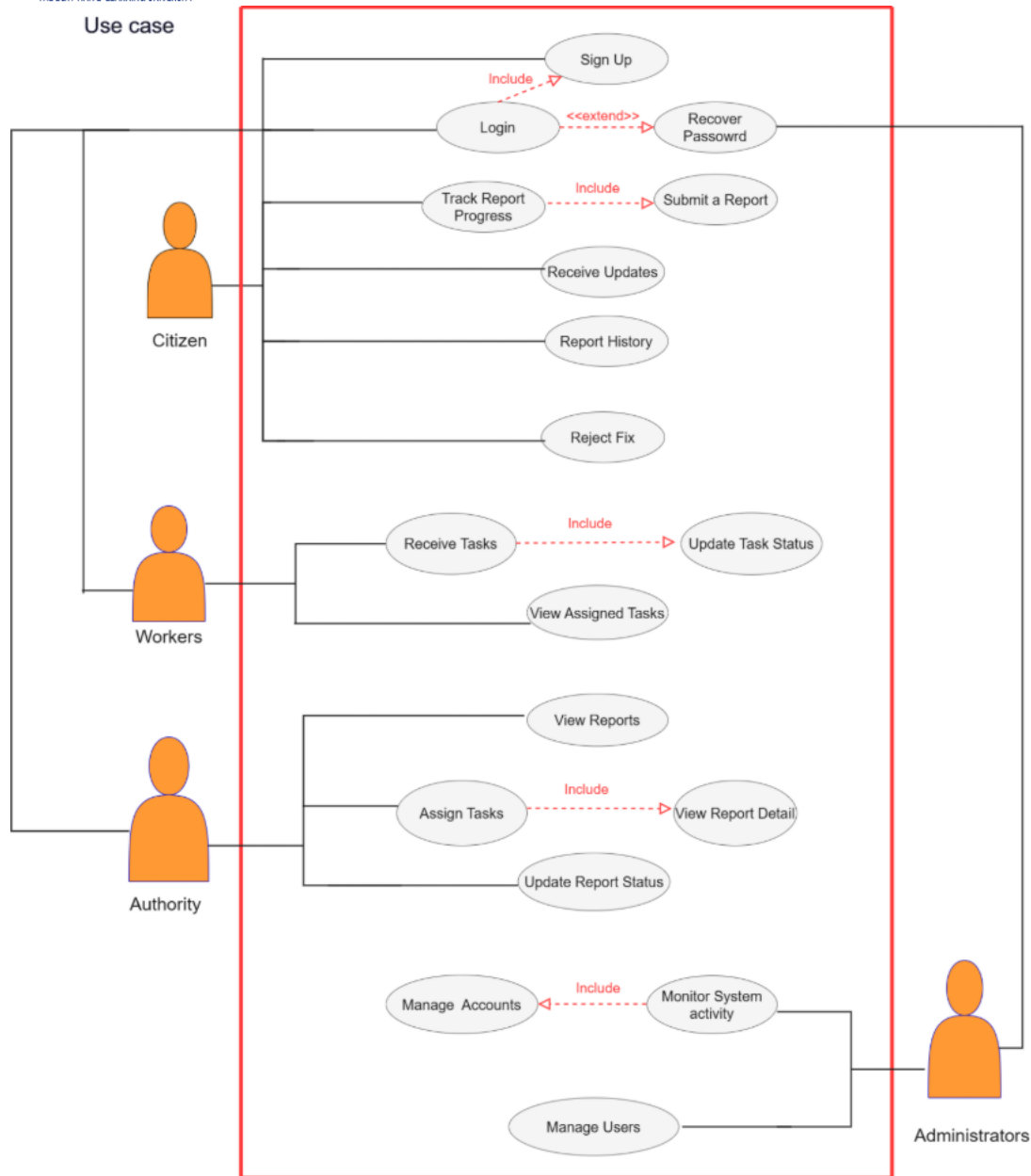


Figure 3.1: Use Case Diagram of the Infrastructure Reporting System

3.3.1.1 Citizens

- ☐ **Report a problem:** Report an infrastructure issue via a mobile app.
- ☐ **Attach photos and location:** Attach photos and use GPS to tag the location of the issue.
- ☐ **Track report status:** Receive notifications and updates on the status of their report.
- ☐ **Participate in verification:** Verify reports submitted by other nearby users.

- ☐ **View issues on a map:** See all reported issues in their area on an interactive map.

3.3.1.2 For Government Authorities

- ☐ **Classify and prioritize reports:** Review reports using a central dashboard, with AI classification and prioritization.
- ☐ **Assign tasks:** Automatically route tasks to the correct department or field crew.
- ☐ **Monitor performance and analytics:** Use a dashboard with analytics and KPIs to monitor system performance.

3.3.1.3 Worker

- ☐ **Receive assignments:** Get assigned tasks via a mobile app.
- ☐ **Locate and navigate to the problem:** Use maps within the system to reach the issue's location.
- ☐ **Document work:** Document their repairs with "before" and "after" photos.
- ☐ **Update task status:** Change the status of a task once it is completed.
- ☐ **Communicate with authorities:** Have tools for communicating with government authorities.

3.3.1.4 Administrator Role

- ☐ **Login & Security:** Securely log in to the administrative dashboard to access system controls and data.
- ☐ **Report Validation (Data Integrity): Validate Reports:** Review and verify the authenticity of citizen-submitted reports and categorize them. This step is crucial to filtering out false submissions.
- ☐ **Task Assignment (Workflow Management): Assign Tasks:** Direct the validated reports as official work assignments to the appropriate **Field Worker** teams.
- ☐ **Crucial Inclusion:** This action **includes** the validation step

(Assign Tasks →<(include)> Validate Reports).

3.3.2 Use Case Scenarios:

Use Case Name	Details
Sign Up	• Actor: Citizen / Guest • Precondition: The user is not already registered. • Main Flow: The user opens the registration page. • The user enters registration details (name, email, password, basic information) • The system validates the entered data. • The system creates a new citizen account. • The system displays a successful registration message • Alternative Flow: If email is already used → show error message and ask for a different email.
Login	• Actor: Citizen / Worker / Administrator / Authority • Precondition: User has a registered account. • Main Flow: User enters email and password. System validates the credentials. System redirects the user to their role-specific dashboard. • Alternative Flow: If password is incorrect → show error message. If account is suspended → show access denied message.
Recover Password	• Actor: Users • Precondition: Registered email • Main Flow: Request reset → receive link → set new password • Alternative Flow: Invalid email → error • Postcondition: Password updated
Submit Report	• Actor: Citizen • Precondition: User is logged in

	• Main Flow: User fills report details and submits → system saves report • Alternative Flow: Missing or invalid data → submission rejected • Postcondition: Report is registered in the system
Track Report Progress	• Actor: Citizen • Precondition: Report already submitted • Main Flow: User views report status and updates • Alternative Flow: Report not found → error message • Postcondition: Report progress is displayed
Receive Updates	• Actor: Citizen • Precondition: Citizen has a submitted report • Main Flow: System sends status updates to citizen • Alternative Flow: Notification not delivered • Postcondition: Citizen is informed about report changes
Report History	• Actor: Citizen • Precondition: Citizen logged in • Main Flow: Citizen views list of previous submitted reports • Alternative Flow: No previous reports found • Postcondition: Report history displayed
Reject Fix	• Actor: Citizen • Precondition: Report marked as fixed • Main Flow: Citizen reviews fix and rejects it • Alternative Flow: Citizen accepts fix • Postcondition: Report reopened for further action
Receive Tasks	• Actor: Worker

	• Precondition: Task assigned by authority • Main Flow: System notifies worker and displays task details • Alternative Flow: — • Postcondition: Task appears in worker task list
View Assigned Tasks	• Actor: Worker • Precondition: Worker logged in • Main Flow: Worker views assigned tasks with details and priorities • Alternative Flow: No tasks assigned • Postcondition: Assigned tasks displayed
Update Task Status	• Actor: Worker • Precondition: Task assigned • Main Flow: Worker updates task progress or completion status → system saves changes • Alternative Flow: Invalid status update • Postcondition: Task status updated
View Reports	• Actor: Authority • Precondition: Authority logged in • Main Flow: Authority views list of submitted reports with details • Postcondition: Reports reviewed
Assign Tasks	• Actor: Authority • Precondition: Report available for processing • Main Flow: Authority selects report → assigns suitable worker • Postcondition: Task assigned successfully

Update Report Status	• Actor: Authority • Precondition: Task status updated by worker • Main Flow: Authority changes report status based on task progress • Alternative Flow: Invalid status selection • Postcondition: Report status updated
Manage Accounts	• Actor: Administrator • Precondition: Admin authenticated • Main Flow: Admin creates or manages authority and worker accounts • Alternative Flow: Duplicate email detected • Postcondition: Privileged accounts updated
Monitor System Activity	• Actor: Administrator • Precondition: Admin logged in • Main Flow: Admin reviews system logs and activities • Alternative Flow: No activity available • Postcondition: System activity monitored
Manage Users	• Actor: Administrator • Precondition: Admin logged in • Main Flow: Admin adds, updates, or removes user accounts • Alternative Flow: Invalid or incomplete data • Postcondition: User records updated

3.3.3 Sequence Diagram:

A **Sequence Diagram** helps identify the interactions and message flow that occur within an application. In this diagram, the main components are **actors or objects**, and the interactions are represented as **messages** exchanged in a specific order. It illustrates how actors and system components communicate over time to achieve a particular functionality. The Sequence Diagram focuses on the **dynamic behavior** of an application, specifically modeling the execution flow of business processes. Additionally, these diagrams support the detailed design of the modeled application and serve as a basis for validating system behavior and creating test cases.

3.3.3.1 Public User Sequence diagram:

3.3.3.1.1 Public User Epic: Authentication & Account Management

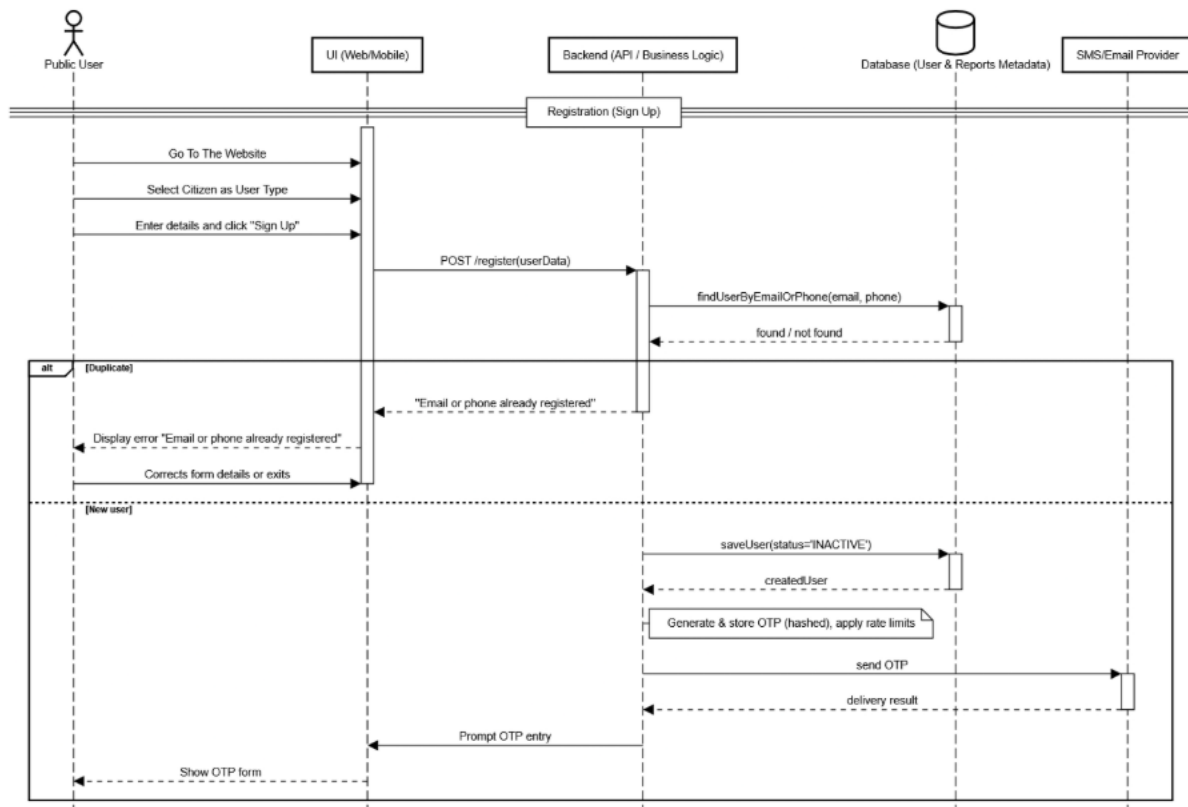
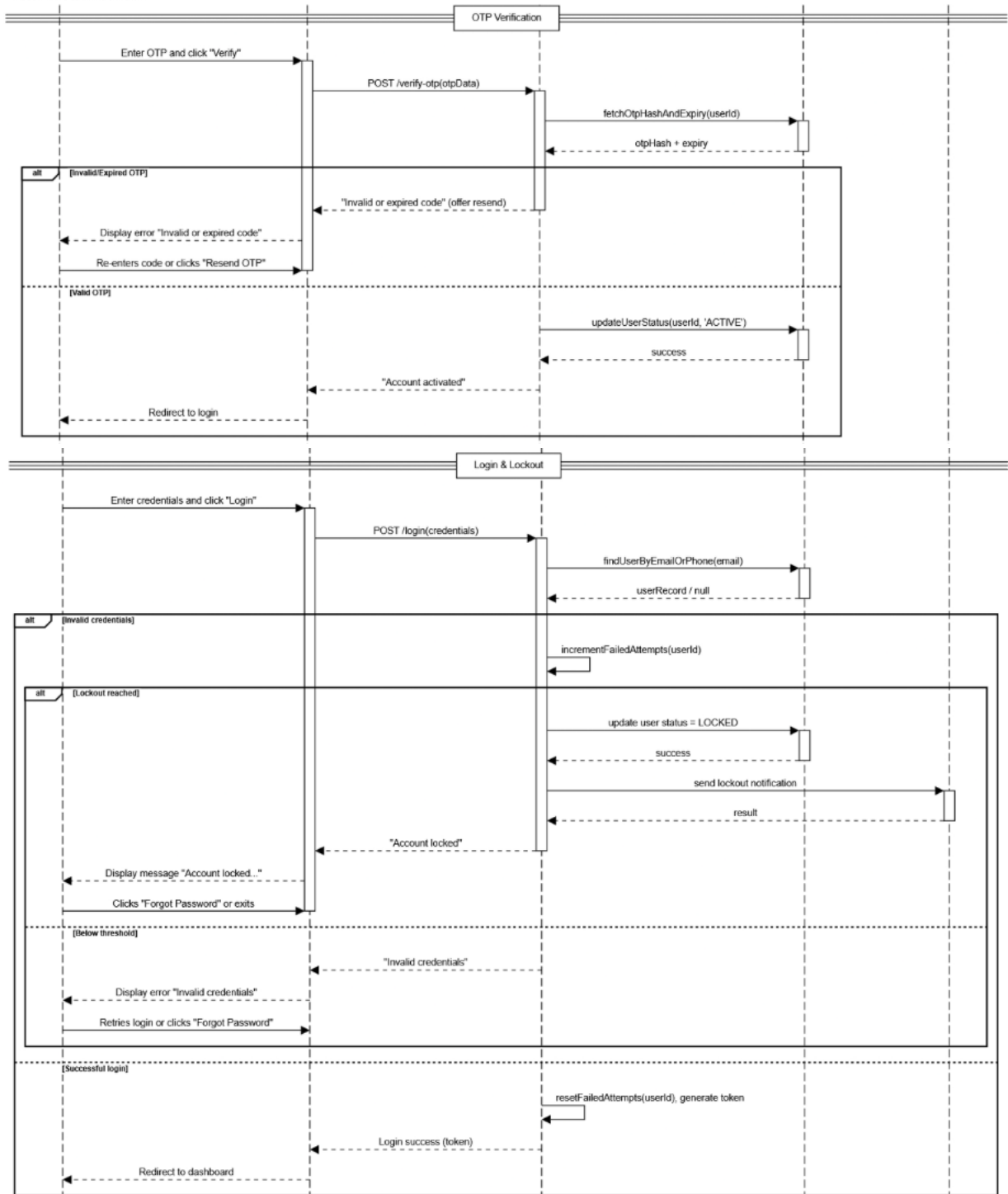


Figure 3.2: Public User Sequence Diagram: Authentication and Account Management (Part 1)



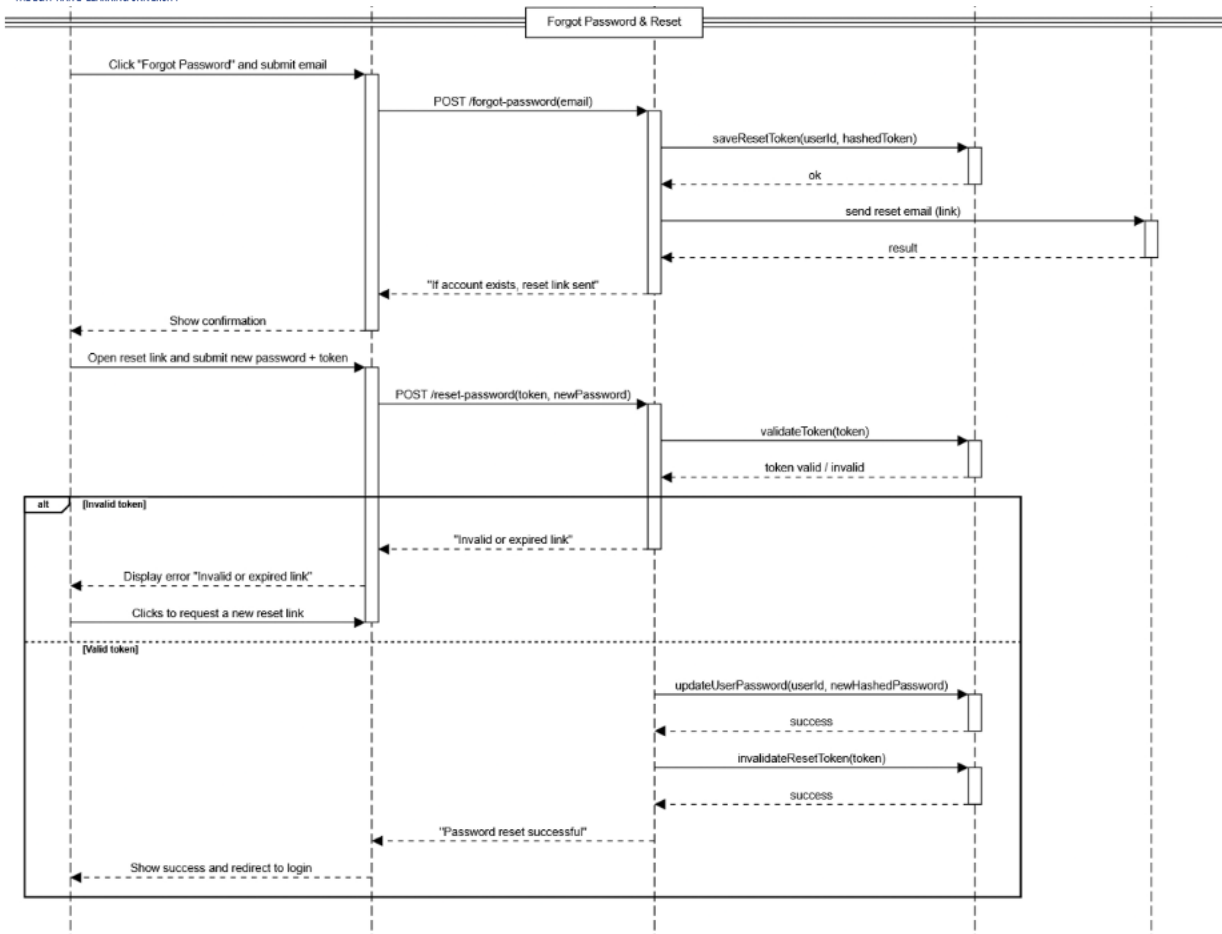


Figure 3.4: Public User Sequence Diagram: Authentication and Account Management (Part 3)

Figure 3.3: Public User Sequence Diagram: Authentication and Account Management (Part 2)

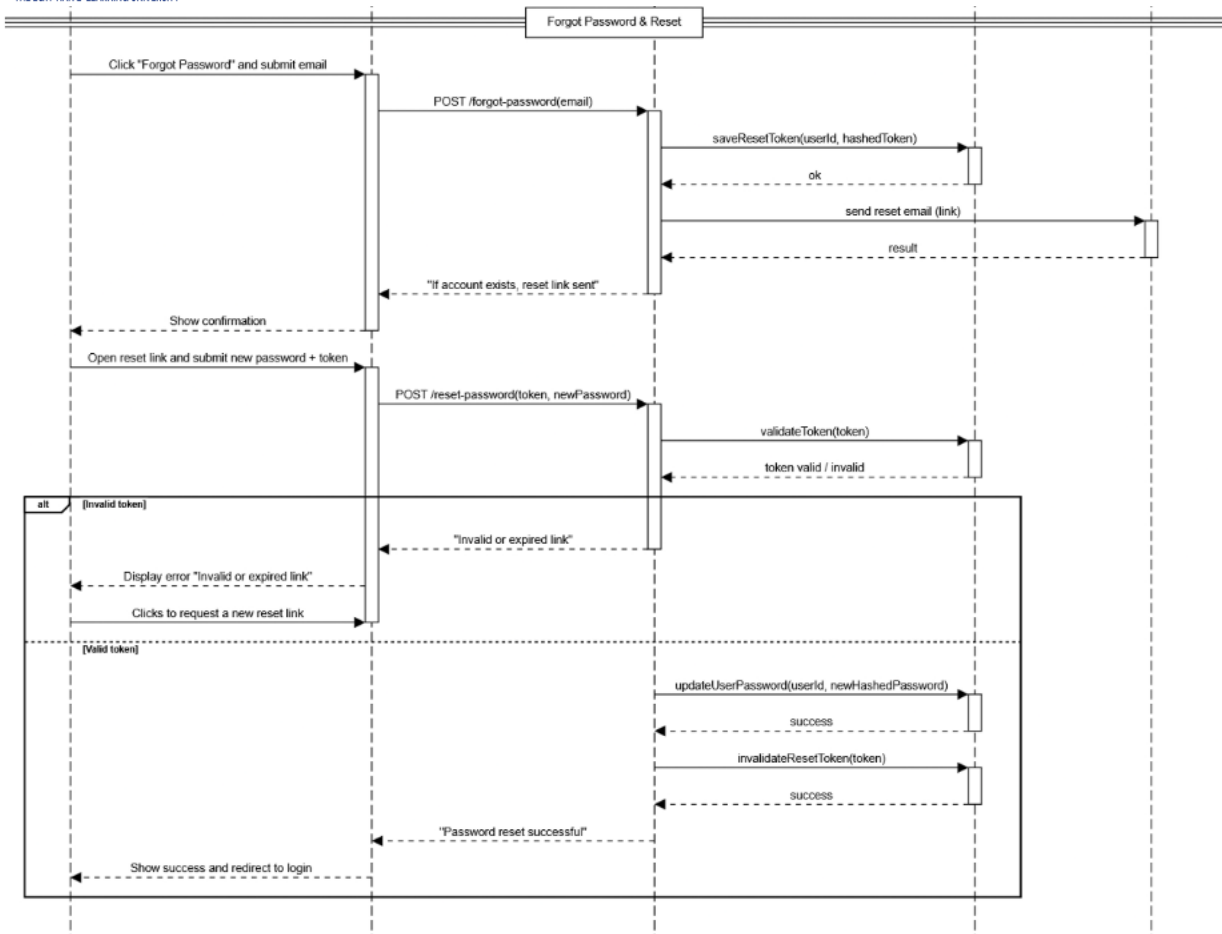


Figure 3.5: Public User Sequence Diagram: Authentication and Account Management (Part 4)

3.3.3.1.2 Public User Epic: Issue Reporting

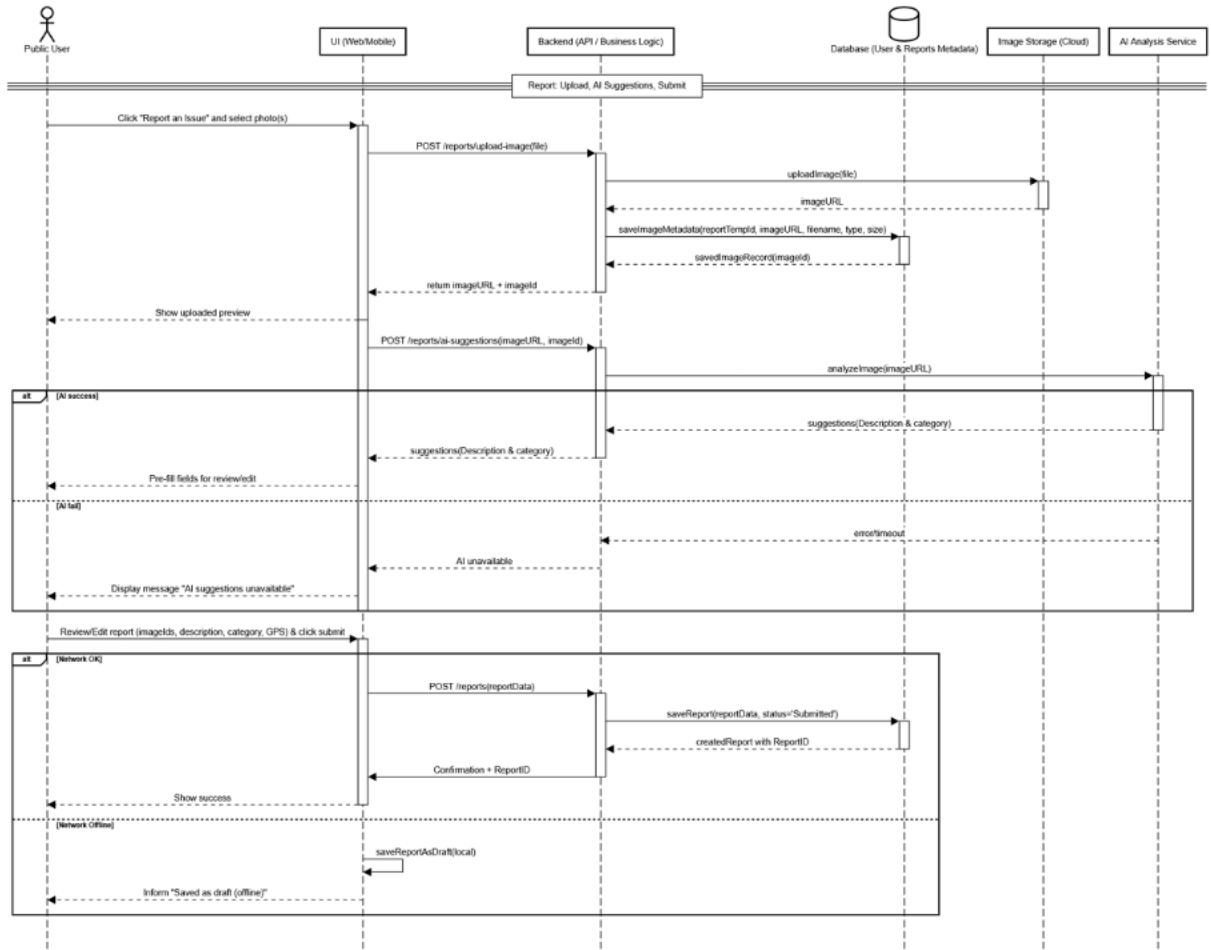


Figure 3.6: Public User Sequence Diagram: Issue Reporting

3.3.3.1.3 Public User Epic: Tracking & Resolution

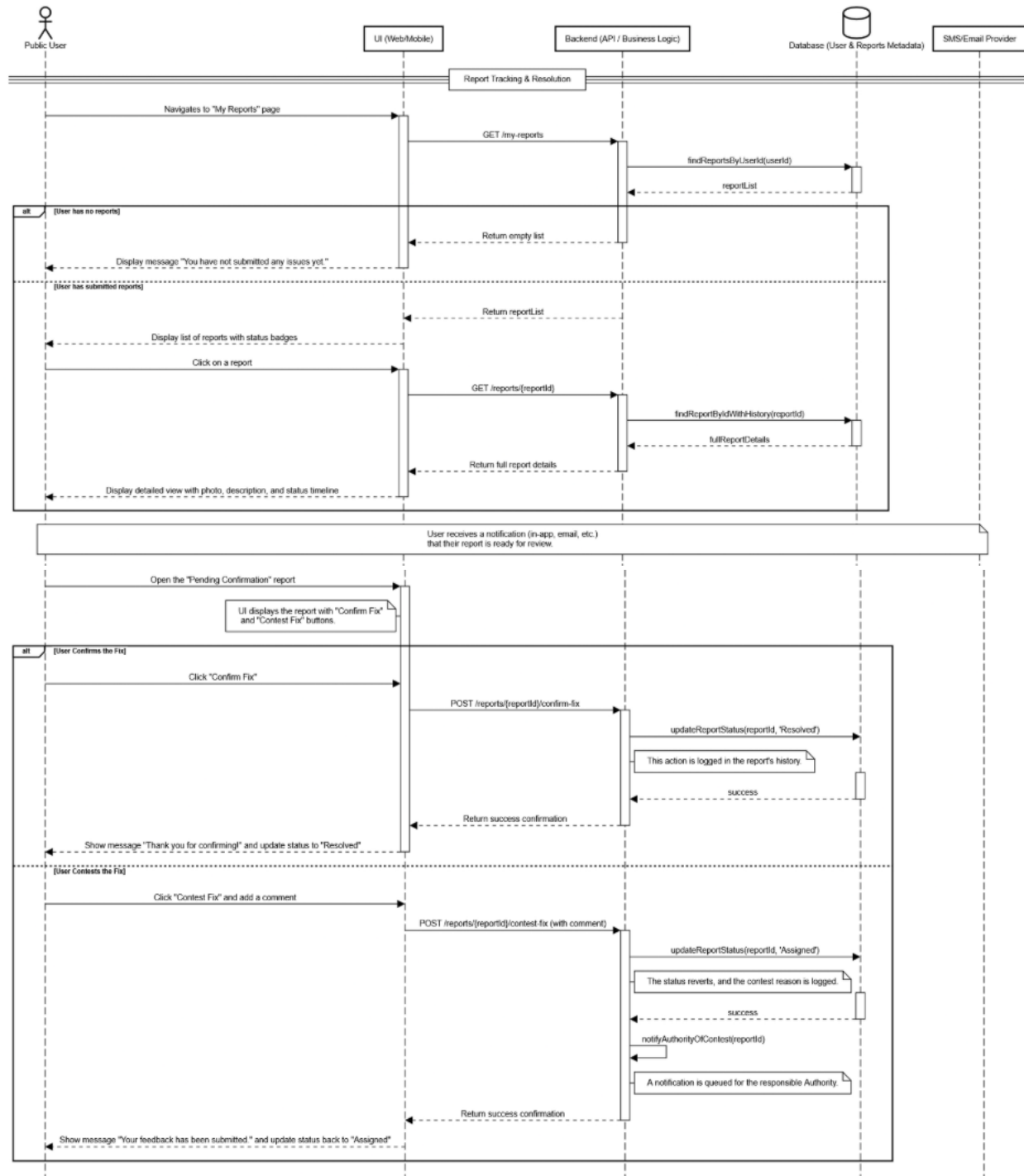
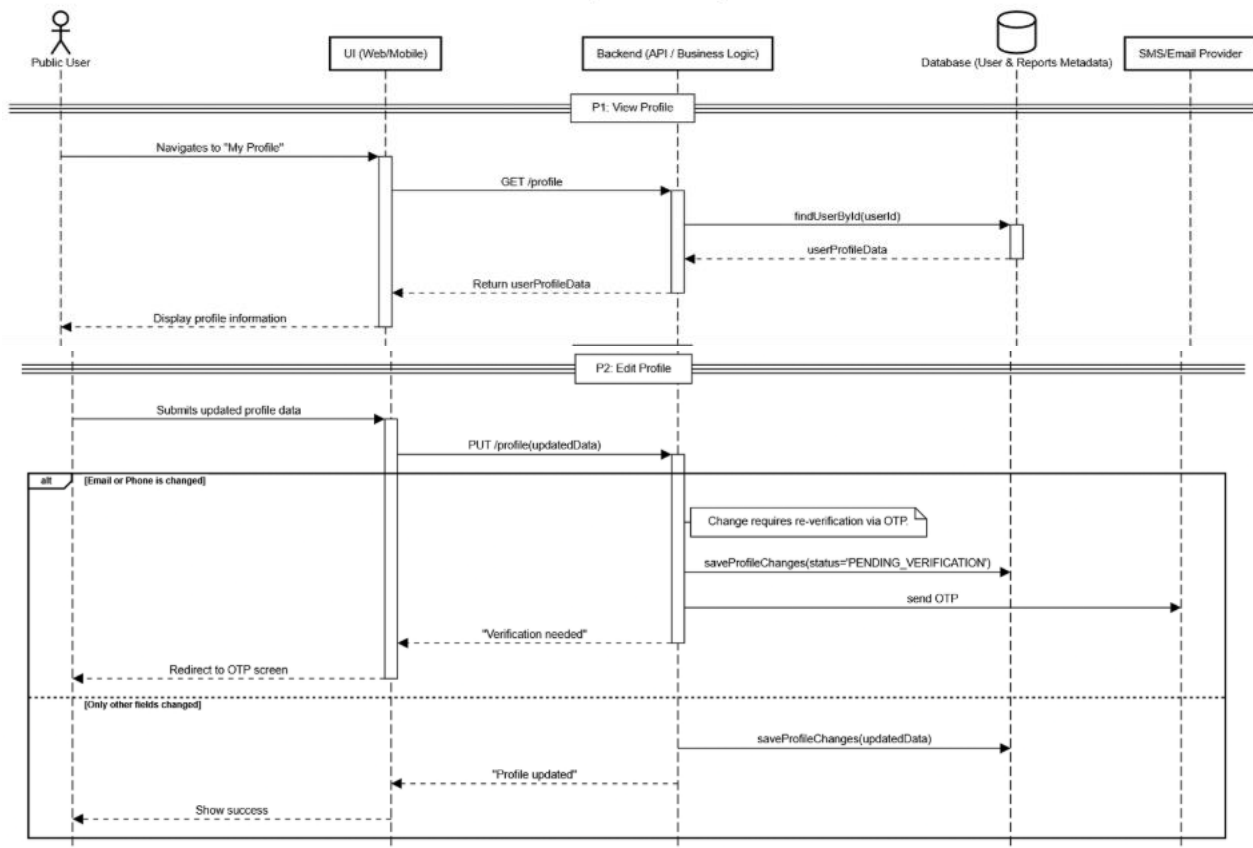


Figure 3.8: Public User Sequence Diagram: Profile Management (Part 1)

Figure 3.7: Public User Sequence Diagram: Tracking and Resolution

3.3.3.1.4 Public User Epic: Profile Management



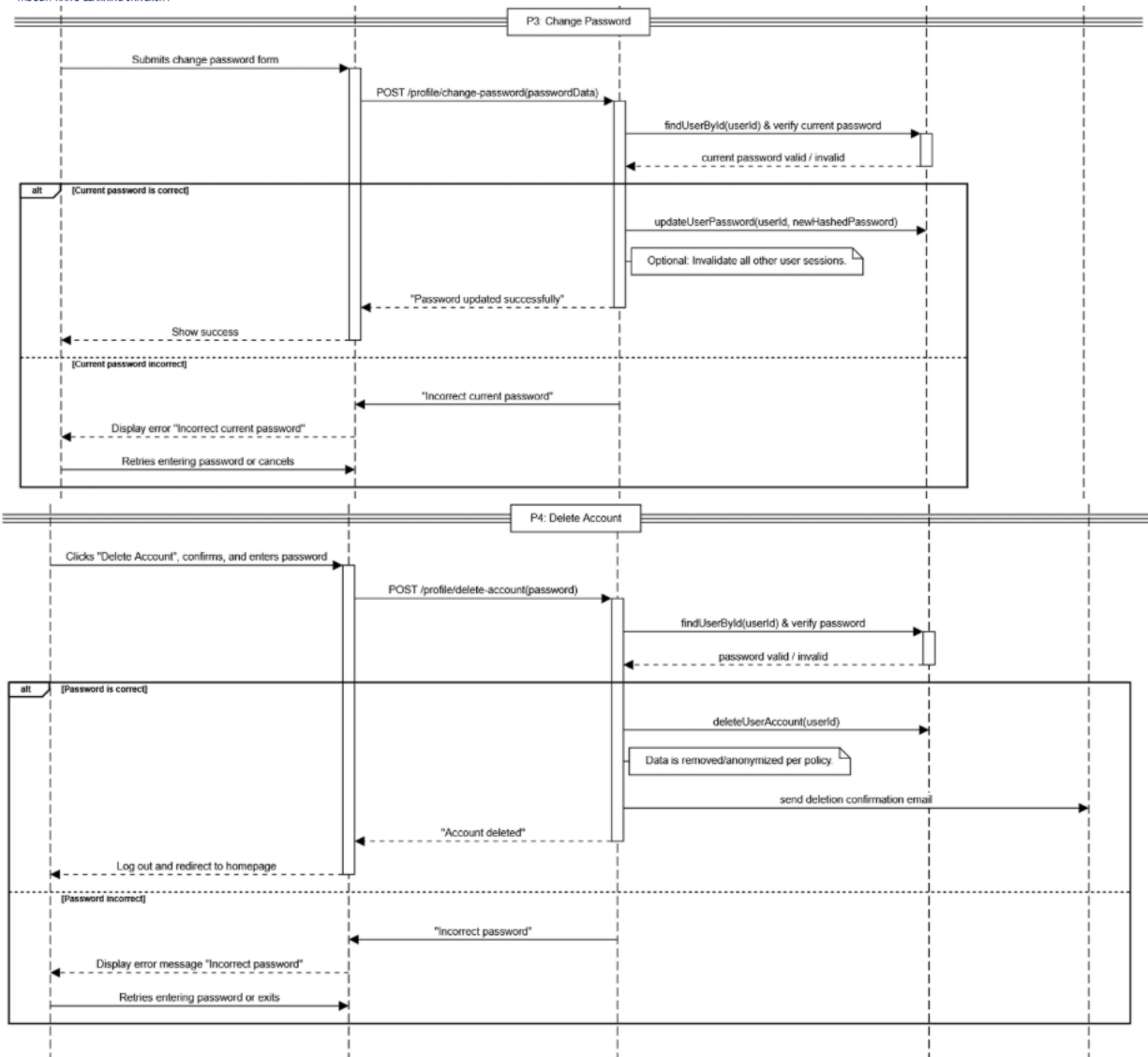


Figure 3.12: Public User Sequence Diagram: Profile Management (Part 5)

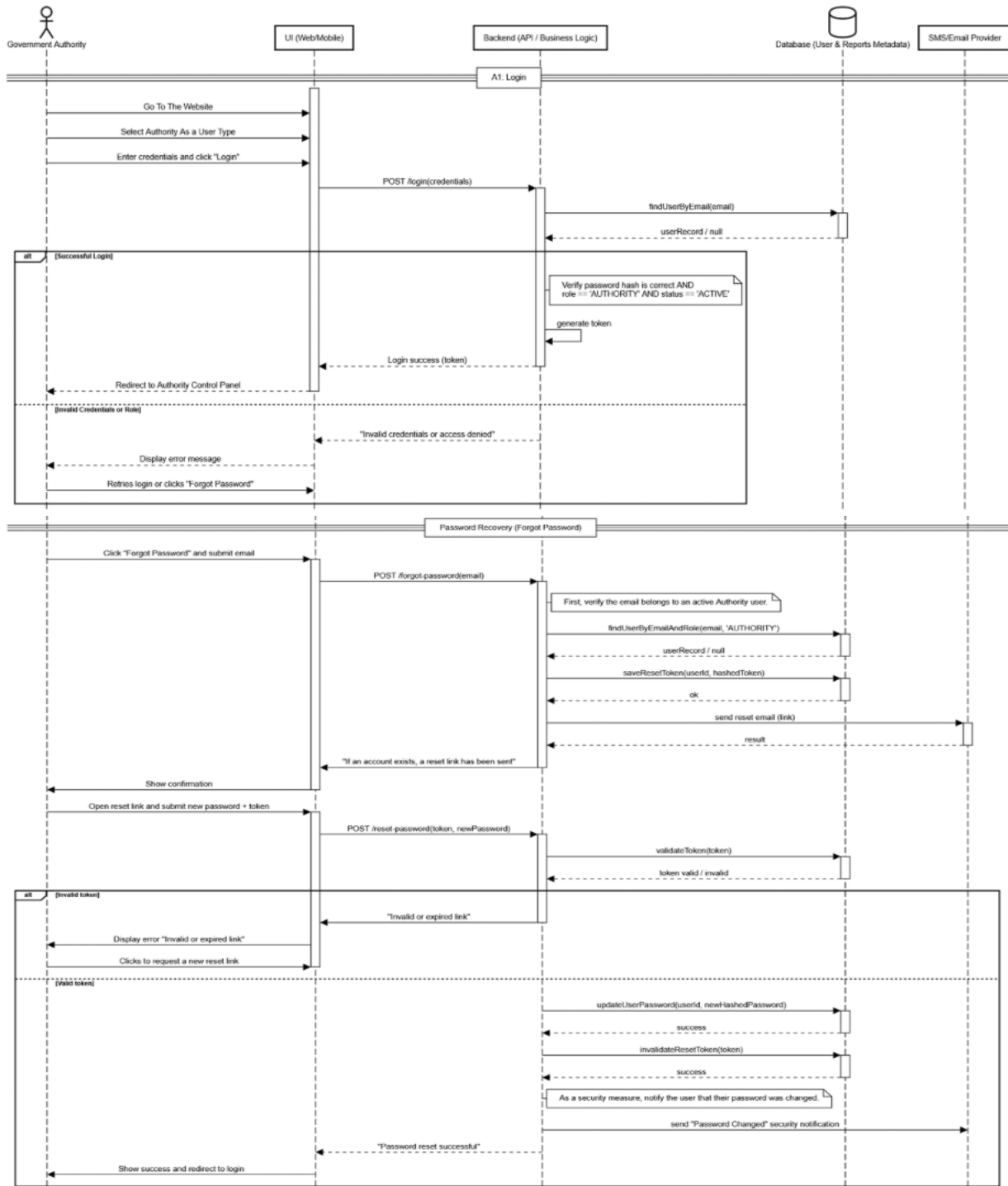
Figure 3.11: Public User Sequence Diagram: Profile Management (Part 4)

Figure 3.10: Public User Sequence Diagram: Profile Management (Part 3)

Figure 3.9: Public User Sequence Diagram: Profile Management (Part 2)

3.3.3.2 *Authority User Sequence diagram:*

3.3.3.2.1 Government Authority Epic: Authentication & Profile Management



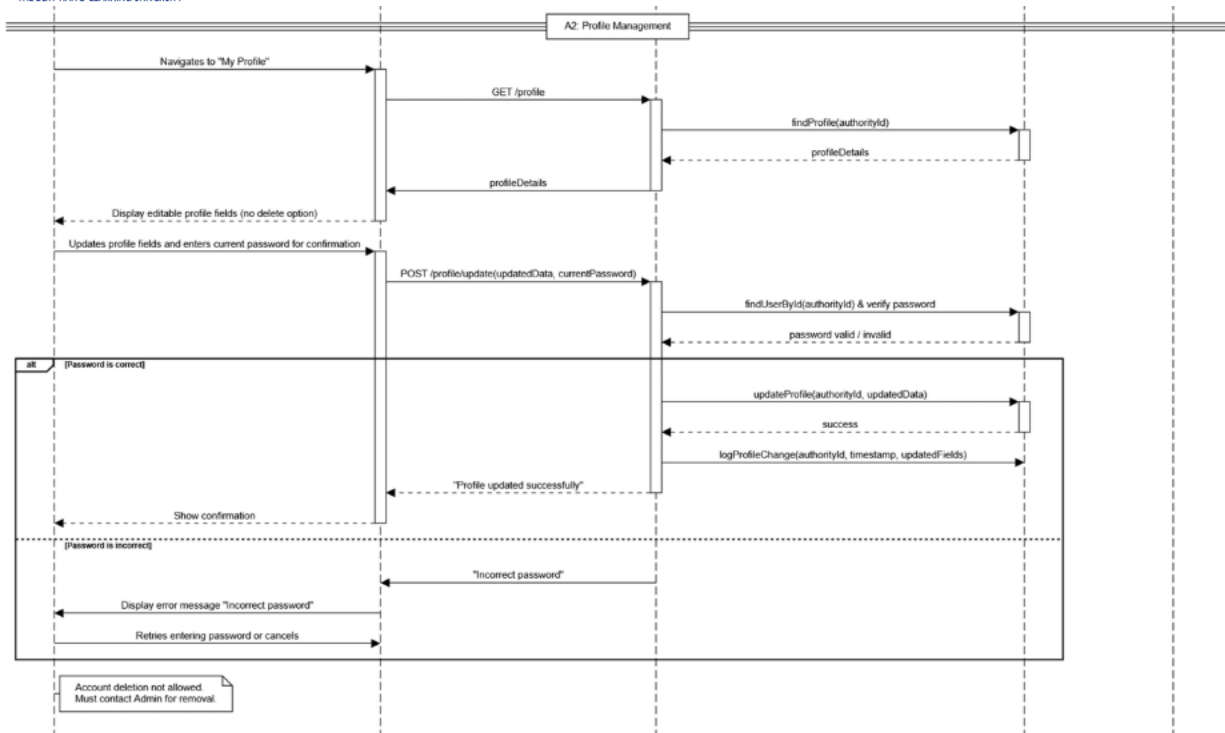


Figure 3.15: Authority Sequence Diagram: Authentication and Profile Management (Part 3)

Figure 3.14: Authority Sequence Diagram: Authentication and Profile Management (Part 2)

Figure 3.13: Authority Sequence Diagram: Authentication and Profile Management (Part 1)

3.3.3.2.2 Government Authority Epic: Report Management & Assignment

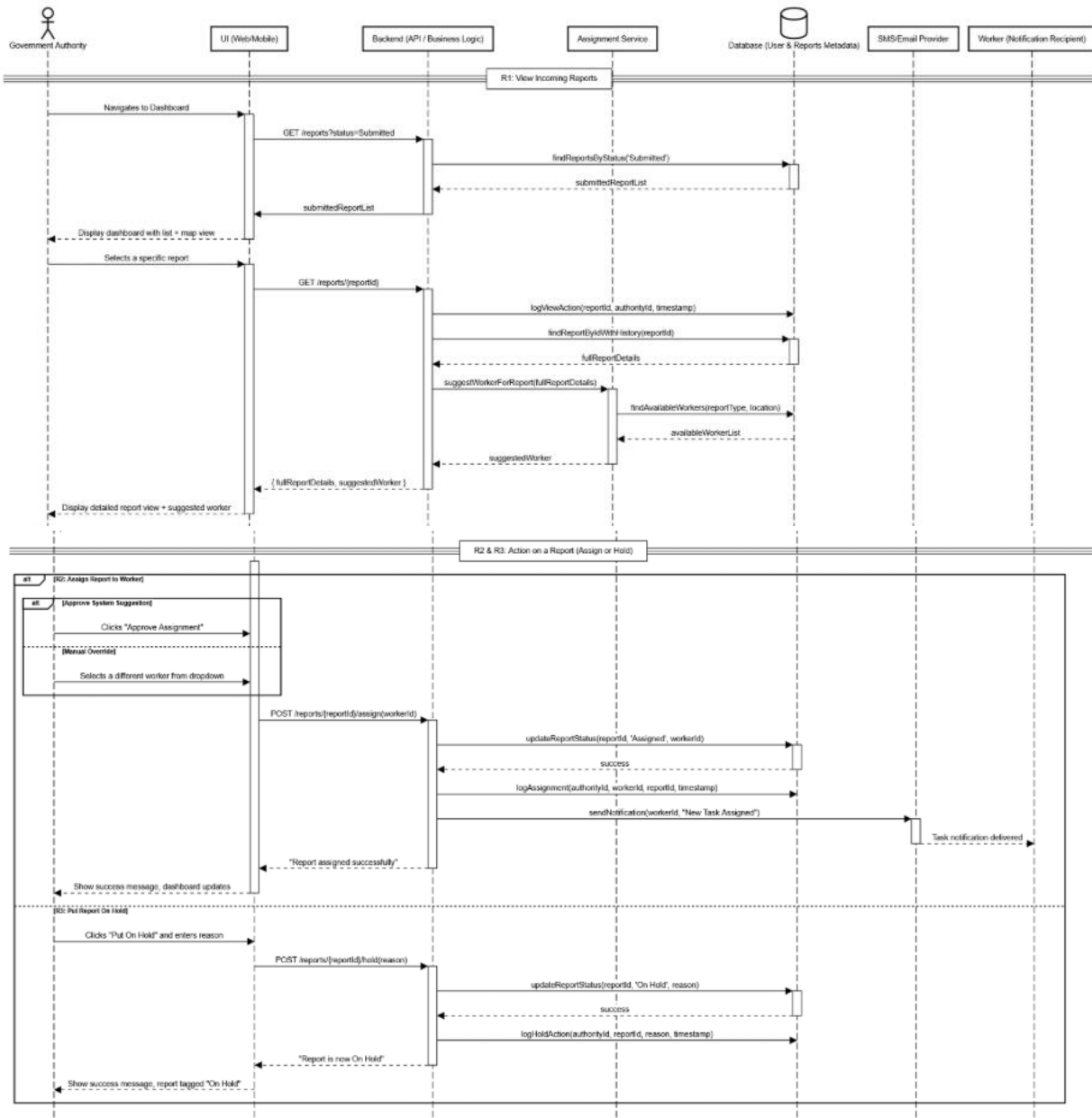
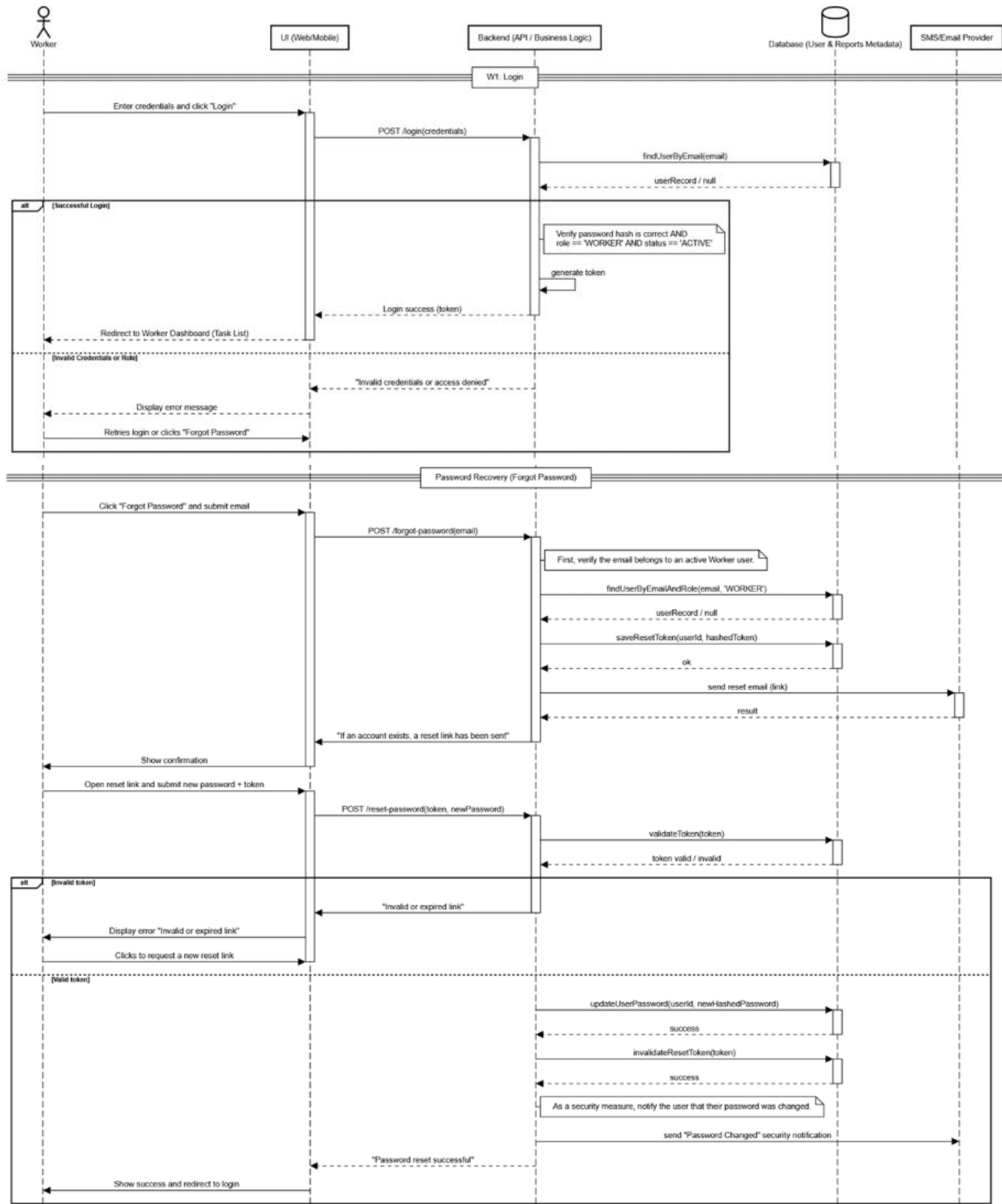


Figure 3.17: Authority Sequence Diagram: Report Management and Assignment (Part 2)

Figure 3.16: Authority Sequence Diagram: Report Management and Assignment (Part 1)

3.3.3.3 Worker User Sequence diagram:

3.3.3.3.1 Worker Epic: Authentication & Profile Management



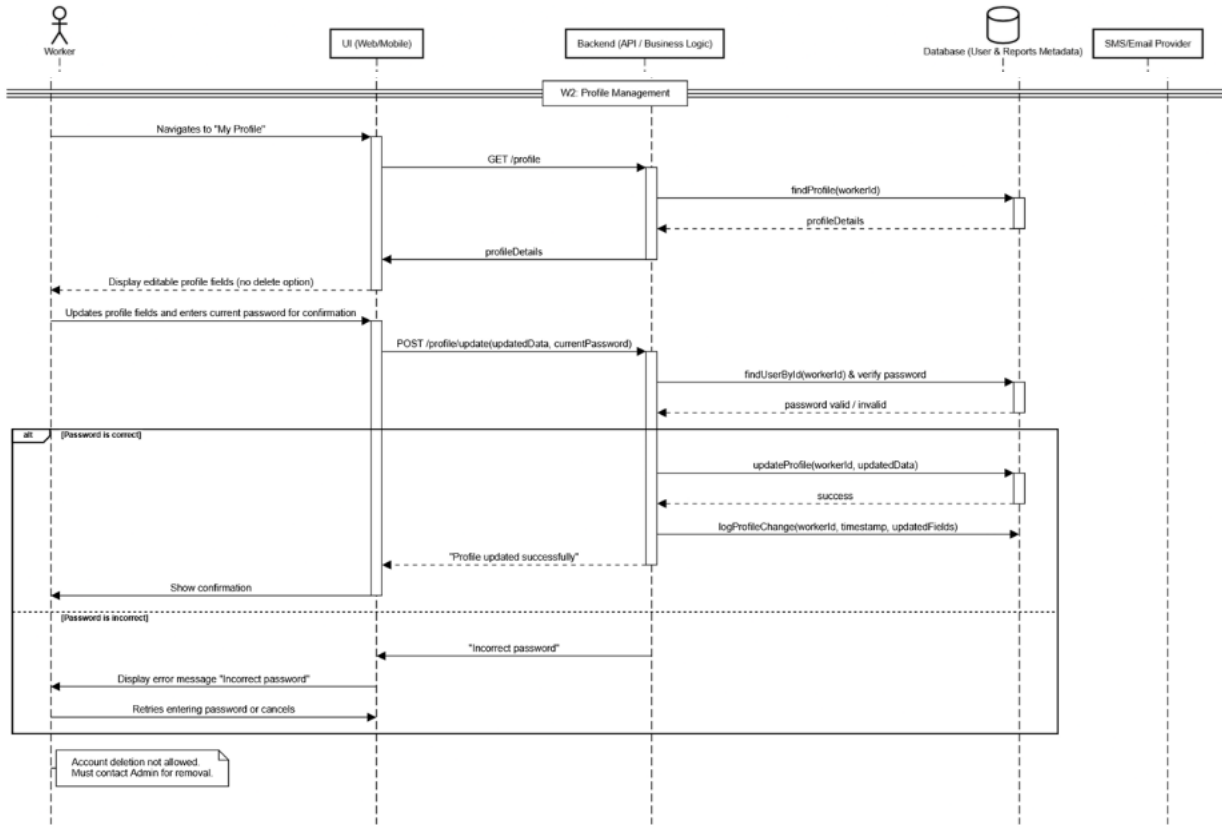


Figure 3.21: Worker Sequence Diagram: Authentication and Profile Management (Part 4)

Figure 3.20: Worker Sequence Diagram: Authentication and Profile Management (Part 3)

Figure 3.19: Worker Sequence Diagram: Authentication and Profile Management (Part 2)

Figure 3.18: Worker Sequence Diagram: Authentication and Profile Management (Part 1)

3.3.3.3.2 Worker Epic: Task Management

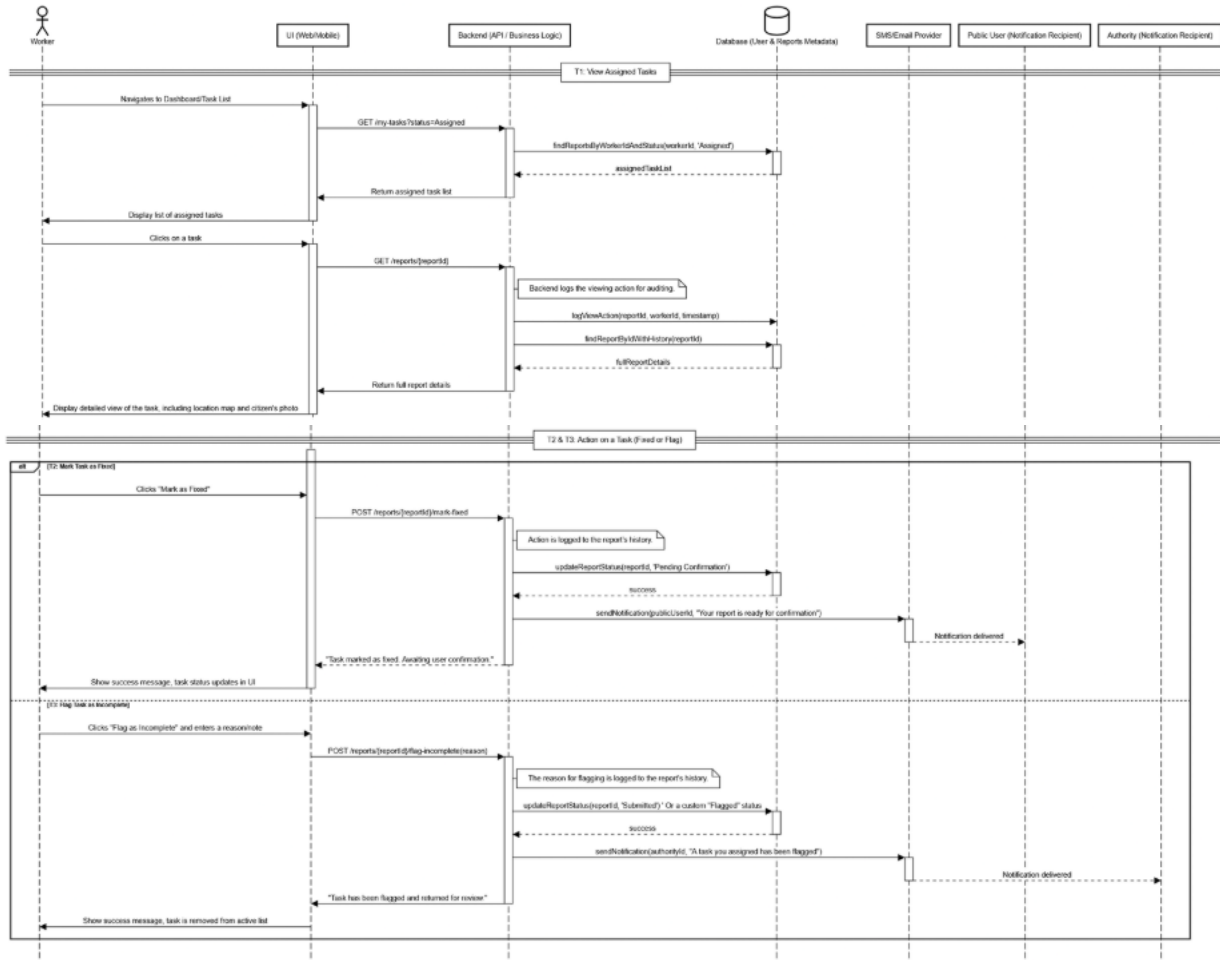


Figure 3.23: Worker Sequence Diagram: Task Management (Part 2)

Figure 3.22: Worker Sequence Diagram: Task Management (Part 1)

3.3.4 Entity Relationship Diagram:

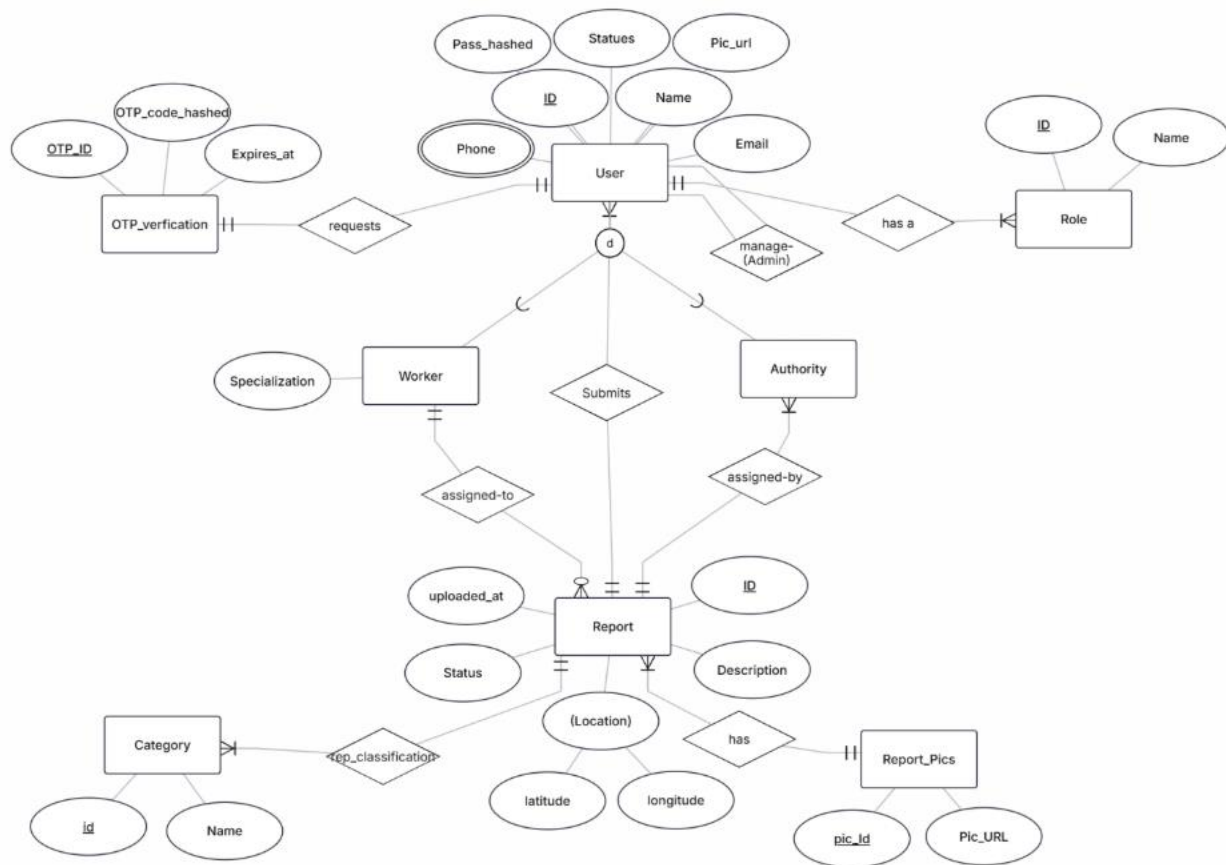


Figure 3.24: Entity Relationship Diagram (ERD)

3.3.5 Data Flow Diagram:

3.3.5.1 context Diagram (Level 0 DFD):

1. Context Diagram (Level 0 DFD)

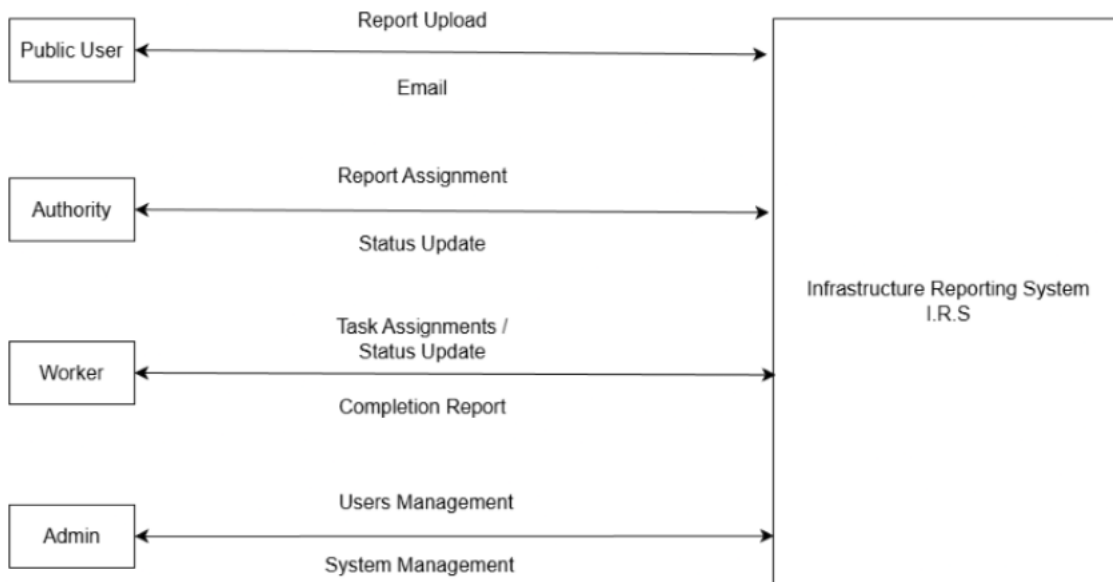


Figure 3.25: Context Diagram (Level 0 DFD)

3.3.5.2 Level 1 DFD:

2. Level 1 DFD:

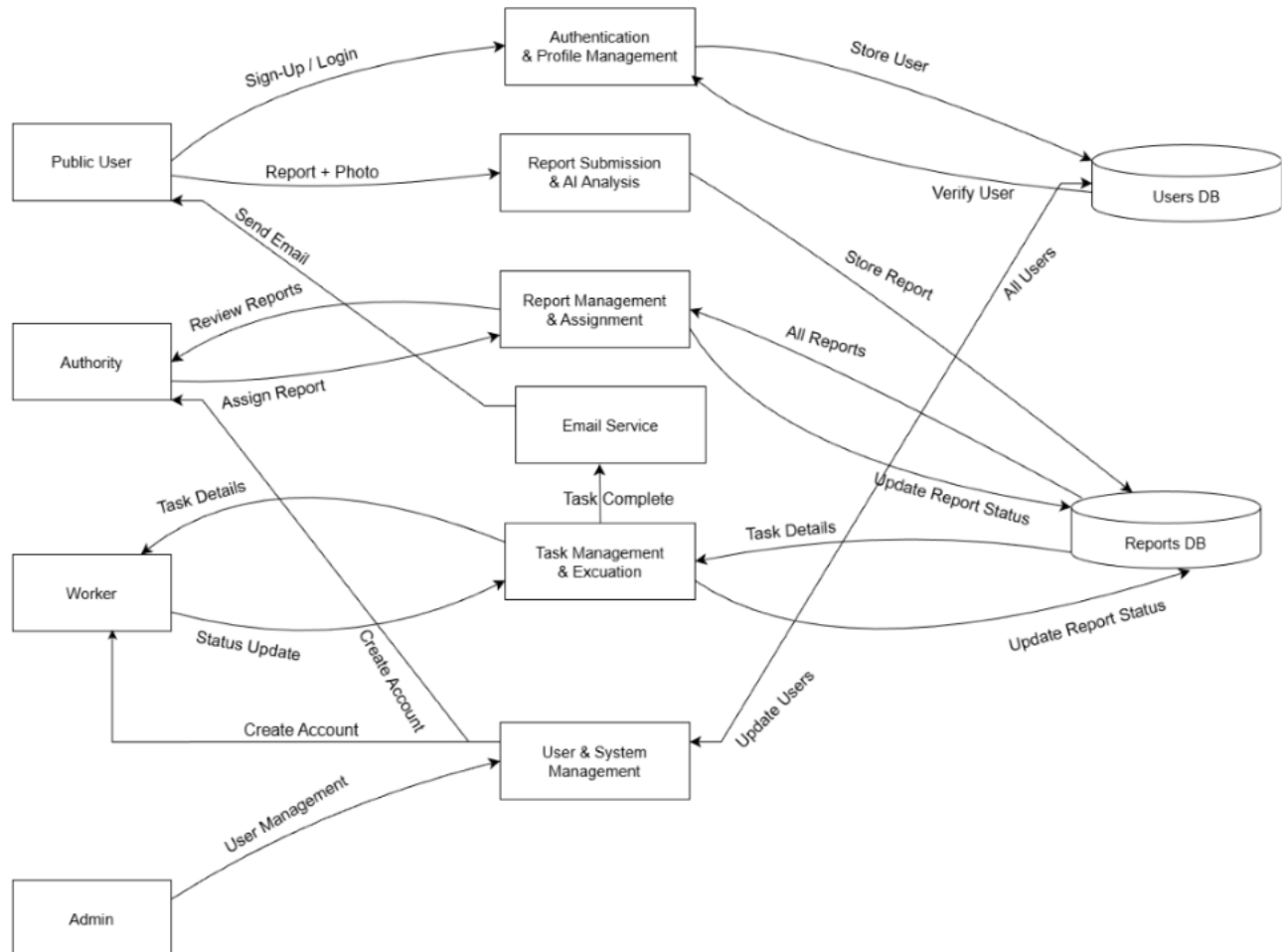


Figure 3.26: Level 1 Data Flow Diagram (DFD)

Chapter 4

Implementation

4.1 Front-End Implementation

4.1.1 Front-End Architecture Overview

The front-end of the Smart Infrastructure Reporting System was implemented as a Single Page Application (SPA) using Angular. The application is designed to provide an interactive, responsive, and user-friendly interface that enables different user roles to interact efficiently with the system.

The architecture follows a component-based design approach in which the application is divided into reusable components, services, and modules. This design improves maintainability, scalability, and code reusability while simplifying future enhancements and modifications.

Communication between the front-end and the back-end is achieved through RESTful APIs. Angular services act as an abstraction layer between user interface components and server-side endpoints, ensuring a clear separation of concerns and reducing code duplication throughout the application.

The system supports multiple user roles, including Public Users, Authorities, Workers, and Administrators. Each role is provided with dedicated interfaces and functionalities according to its responsibilities within the infrastructure reporting workflow.

4.1.2 Front-End Project Structure

The Angular application is organized using a feature-based structure to improve maintainability and scalability. Each major system functionality is grouped into a dedicated module containing its associated components, services, models, and routing configuration.

The project structure consists of the following major areas:

- Authentication Module
- Public User Module

- Authority Module
- Worker Module
- Administrator Module
- Profile Management Module
- Shared Components Module
- Core Services Module

This organization allows each feature to be developed and maintained independently while maintaining consistency across the entire application.

4.1.3 Technologies and Development Tools

The front-end implementation utilizes several modern web technologies and development tools to ensure performance, maintainability, and scalability.

Angular:

Angular serves as the primary framework for developing the web application. It provides component-based development, dependency injection, routing, and state management capabilities.

TypeScript:

TypeScript is used as the primary programming language. Its static typing features improve code quality, maintainability, and error detection during development.

HTML5 and CSS3:

HTML5 is used to structure the application pages, while CSS3 is used to create responsive and visually appealing user interfaces.

Angular Router:

Angular Router manages navigation between pages and supports role-based access control through route guards.

Angular HTTP Client:

The HTTP Client module is used to communicate with the RESTful API and exchange data between the client and server.

Git and GitHub are used for source control, collaboration, version tracking, and project management

The combination of these technologies enabled the development of a modern web application capable of supporting all functional requirements of the system.

4.1.4 Routing and Navigation System

Navigation within the application is managed using Angular Router. The routing system enables seamless navigation between different pages without requiring full page reloads, providing a smooth user experience.

Role-based routing was implemented to ensure that users can only access pages relevant to their assigned roles. Route Guards are used to protect restricted routes and prevent unauthorized access to sensitive functionalities.

Authentication status is verified before accessing protected pages, and unauthorized users are automatically redirected to the login page.

4.1.5 Front-End Modules

The application consists of several functional modules that correspond to the different user roles and system requirements.

4.1.5.1 Authentication Module

The Authentication Module manages user registration, login, OTP verification, password recovery, and session management. It communicates directly with the back-end authentication services and handles JWT token storage and validation.

4.1.5.2 Public User Module

This module allows citizens to submit infrastructure reports, upload supporting images, view nearby reports, track report status, and manage their personal information.

4.1.5.3 Authority Module

The Authority Module enables authorities to review incoming reports, assign workers, monitor progress, and supervise issue resolution activities.

4.1.5.4 Worker Module

The Worker Module provides workers with access to assigned tasks, task history, progress updates, and report completion functionalities.

4.1.5.5 Administrator Module

The Administrator Module provides administrative functionalities including user management, account creation, audit log monitoring, and system supervision.

4.1.5.6 Profile Management Module

The Profile Management Module allows users to view and update personal information, manage account settings, and maintain profile details.

4.1.5.7 Shared Components and Services

Several reusable components and services were implemented to maintain consistency across the application and reduce duplicated code. These shared resources include navigation components, API communication services, and common user interface elements.

4.1.6 API Integration

The front-end communicates with the ASP.NET Core back-end through RESTful APIs using Angular HTTP Client.

Dedicated Angular services were developed to handle API requests and responses. These services encapsulate communication logic and provide reusable methods for retrieving and updating data.

JWT authentication tokens are automatically attached to protected requests using HTTP Interceptors, ensuring secure communication between the client and server.

Error handling mechanisms were also implemented to process API errors and provide meaningful feedback to users.

4.1.7 Challenges Faced and Solutions

During the development of the front-end application, several technical challenges were encountered and successfully resolved.

Challenge 1: Managing Multiple User Roles

The system supports Public Users, Authorities, Workers, and Administrators, each requiring different interfaces and permissions.

Solution:

Role-based routing and access control mechanisms were implemented to ensure that each user can only access the features associated with their assigned role.

Challenge 2: Integration with the Back-End API

Ensuring reliable communication between Angular and ASP.NET Core required handling authentication tokens, API responses, and error scenarios.

Solution:

Angular HTTP Client and HTTP Interceptors were implemented to manage API communication and automatically attach JWT tokens to protected requests.

Challenge 3: Form Validation and Data Accuracy

The application contains numerous forms such as registration, login, profile management, and report submission forms.

Solution:

Reactive Forms were utilized to implement client-side validation, improve data quality, and provide immediate user feedback.

Challenge 4: Responsive User Interface

The system needed to provide a consistent experience across different screen sizes and devices.

Solution:

Responsive design principles were applied using flexible layouts and adaptive styling techniques to ensure compatibility with desktops, tablets, and mobile devices.

Challenge 5: Navigation and Session Management

Managing navigation while maintaining secure user sessions introduced additional complexity.

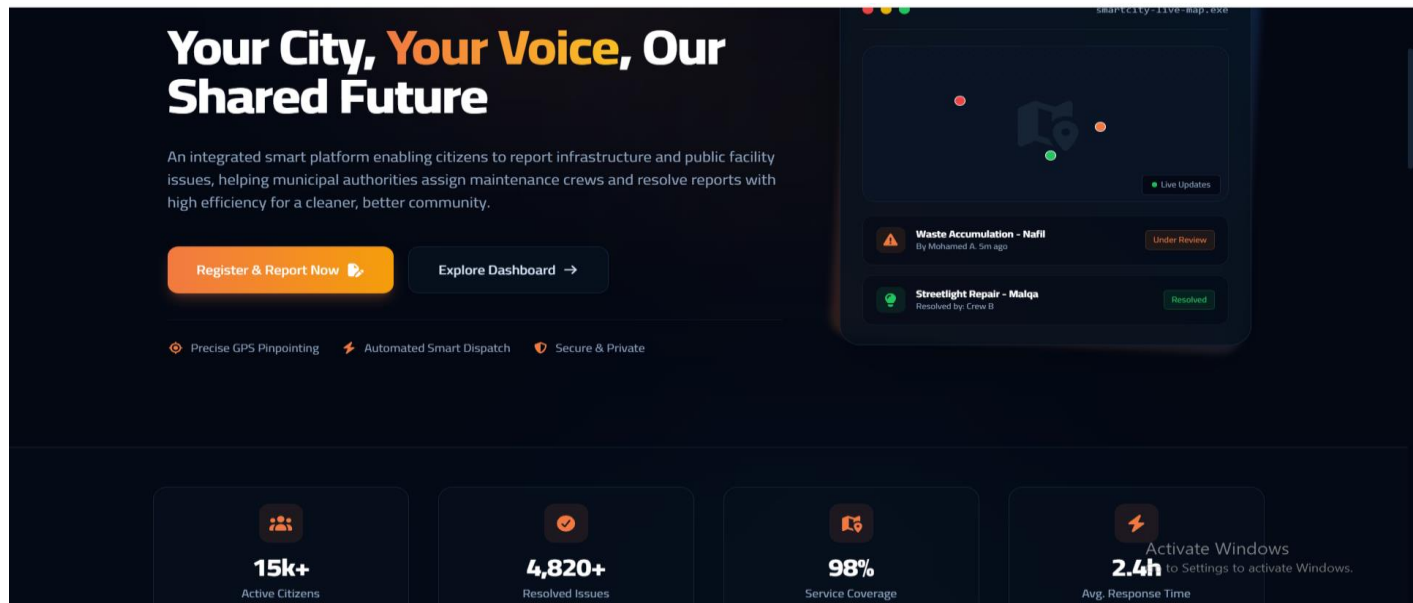
Solution:

Angular Router and Route Guards were used to control access to protected pages and maintain smooth navigation throughout the application.

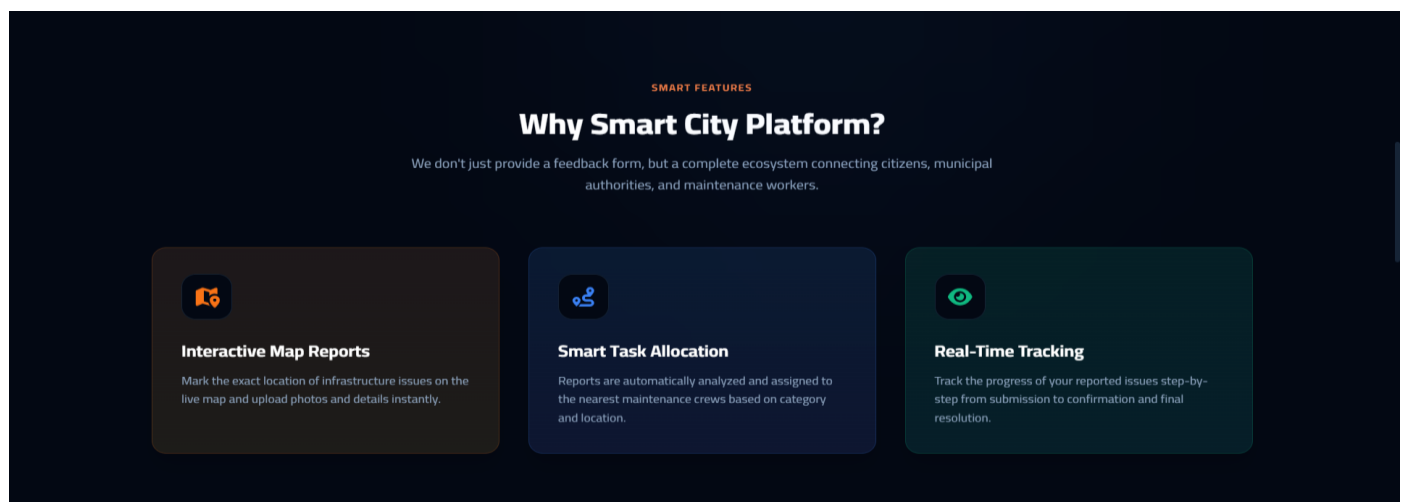
4.1.8 User Interface Implementation

This section presents the implemented user interfaces of the system. Screenshots of the developed pages are provided along with descriptions of their functionalities and interactions.

The landing page:

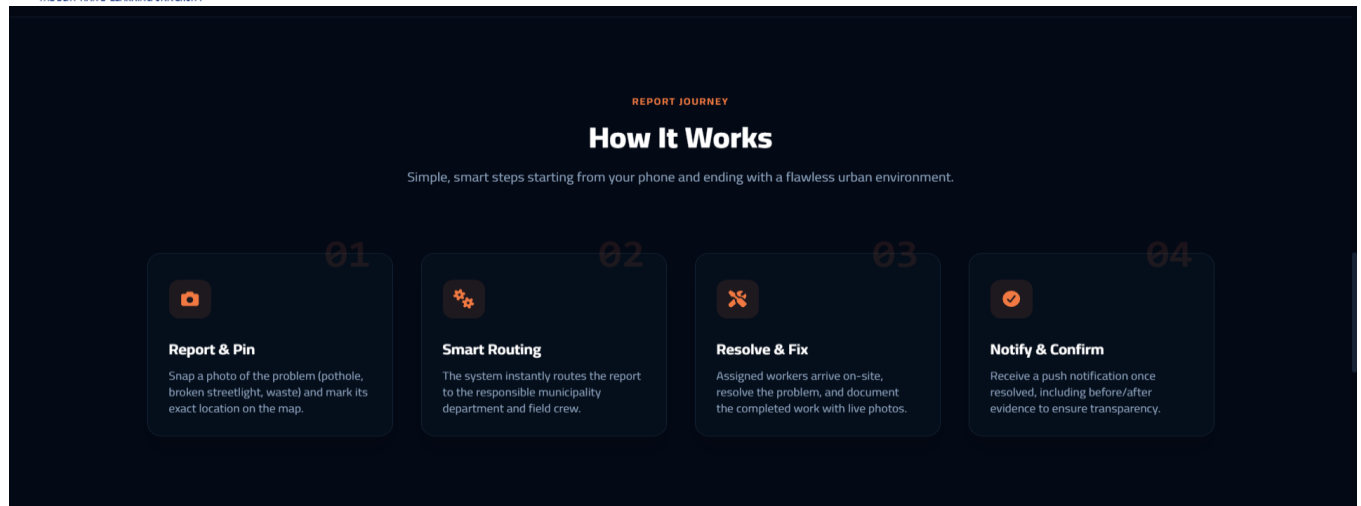


The landing page provides an overview of the Smart Infrastructure Reporting System, presenting its objectives, key functionalities, and navigation options to facilitate user engagement and access to system services.



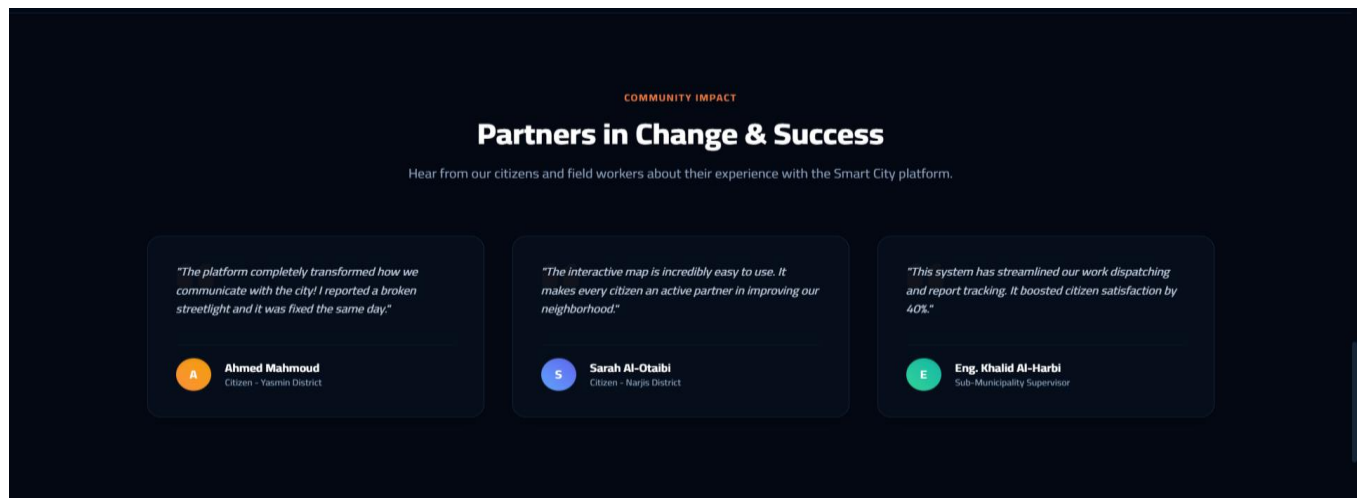
Smart Features Section:

This section presents the key functionalities of the platform that support efficient infrastructure issue reporting and management.



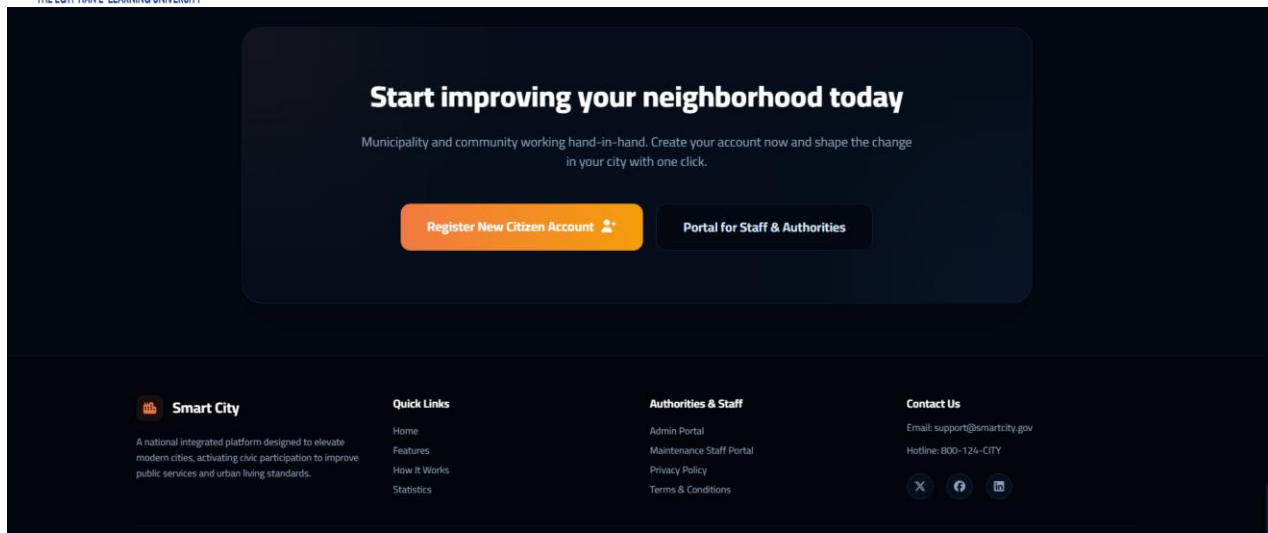
How It Works Section:

This section demonstrates the main steps involved in reporting, processing, and resolving infrastructure issues.



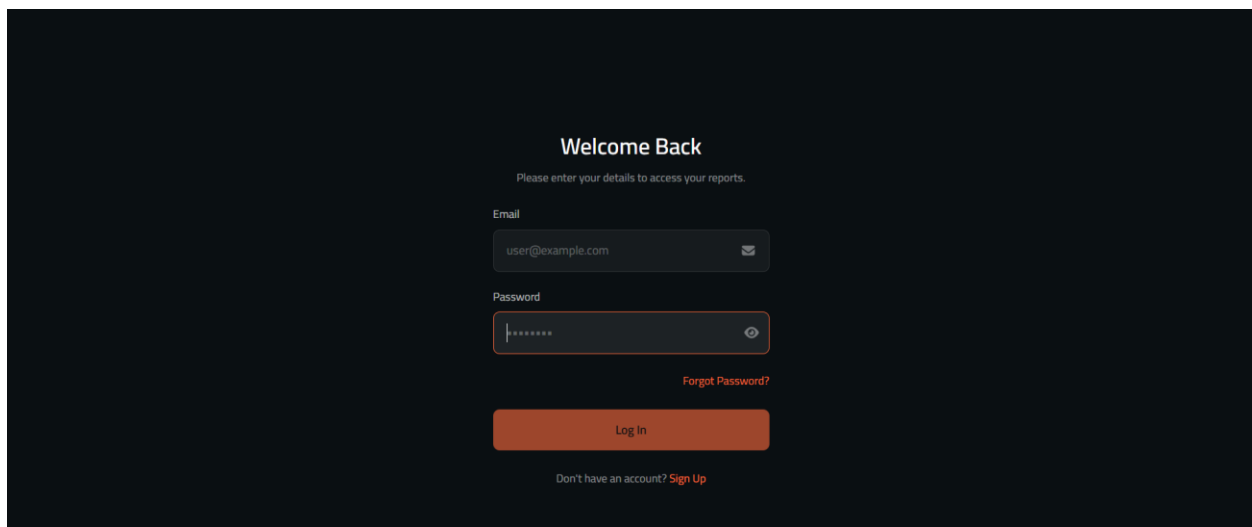
Partners in Change & Success Section:

This section reflects the real-world value of the platform by showcasing feedback from citizens and field workers, highlighting how the system fosters active community engagement and improves operational efficiency.



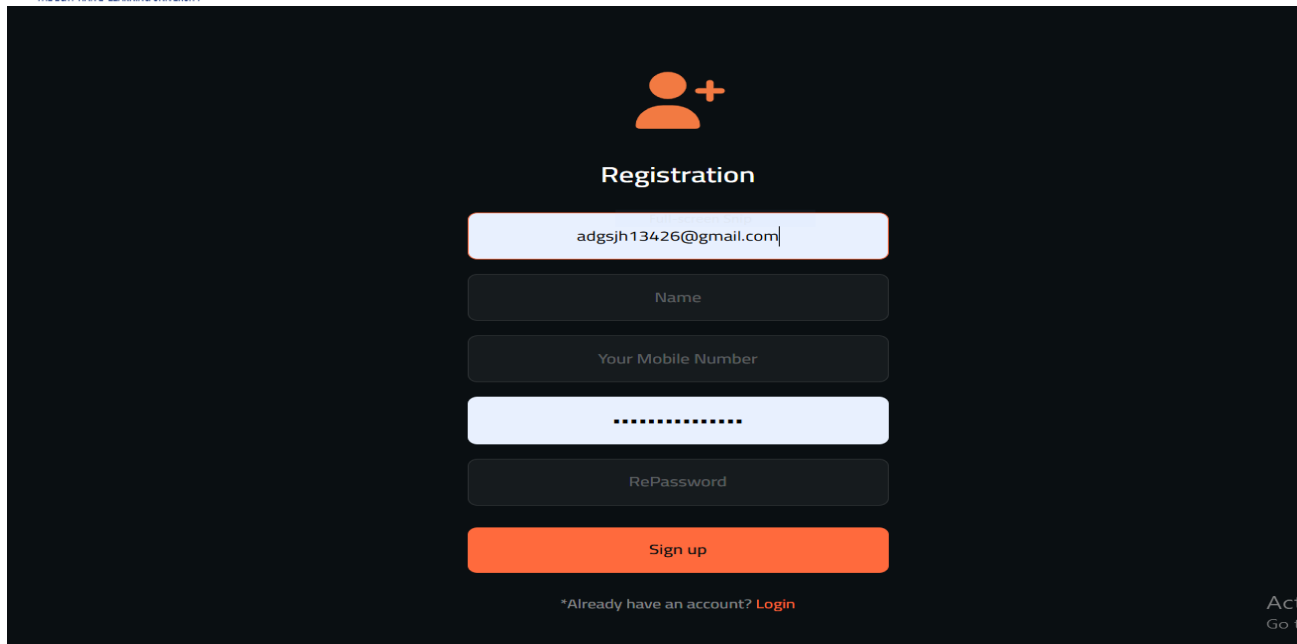
Empowering Community Action Section:

This interface serves as the gateway to the system, facilitating secure access for various roles—including Public Users, Authorities, and Workers—to initiate their participation in the infrastructure reporting workflow.



Authentication: Login Portal

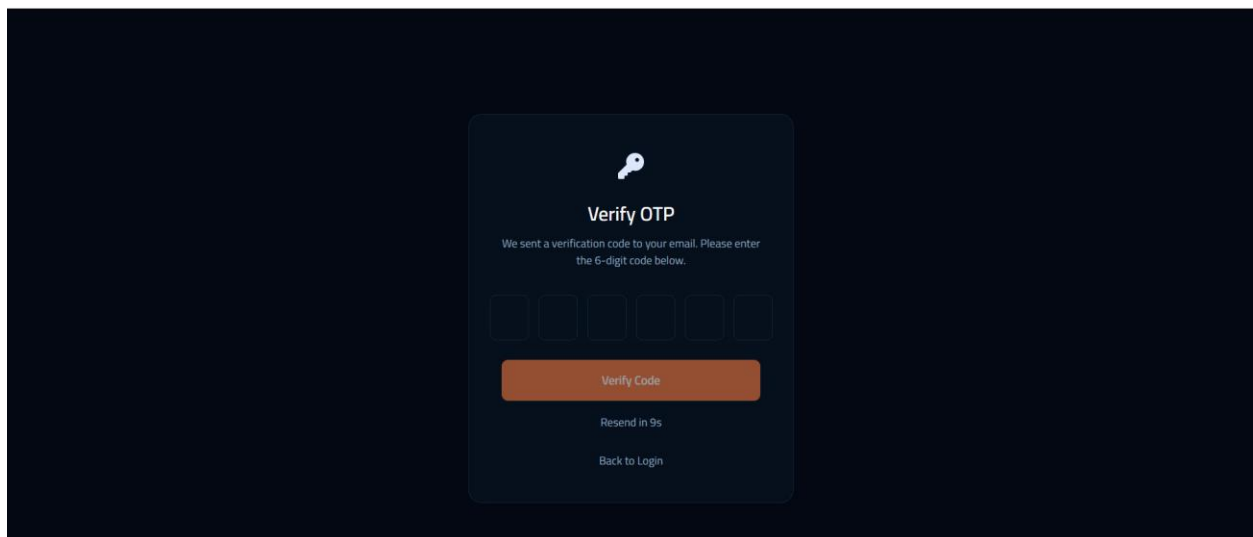
This module provides a secure authentication interface, enabling authorized users to access their respective dashboards and manage reporting tasks efficiently.



The registration form is centered on a dark background. At the top is an orange icon of a person with a plus sign. Below it is the title "Registration". The form consists of several input fields: an email field containing "adgsjh13426@gmail.com", a "Name" field, a "Your Mobile Number" field, a password field with dots, and a "RePassword" field. A prominent orange "Sign up" button is at the bottom of the form. Below the button is a link: "*Already have an account? [Login](#)".

Authentication: User Registration Portal

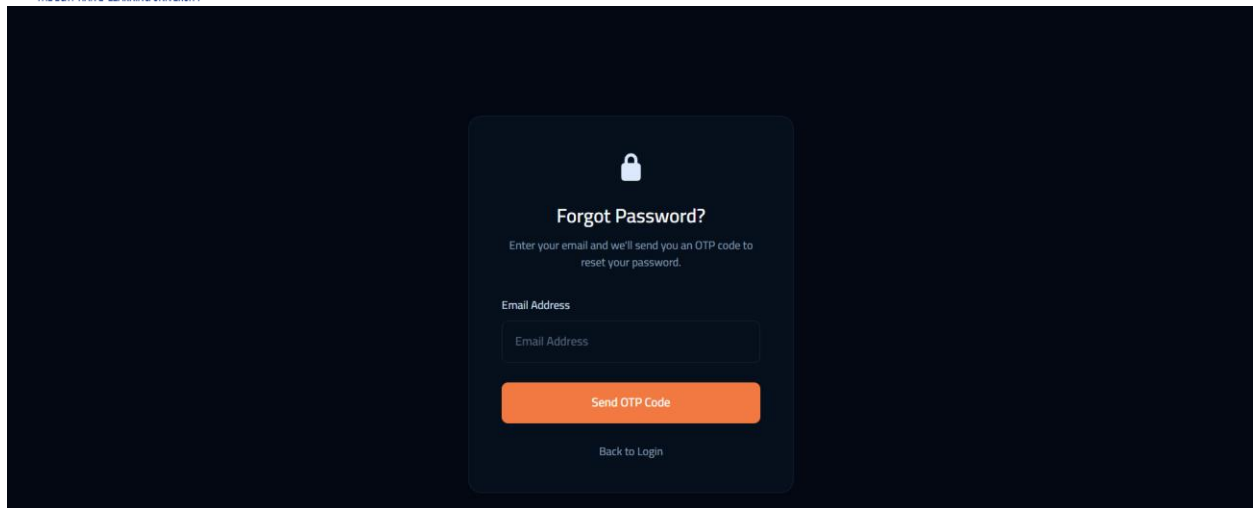
This component provides a streamlined interface for new users to register within the system, ensuring secure data capture and integration with the backend authentication services to establish new user roles.



The OTP verification screen is a dark modal box. At the top is a key icon and the title "Verify OTP". Below the title is a message: "We sent a verification code to your email. Please enter the 6-digit code below." There are six empty square boxes for entering the digits. Below these is an orange "Verify Code" button. At the bottom are two links: "Resend in 9s" and "Back to Login".

Authentication: OTP Verification Module

This interface facilitates the final stage of secure account validation, where users verify their identity by entering the one-time code sent to their registered contact, ensuring data integrity and preventing unauthorized account creation, then return to login page.



Forgot Password?

Enter your email and we'll send you an OTP code to reset your password.

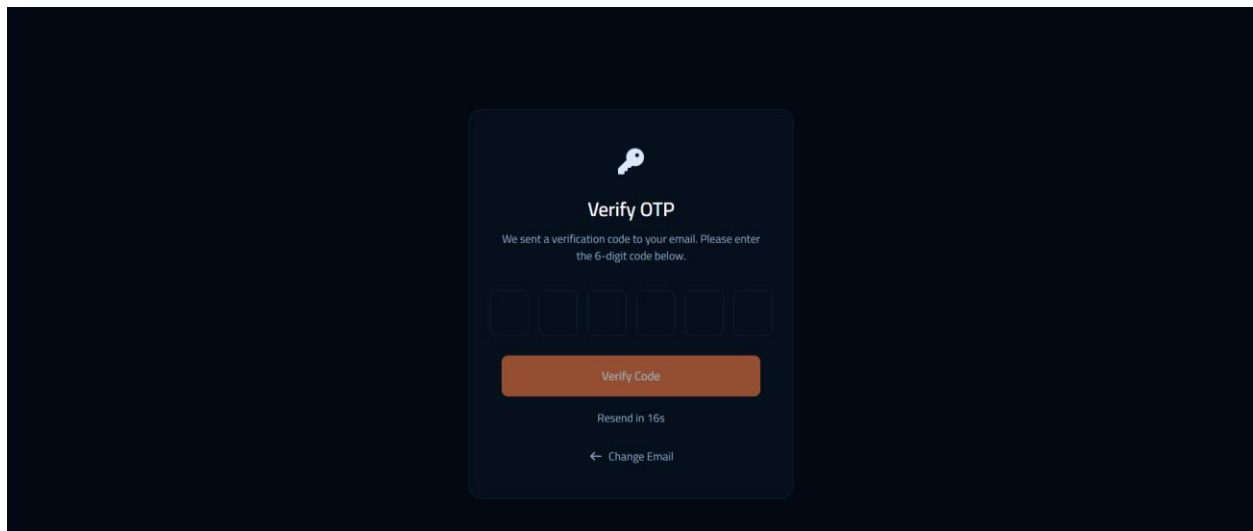
Email Address

Send OTP Code

[Back to Login](#)

uthentication: Password Recovery Module

This feature ensures system security by providing a reliable mechanism for password recovery via OTP (One-Time Password) verification, integrated directly with the backend authentication services to maintain secure user access control



Verify OTP

We sent a verification code to your email. Please enter the 6-digit code below.

Verify Code

Resend in 16s

[← Change Email](#)

Authentication: Password Reset Verification

This module validates the user's request to reset their credentials by verifying the unique OTP code, ensuring that the password update process is only accessible to the authorized account owner.

Create New Password

Enter your email, OTP code, and new password

Email

OTP Code

New Password

Strength: **Good**

- ✓ At least 8 characters
- ✓ Contains a number
- ✓ Contains a special character

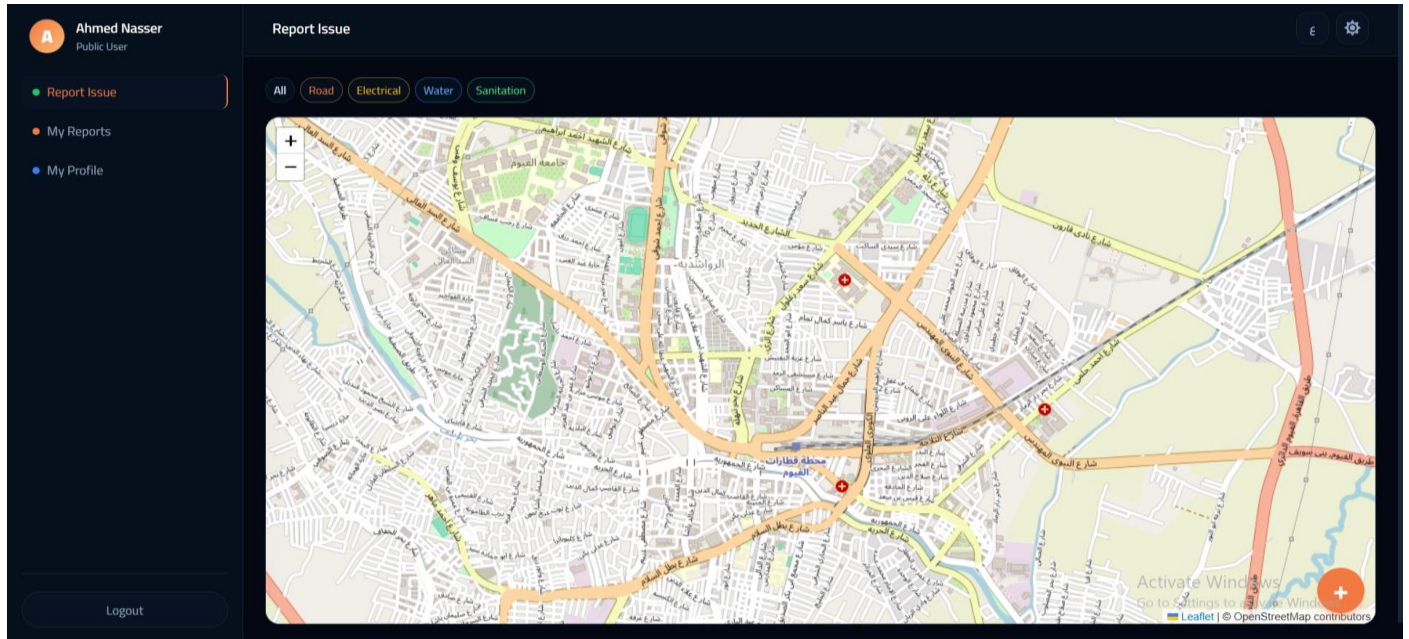
Confirm Password

[← Back to login](#)

Authentication: Password Reset Completion

This interface allows users to finalize the recovery process by defining a new, secure password, which is then validated and updated within the system's authentication backend to restore full account access, then return login page.

Public User Dashboard: Issue Reporting Interface:



Public User Dashboard: Issue Reporting Interface

This primary dashboard serves as the central hub for public users, providing an interactive map-based interface to report and track infrastructure issues, thereby facilitating direct community engagement with the system.

A Ahmed Nasser
Public User

Report Issue

My Reports

My Profile

Logout

Report Issue

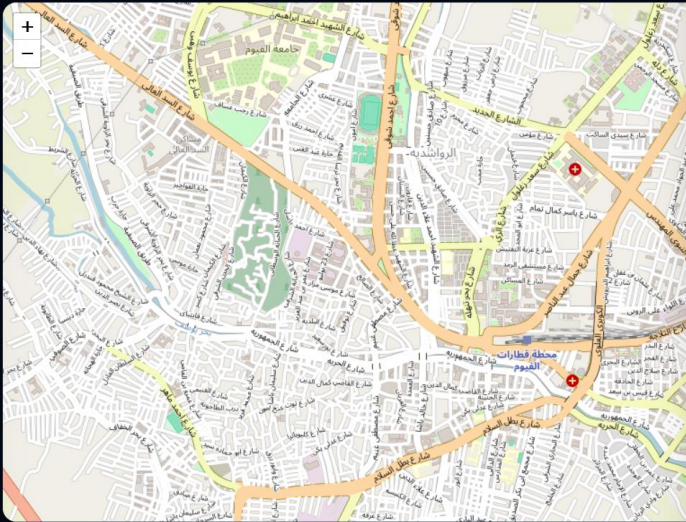
All

Road

Electrical

Water

Sanitation



SUBMIT NEW REPORT

Evidence Photos (Max 3)



+

+

Category

Road Maintenance

AI Confidence: 81%

Description

```
{
  "image_description": "A narrow asphalt or concrete path surrounded by grassy areas, showing severe surface degradation and structural failure.",
  "main_problem": "Severe pavement failure, including deep potholes, extensive cracking, and collapse",
}
```

Location

el fayoum

Activate Windows
Go to Settings to activate Windows.

AI-Powered Reporting: Automated Issue Analysis

This feature integrates AI capabilities to automatically analyze submitted evidence photos, generating detailed descriptions and categorizing infrastructure issues to enhance the accuracy and efficiency of the reporting workflow.

A Ahmed Nasser
Public User

Report Issue

My Reports

My Profile

My Reports

83412

WAITING CONFIRMATION

Road Maintenance

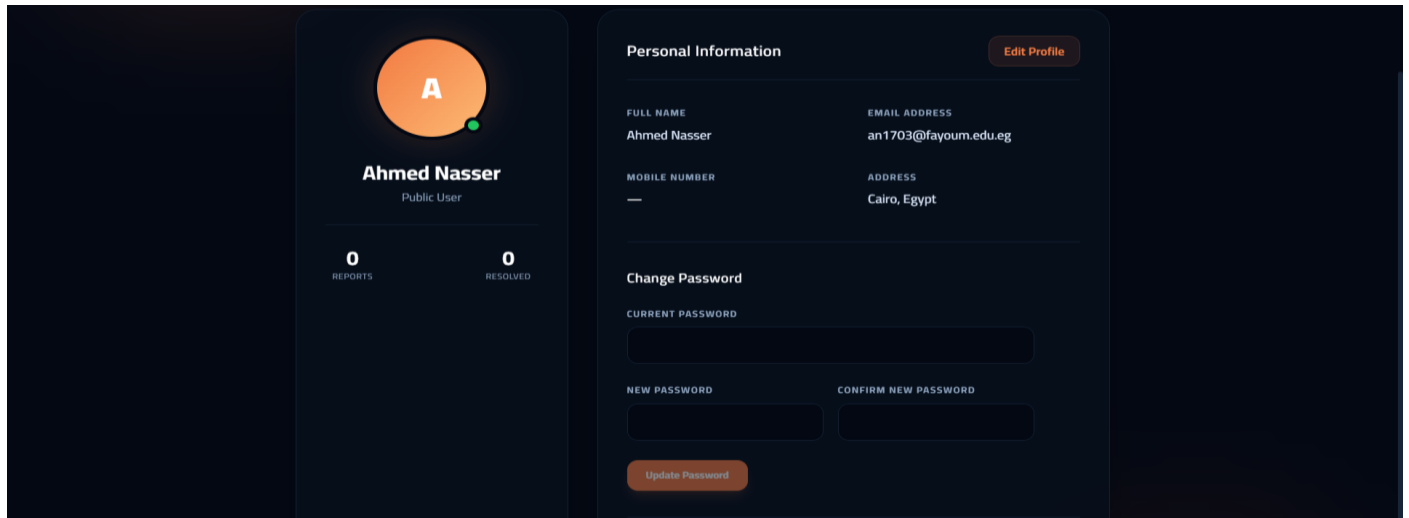
```
{
  "image_description": "A narrow asphalt or concrete path surrounded by grassy areas, showing severe surface degradation and structural failure.",
  "main_problem": "Severe pavement failure, including deep potholes, extensive cracking, and collapse of the road surface.",
  "possible_cause": "Subgrade erosion, water damage, ground subsidence, or lack of maintenance over time.",
  "severity_level": "High",
  "supporting_details": [
    "A large, deep trench-like pothole spanning across the path, exposing the underlying gravel and dirt sub-base.",
    "Widespread alligator cracking and fragmented pieces of pavement adjacent to the main collapse zone."
  ],
  "confidence": "High"
}
```

EL FAYOUM



User Dashboard: Report Tracking Interface

This interface provides users with a comprehensive overview of their submitted infrastructure reports, enabling them to monitor the real-time status and progress of their requests within the system's maintenance workflow.



The interface is a dark-themed user profile management page. On the left, a sidebar contains a profile card for 'Ahmed Nasser' (Public User) with an orange circular avatar containing the letter 'A'. Below the name are two statistics: '0 REPORTS' and '0 RESOLVED'. The main content area is divided into two sections. The top section, 'Personal Information', includes an 'Edit Profile' button and fields for 'FULL NAME' (Ahmed Nasser), 'EMAIL ADDRESS' (an1703@fayoum.edu.eg), 'MOBILE NUMBER' (a redacted field), and 'ADDRESS' (Cairo, Egypt). The bottom section, 'Change Password', contains fields for 'CURRENT PASSWORD', 'NEW PASSWORD', and 'CONFIRM NEW PASSWORD', followed by an 'Update Password' button.

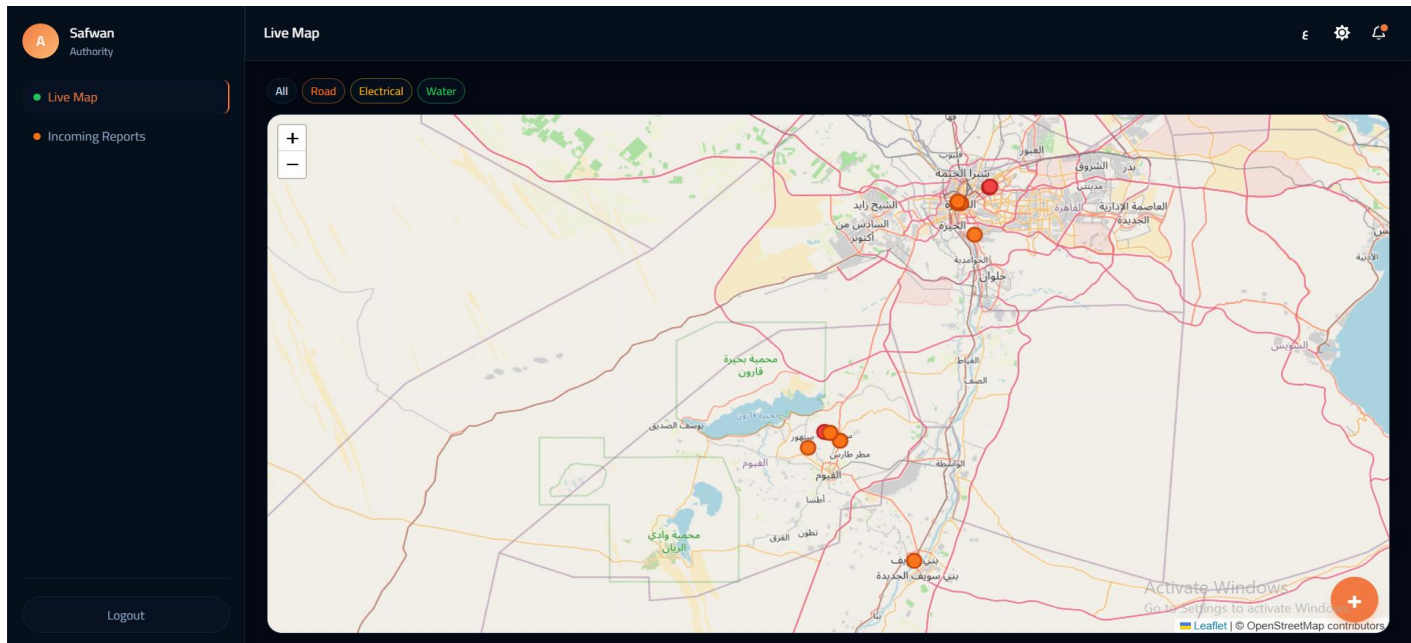
Personal Information	
FULL NAME	EMAIL ADDRESS
Ahmed Nasser	an1703@fayoum.edu.eg
MOBILE NUMBER	ADDRESS
—	Cairo, Egypt

Change Password	
CURRENT PASSWORD	
NEW PASSWORD	CONFIRM NEW PASSWORD
<button>Update Password</button>	

User Profile: Personal Information Management

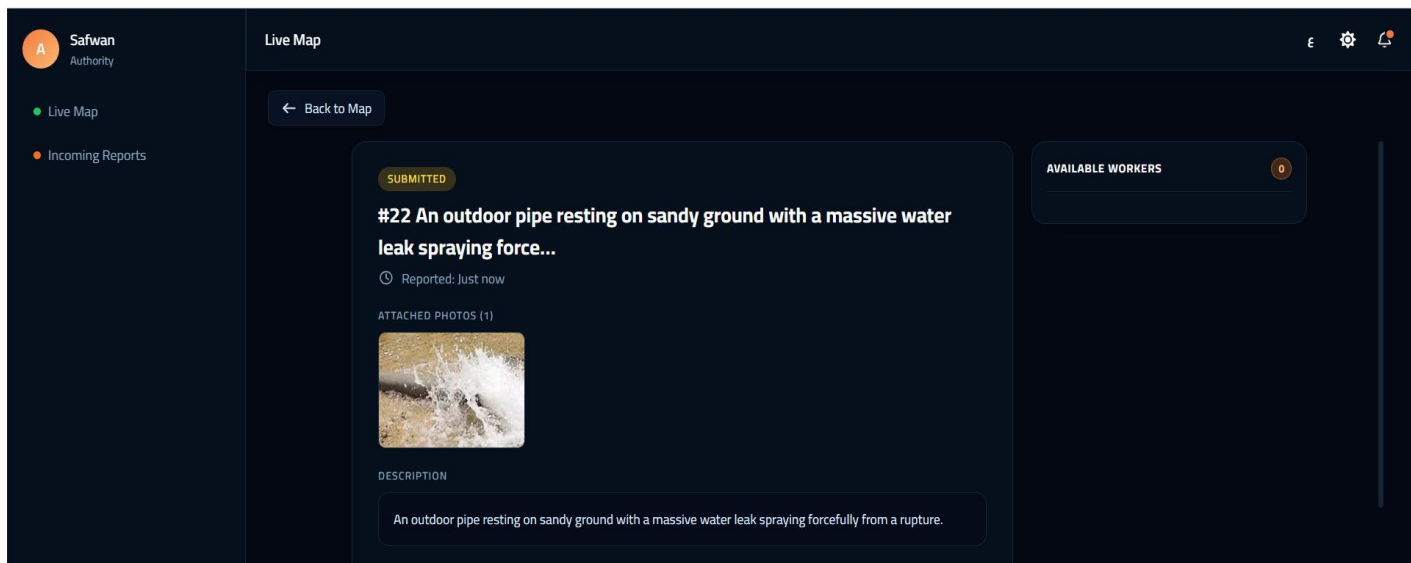
This interface allows users to manage their personal details and update account security settings, ensuring a personalized and secure experience aligned with the system's role-based access requirements.

Authority Dashboard:



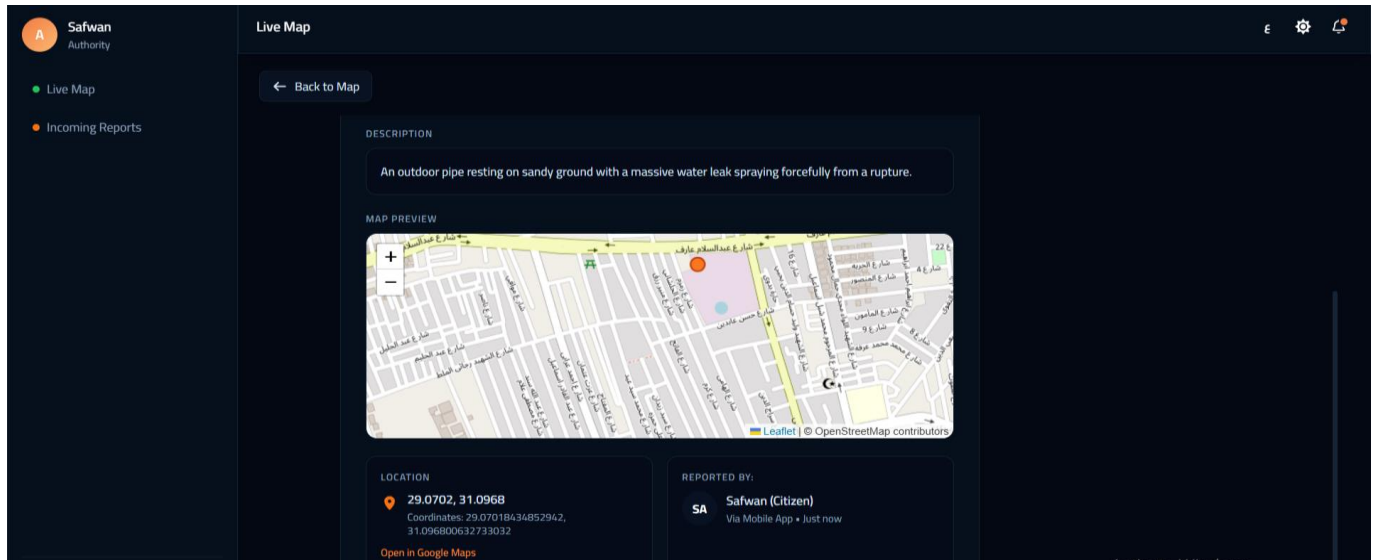
Authority Dashboard: Live Monitoring Interface

This interface provides authorities with a real-time, map-based visualization of reported infrastructure issues, enabling efficient oversight and geographical tracking of active reports within the system's workflow.



Authority Dashboard: Incoming Report Details

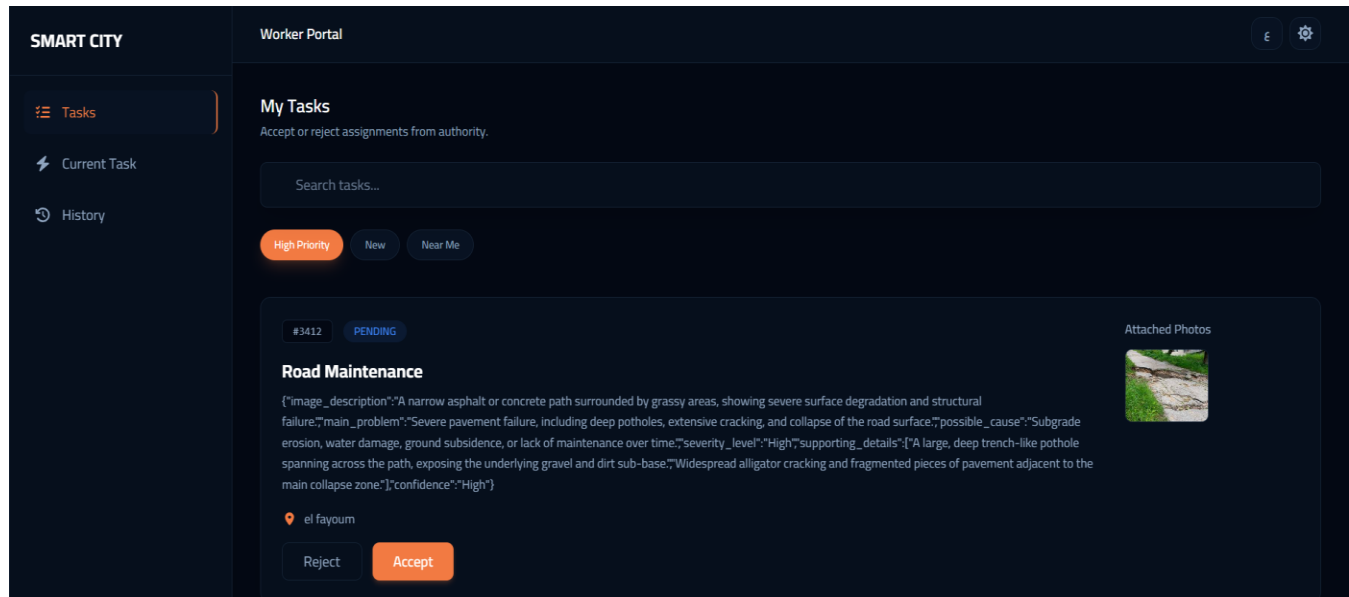
This detailed view allows authorities to inspect specific infrastructure reports, providing access to essential data including report status, attached evidence, and description, which supports informed decision-making and efficient task dispatching.



Authority Dashboard: Geographical Report Mapping

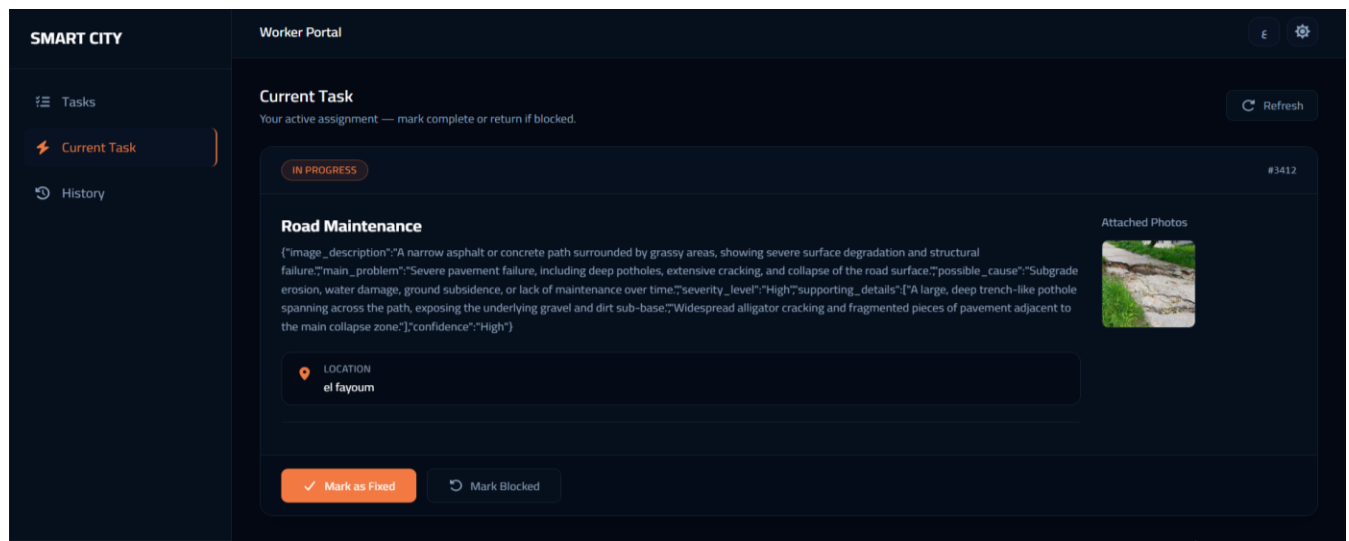
This feature provides a detailed geographical visualization of infrastructure issues, offering precise coordinates and map previews that enable authorities to pinpoint the location of reported incidents for accurate assessment and effective maintenance planning.

Worker Dashboard:



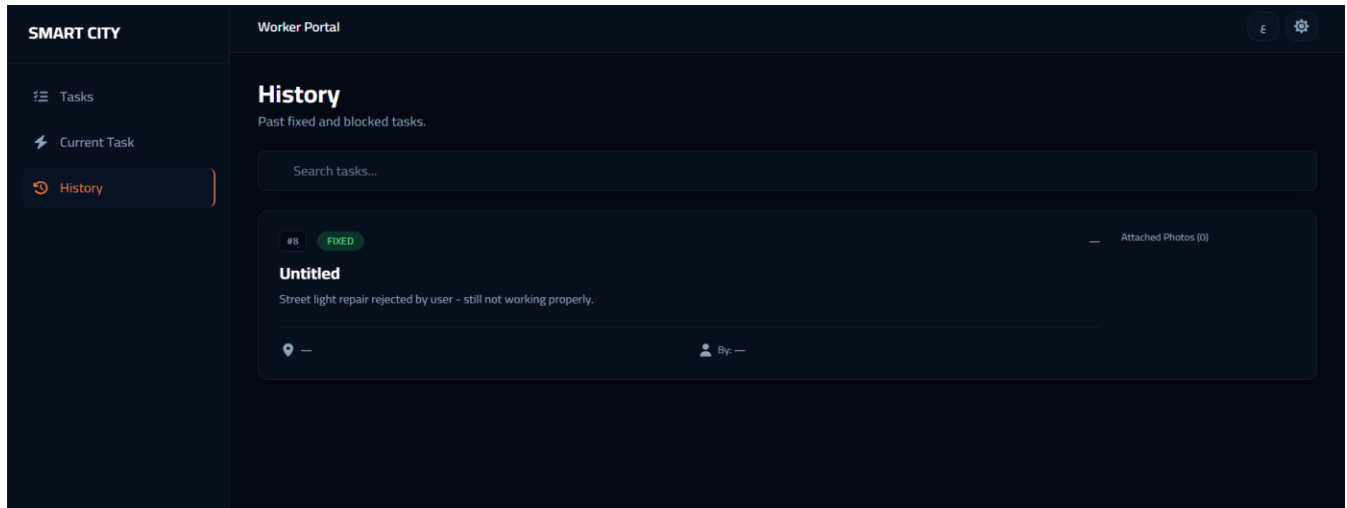
Worker Portal: Task Management Interface

This interface facilitates efficient task management for field workers, allowing them to view, accept, or reject maintenance assignments dispatched by authorities, thereby ensuring streamlined operational workflow within the Smart Infrastructure Reporting System.



Worker Portal: Active Task Management

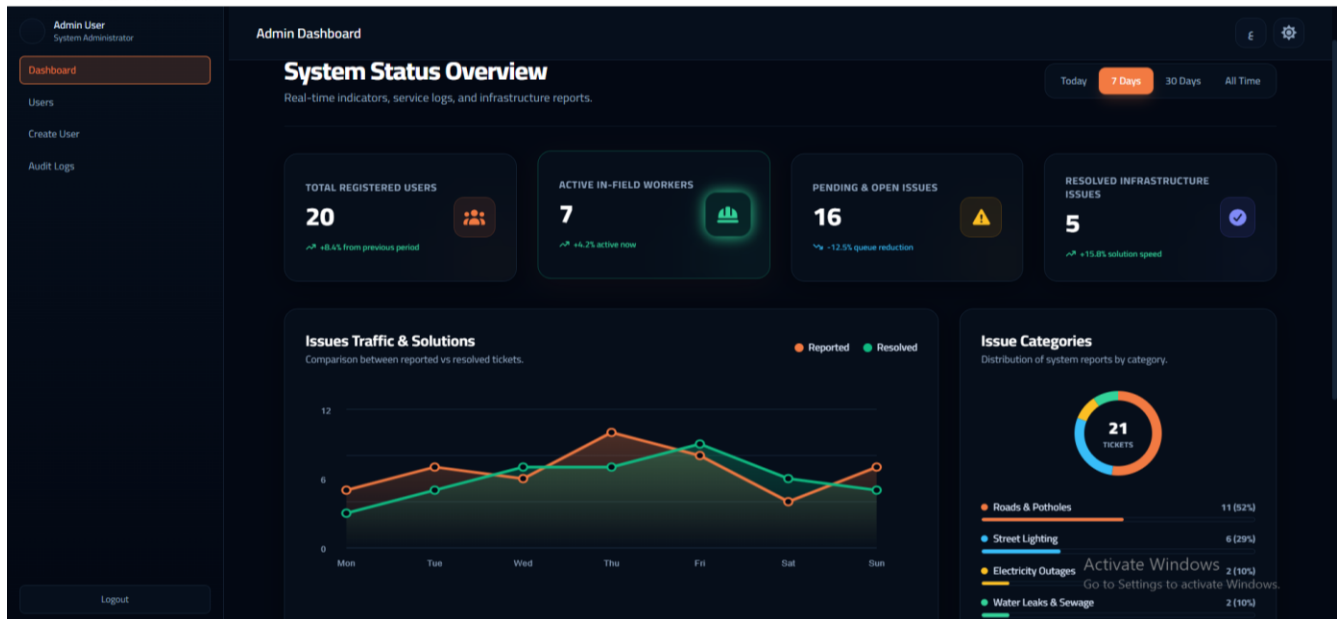
This interface provides field workers with a dedicated view of their current assignment, enabling them to actively manage maintenance work, update task status, and report potential blocks, thus ensuring seamless integration within the system's operational workflow.



Worker Portal: Task History Archive

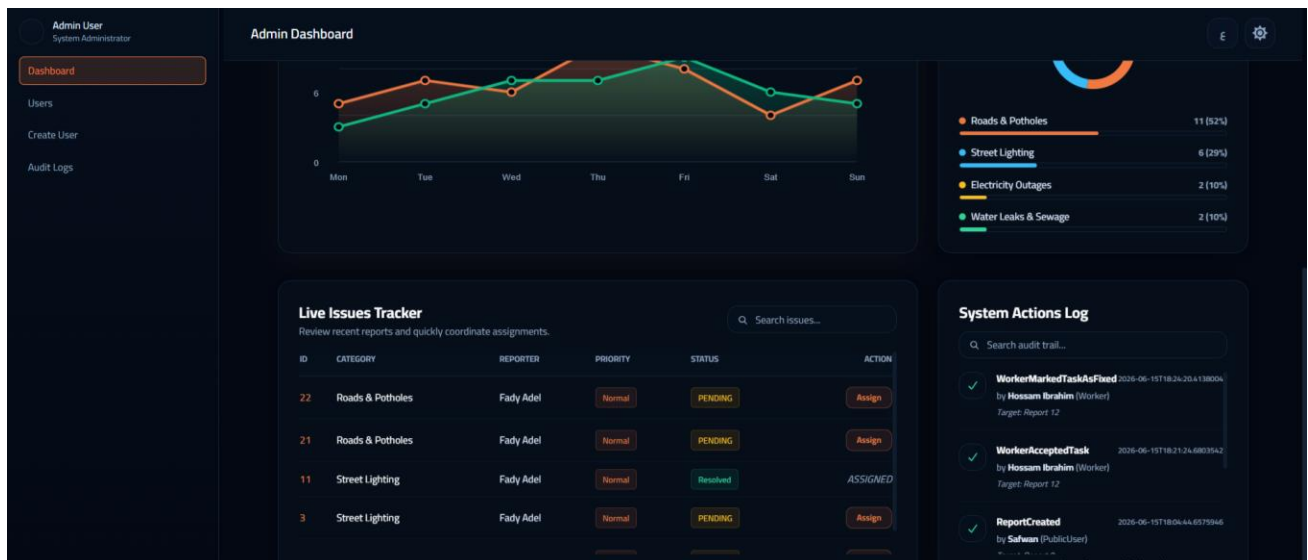
This interface maintains a comprehensive record of completed and blocked maintenance tasks, allowing field workers to review their past performance and analyze historical data within the system's operational workflow.

Administrator Dashboard:



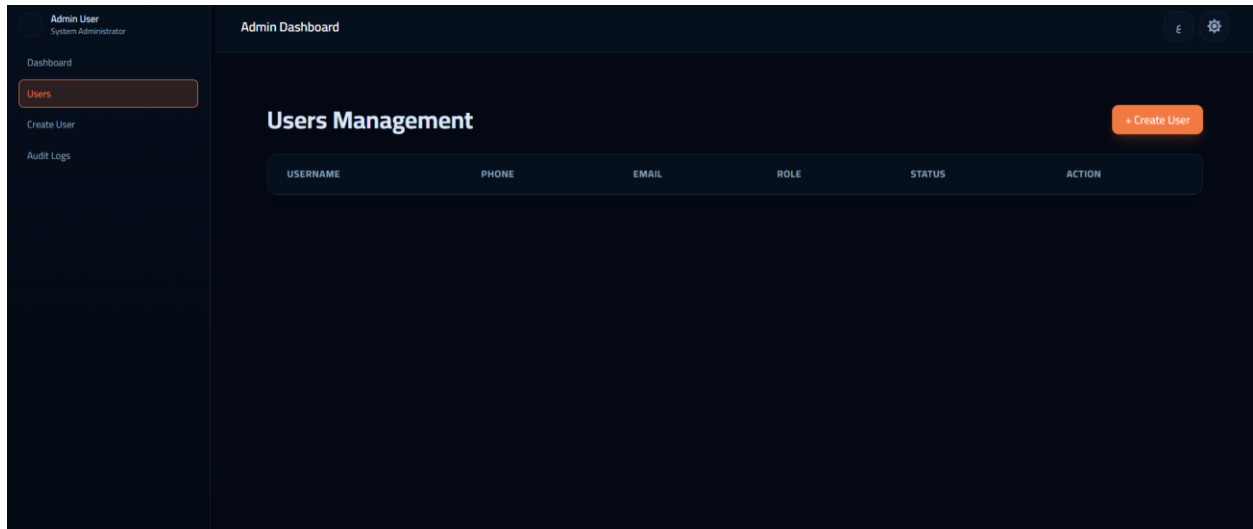
Administrator Dashboard: System Status Overview

This centralized dashboard provides administrators with high-level analytical insights and real-time system metrics, including key performance indicators (KPIs), infrastructure report statistics, and category distribution to facilitate effective system management and data-driven decision-making



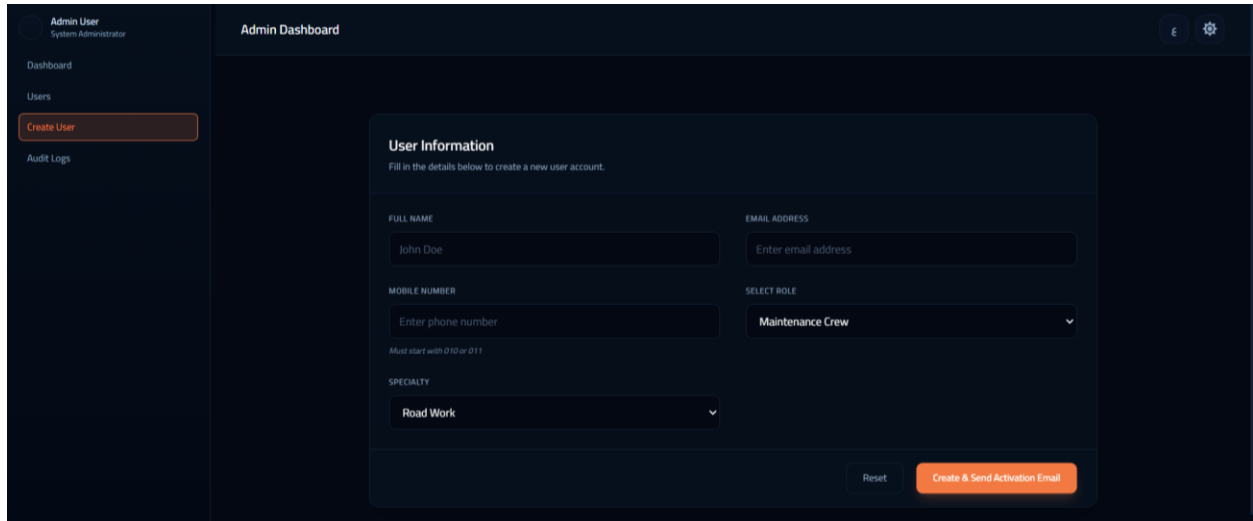
Administrator Dashboard: Operational Oversight & Audit

This interface facilitates comprehensive administrative control by providing a "Live Issues Tracker" for real-time task coordination and a "System Actions Log" for auditing activities, ensuring full transparency and efficient management of the infrastructure reporting workflow.



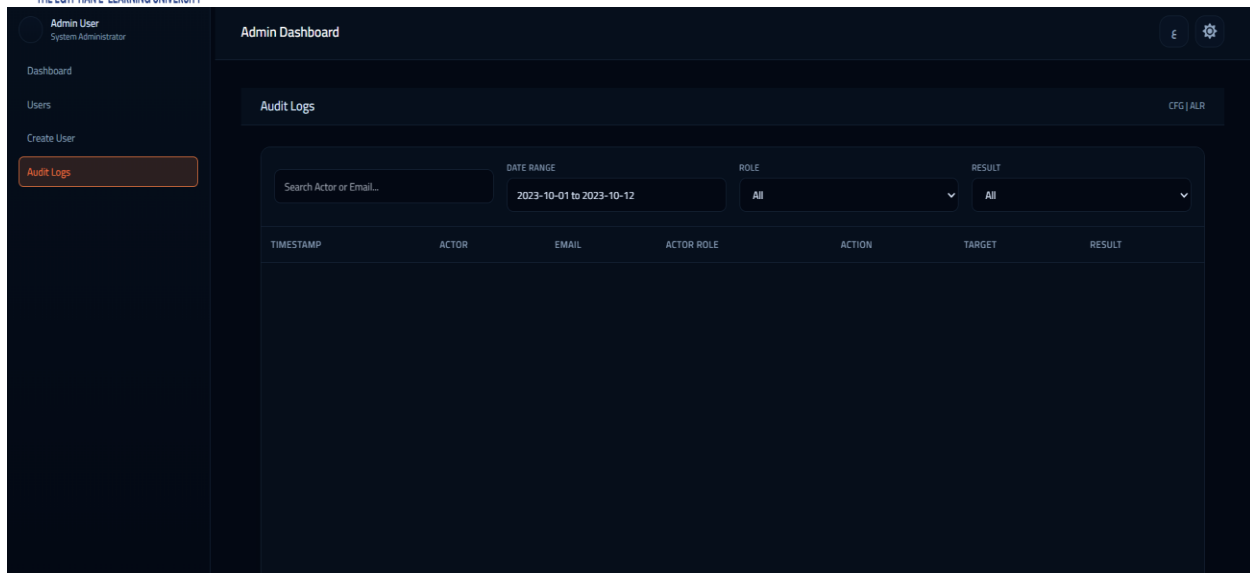
Administrator Dashboard: Users Management

This interface provides administrative oversight for managing system accounts, enabling the efficient addition, monitoring, and administration of registered users to maintain security and control over platform access.



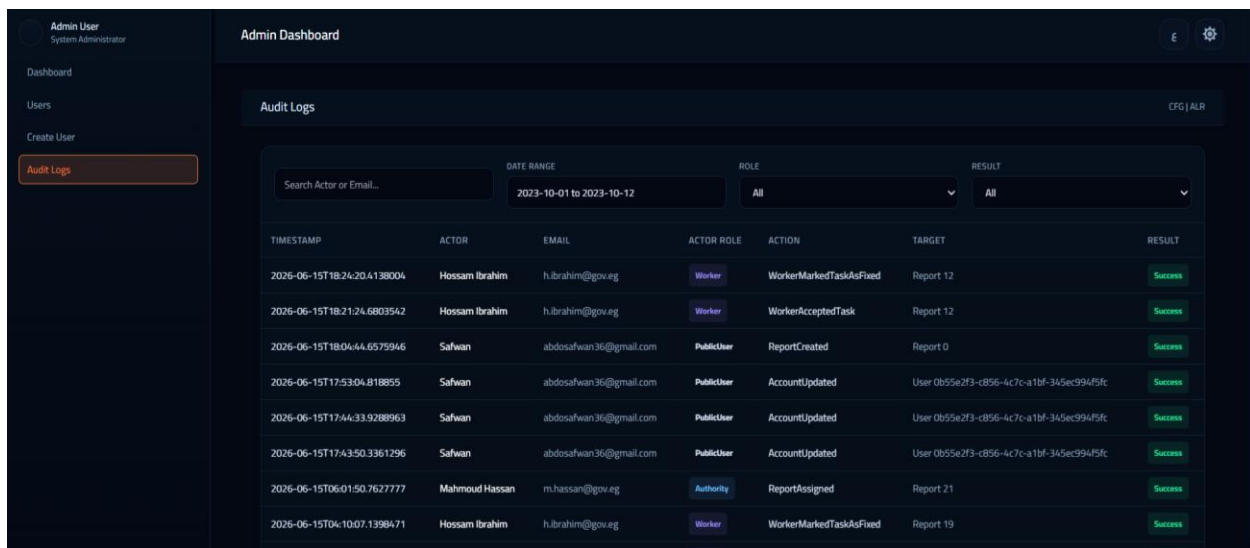
Administrator Dashboard: User Creation & Provisioning

This interface enables administrators to securely create and provision new system accounts by inputting user details, defining roles, and specifying specialties, facilitating seamless integration of new personnel into the system's operational hierarchy.



Administrator Dashboard: System Audit Logs

This interface provides a secure and transparent mechanism for tracking and reviewing system activities, enabling administrators to monitor user actions, filter logs by date and role, and ensure accountability within the platform's operations.



TIMESTAMP	ACTOR	EMAIL	ACTOR ROLE	ACTION	TARGET	RESULT
2026-06-15T18:24:20.4138004	Hossam Ibrahim	h.ibrahim@gov.eg	Worker	WorkerMarkedTaskAsFixed	Report 12	Success
2026-06-15T18:21:24.6803542	Hossam Ibrahim	h.ibrahim@gov.eg	Worker	WorkerAcceptedTask	Report 12	Success
2026-06-15T18:04:44.6575946	Safwan	abdosaifan36@gmail.com	PublicUser	ReportCreated	Report 0	Success
2026-06-15T17:53:04.818855	Safwan	abdosaifan36@gmail.com	PublicUser	AccountUpdated	User 0b55e2f3-c856-4c7c-a1bf-345ec994f5fc	Success
2026-06-15T17:44:33.9288963	Safwan	abdosaifan36@gmail.com	PublicUser	AccountUpdated	User 0b55e2f3-c856-4c7c-a1bf-345ec994f5fc	Success
2026-06-15T17:43:50.3361296	Safwan	abdosaifan36@gmail.com	PublicUser	AccountUpdated	User 0b55e2f3-c856-4c7c-a1bf-345ec994f5fc	Success
2026-06-15T06:01:50.7627777	Mahmoud Hassan	m.hassan@gov.eg	Authority	ReportAssigned	Report 21	Success
2026-06-15T04:10:07.1398471	Hossam Ibrahim	h.ibrahim@gov.eg	Worker	WorkerMarkedTaskAsFixed	Report 19	Success

Administrator Dashboard: Detailed System Audit Trail

This interface presents a detailed log of system events, offering administrators granular visibility into user actions, role-specific operations, and task outcomes, which serves as a critical mechanism for maintaining system integrity and operational accountability within the Smart Infrastructure Reporting System.

4.1.9 State Management and Data Flow

The front-end application follows a service-based state management approach to handle data exchange between components and the back-end API. Angular services act as centralized data providers, allowing multiple components to access and update shared information without unnecessary duplication.

Data retrieved from the back-end APIs is processed within dedicated services before being presented to the user interface. This approach improves maintainability, reduces coupling between components, and simplifies future enhancements.

The use of observables and asynchronous programming techniques enables the application to react efficiently to data updates and user interactions while maintaining a responsive user experience.

4.1.10 Security Implementation

Security was considered throughout the front-end implementation to ensure that sensitive system functionalities remain protected.

Authentication is based on JSON Web Tokens (JWT) issued by the back-end authentication service. After successful login, the token is securely stored and automatically attached to protected API requests using Angular HTTP Interceptors.

Route Guards are utilized to prevent unauthorized users from accessing restricted pages. Additionally, role-based access control mechanisms ensure that users can only access interfaces and functionalities associated with their assigned roles.

Client-side validation is also applied to reduce invalid requests and improve overall application security and usability.

4.1.11 Error Handling and User Feedback

The application incorporates comprehensive error handling mechanisms to improve usability and system reliability.

API errors are captured and processed through dedicated service layers before being displayed to users. Meaningful error messages are provided for common scenarios such as authentication failures, invalid form submissions, network issues, and unauthorized access attempts.

Loading indicators and validation messages are used throughout the application to provide users with immediate feedback regarding system operations and request processing status.

These mechanisms contribute to a more user-friendly experience and help users understand system responses effectively.

4.1.12 Reusable Components and Code Reusability

To improve maintainability and reduce code duplication, several reusable components were developed and shared across different modules of the application.

Examples include:

- Navigation components
- Header and footer components
- Form validation components
- Loading indicators
- Notification and alert components
- Shared UI elements

Reusable Angular services were also implemented for authentication, report management, profile management, and API communication.

This approach improves consistency across the application while simplifying future development and maintenance activities.

4.1.13 Front-End Performance Considerations

Several techniques were applied to improve front-end performance and responsiveness.

Lazy loading principles were considered when organizing application modules to reduce initial loading time and improve user experience.

API requests were optimized to retrieve only the required data, minimizing unnecessary network traffic between the client and server.

Component reusability and efficient routing mechanisms further contribute to reducing resource consumption and improving overall application responsiveness.

These optimizations help ensure that the application remains efficient even as additional features are introduced in future versions.

4.1.14 Summary

The front-end implementation successfully provided a responsive, interactive, and role-based user interface for the Smart Infrastructure Reporting System.

By utilizing Angular and modern web development technologies, the application delivers an efficient user experience while maintaining scalability, maintainability, and security. The modular architecture, reusable components, API integration mechanisms, and role-based navigation system collectively support the operational requirements of the infrastructure reporting platform.

The implemented user interfaces enable seamless interaction between citizens, authorities, workers, and administrators, contributing to effective infrastructure issue reporting and management.

4.2 Back-End Implementation

4.2.1 Architecture Overview

The back-end of the Infrastructure Reporting System (IRS) is implemented as a **RESTful Web API** built on the **.NET 9** platform using **ASP.NET Core**. The solution adopts a **Clean Architecture** approach, organizing all projects into three solution folders that correspond to distinct architectural layers: the *Application Core Layer*, the *Infrastructure Layer*, and the *Presentation Layer*. This separation enforces a strict, unidirectional dependency flow that prevents lower-level implementation details from contaminating higher-level business logic.

The dependency flow is defined as follows: a Controller in the Presentation Layer depends only on a service interface defined in the ServiceAbstractions project. The concrete service implementation, residing in the Services project, depends on a repository interface also defined in ServiceAbstractions. The concrete repository implementation, residing in the Persistence project, depends on the AppDbContext. This chain ensures that no project ever holds a direct reference to a project above it in the hierarchy, and that the Domain and ServiceAbstractions layers remain free of infrastructure concerns.

A conceptual diagram of this dependency flow is presented below:

This architecture yields several critical engineering benefits: the system can be tested by substituting mock implementations of any interface without modifying production code; external dependencies such as the database engine or email provider can be swapped with minimal

disruption; and each project has a single, well-defined responsibility that is traceable to a specific layer of the system.

4.2.2 Project Structure

The solution contains seven projects distributed across three solution folders. Each project holds a precisely scoped responsibility.

The ApplicationCoreLayer solution folder contains three projects:

InfraReportingSystem.Domain defines the core business entities and domain enumerations. It holds no dependencies on any other project in the solution, making it the foundational layer that the entire system builds upon.

InfraReportingSystem.ServiceAbstractions defines all contracts for the system, including repository interfaces and service interfaces. By centralising all abstractions in a dedicated project, the system prevents upper layers from taking dependencies on concrete implementations. The Controllers project references ServiceAbstractions to obtain service contracts, and the Services project references the same project to obtain repository contracts, without either layer knowing about the other's internal structure.

InfraReportingSystem.Services contains the concrete implementations of all service interfaces. It references ServiceAbstractions to fulfil interface contracts and to call repository interfaces. It contains no references to the Persistence project, ensuring that the business logic layer remains decoupled from the specific database technology in use.

The InfrastructureLayer solution folder contains one project:

InfraReportingSystem.Persistence hosts the Entity Framework Core (EF Core) DbContext, all Fluent API entity configurations, code-first database migrations, the concrete repository implementations, and the startup data seeder. It references the Domain and ServiceAbstractions projects.

The PresentationLayer solution folder contains three projects:

InfraReportingSystem.Presentation hosts middleware components, action filters, and IServiceCollection extension methods that wire dependencies together. These extension methods are called from Program.cs to register all services, repositories, and third-party clients with the dependency injection container.

InfrastructureReportingSystem is the executable API project. It contains the ASP.NET Core controllers, Swagger configuration, Program.cs entry point, and application settings files. It is the only deployable artefact.

InfraReportingSystem.Shared is a cross-cutting library containing Data Transfer Objects (DTOs), strongly-typed settings classes, and HTML email templates. It has no dependencies on any other project in the solution and can be referenced by any layer that needs to exchange data.

4.2.3 Domain Layer

The Domain Layer defines the persistent entities and enumerations that represent the core concepts of the infrastructure reporting domain. All entities reside in the `InfraReportingSystem.Domain/Entities` directory and are mapped to database tables by Entity Framework Core in the Persistence Layer.

User is the primary identity entity and extends `IdentityUser` from ASP.NET Core Identity. It introduces additional properties including `Name` (the user's display name), `ProfilePictureUrl` (a URL pointing to the user's profile photograph stored on Cloudinary), `Status` (a `UserStatus` enumeration tracking whether the account is Active, Inactive, Suspended, or Locked), and `CreatedAt`. The User entity owns navigation collections for submitted reports, OTP verifications, audit log entries, refresh tokens, and reports the user has marked as affected.

Authority extends User and represents a municipal authority account. It holds a navigation collection of reports that the authority has assigned to workers.

Worker also extends User and represents a field worker account. In addition to inherited User properties, the Worker entity includes a `Specialization` property that describes the worker's technical area of expertise (for example, "Water & Plumbing" or "Roads & Asphalt"). It holds a navigation collection of reports assigned to it.

Report is the central domain object. A Report is submitted by a `PublicUser` and progresses through a lifecycle managed by Authority and Worker roles. Its properties include a textual `Description`, a `ReportStatus` enumeration value (described below), geographic coordinates (Latitude and Longitude stored as SQL floats), timestamps for when it was uploaded (`UploadedAt`), last updated (`UpdatedAt`), and assigned to a worker (`AssignedAt`). The Report entity also stores an optional `RejectionReason`. It maintains foreign-key relationships to the submitting User, an optional assigned Worker, an optional assigning Authority, a Category, and collections of `ReportPic` and `ReportAffectedUser` records.

The `ReportStatus` enumeration captures the full lifecycle of a report through nine discrete states: Submitted (the initial state upon creation), Assigned (a worker has been assigned by an authority), OnHold (work is temporarily paused), Blocked (the worker cannot proceed due to an external obstacle), InProgress (the worker is actively executing the task), PendingConfirmation (the worker has marked the issue as fixed and awaits citizen confirmation), Resolved (the citizen

has confirmed the fix), Rejected (the report or fix was rejected), and FixRejected (the citizen has disputed the worker's fix).

ReportPic stores the Cloudinary image URL (PicUrl) and optionally a PublicId for each photograph attached to a Report, enabling programmatic deletion of images from Cloudinary when required.

Category classifies reports into problem types. The seeded categories include "Roads & Potholes", "Street Lighting", "Water Leaks & Sewage", and "Electricity Outages".

OtpVerification persists One-Time Password (OTP) codes used for email confirmation and password recovery. Each record stores the six-digit code, its creation timestamp, its expiration timestamp, a boolean IsUsed flag, and an OtpPurpose enumeration value (either EmailConfirmation or PasswordReset). Multiple OTP records may exist per user, as the system generates a fresh code on each request.

RefreshToken stores the token string, the creation timestamp (CreatedOn), the expiration timestamp (ExpiresOn), and an optional revocation timestamp (RevokedOn). Two computed properties, IsExpired and IsActive, derive their values from these timestamps at runtime without being persisted to the database. This design allows the system to determine a token's validity without an additional database query.

AuditLog records critical system events for accountability purposes. Each entry captures the acting user's identifier (UserId, which is nullable to accommodate failed login attempts from unrecognised users), an AuditActionType enumeration value identifying the category of action, the name of the affected entity (EntityName), the identifier of the affected entity (EntityId), a human-readable Details string, and a Timestamp. The AuditActionType enumeration defines 23 distinct event types grouped into authentication events (Login, LoginFailed, Logout, PasswordReset, PasswordChanged), account events (AccountCreated, AccountDeleted, AccountSuspended, AccountReactivated, AccountActivated, AccountUnlocked, AccountLocked, AccountUpdated, AccountVerified), report events (ReportCreated, ReportUpdated, ReportAssigned, ReportBlocked, ReportResolved, ReportRejected), and worker events (WorkerAcceptedTask, WorkerRejectedTask, WorkerMarkedTaskAsFixed).

ReportAffectedUser represents the many-to-many join entity between Report and User for the "Also Suffer" feature. Each record stores the report identifier, the affected user's identifier, and a CreatedAt timestamp recording when the user registered their association.

Role extends IdentityRole to allow the system to attach custom metadata to roles if required in future iterations.

4.2.4 Database Design and Persistence Layer

The Persistence Layer manages all interaction between the application and the SQL Server database through Entity Framework Core 9 (EF Core), a modern Object-Relational Mapping (ORM) framework that translates C# object operations into SQL queries.

The `AppDbContext` class inherits from `IdentityDbContext`, which automatically configures the full set of ASP.NET Core Identity tables (for users, roles, claims, and tokens) alongside the application's own `DbSet` properties. The following `DbSet` properties are declared: `Workers`, `Authorities`, `Reports`, `Categories`, `ReportPics`, `AuditLog`, `OtpVerifications`, `RefreshTokens`, and `ReportAffectedUsers`.

`AppDbContext` overrides `SaveChangesAsync` to implement an automatic auditing concern: whenever a `Report` entity is in the `EntityState.Modified` state at the time of persistence, the `UpdatedAt` property is automatically set to `DateTime.UtcNow` before the underlying base method is called. This ensures that the last-modified timestamp is always accurate without requiring calling code to set it manually.

Entity configurations are defined using the Fluent API pattern by implementing the `IEntityTypeConfiguration` interface in individual configuration classes stored in the `Persistence/Configurations` directory. The call `modelBuilder.ApplyConfigurationsFromAssembly()` in `OnModelCreating` discovers and applies all configuration classes automatically, keeping the `DbContext` itself free of configuration details.

The `ReportConfiguration` class illustrates the breadth of configuration applied. It restricts the `Description` property to a maximum length of 1,000 characters, stores the `ReportStatus` enumeration as an integer in the database, enforces geographic validity through database-level CHECK constraints (`CK_Report_Latitude` and `CK_Report_Longitude` ensuring latitude is within -90 to 90 and longitude is within -180 to 180), and sets the `UploadedAt` default value using the SQL function `GETUTCDATE()`. It also defines six database indexes to optimise the most frequently executed queries: individual indexes on `Status`, `CategoryId`, `SubmittedById`, `AssignedWorkerId`, and `UploadedAt`, plus two composite indexes—one on (`AssignedWorkerId`, `Status`) for worker task queries and one on (`SubmittedById`, `Status`) for citizen report history queries. Relationships are configured with explicit delete behaviours using `Restrict` to prevent cascading deletions across report participants, and `Cascade` for `ReportPic` records, which are considered children of their parent `Report`.

Database schema evolution is managed through EF Core code-first migrations. Schema changes are authored by running the `dotnet ef migrations add` command in the development environment, which generates a timestamped migration file. For production deployment, migrations are applied using the `Script-Migration` command to produce a SQL script that is executed against the production database in a controlled manner.

The DataSeeder class is responsible for ensuring that the database contains a minimum set of data required for the system to operate. On application startup, the seeder checks whether any pending migrations exist and applies them if necessary. It then seeds the four application roles (Admin, Authority, Worker, PublicUser), seeds an initial Admin user account alongside representative Authority, Worker, and PublicUser accounts, seeds the four default report categories, and seeds a set of sample reports covering every ReportStatus value to support functional testing. The seeder is designed to be idempotent; each seeding operation first checks whether the target data already exists before inserting it, preventing duplicate records on subsequent startups. Error handling is implemented using structured try-catch blocks with EF Core's EnableRetryOnFailure policy on the connection string, which provides resilience against transient SQL Server connectivity failures during startup.

4.2.5 Feature Organization Pattern

All layers of the system — Service Abstractions, Services, and Controllers — follow an identical screen-based folder hierarchy. The hierarchy is organised first by role (Admin, Authority, PublicUser, Worker) and then by the specific functional screen within that role. Shared cross-cutting concerns reside in a separate Shared folder. This organisational pattern means that every feature can be traced vertically through all layers of the system by following an identical path.

For example, the Submit Report feature for the Public User role is implemented as follows: the repository interface `IPublicSubmitReportRepository` resides at `ServiceAbstractions/Repositories/Users/PublicUser/SubmitReport/`, the service interface `IPublicSubmitReportService` resides at `ServiceAbstractions/Users/PublicUser/SubmitReport/`, the service implementation `PublicSubmitReportService` resides at `Services/Users/PublicUser/SubmitReport/`, the controller resides at `Controllers/PublicUser/SubmitReport/`, and the relevant DTOs reside in the Shared project under a matching path. A developer who identifies a defect in the Submit Report feature can immediately locate every relevant file by following this consistent naming convention, without needing to search across the project.

The Shared project mirrors this structure for its DTOs and settings, further reinforcing the traceability principle. The complete screen hierarchy is as follows:

Shared/Auth, Shared/Email, Shared/Images, Shared/Profile
 Users/Admin/AuditLogsScreen, DashboardScreen, UserCreationScreen, UsersManagementScreen
 Users/Authority/AssignWorker, IncomingReports, Map, MarkerPopup, Workers
 Users/PublicUser/AlsoSuffer, Map, MarkerPopup, MyReports, NearbyReports, SubmitReport
 Users/Worker/CurrentTaskScreen, TasksHistoryScreen, TasksScreen

4.2.6 Core Business Features: Public User

The Public User role represents citizens who report infrastructure problems through the system. Six feature screens serve this role.

Submit Report

The Submit Report feature is the primary entry point for citizen participation. It is implemented in `PublicSubmitReportService` and exposes two AI-assisted endpoints (`GenerateCategoryAsync` and `GenerateDescriptionAsync`) in addition to the core submission endpoint.

The submission flow, implemented in `SubmitReportAsync`, proceeds through six sequential steps. First, the category identifier provided by the user is validated by querying the database; an exception is raised if the category does not exist. Second, the image attached to the request is uploaded to Cloudinary via the `IImageService` abstraction; if the upload returns an empty URL, the operation is aborted with an error. Third, a new `Report` entity is constructed with the description, geographic coordinates (Latitude and Longitude), a default status of `ReportStatus.Submitted`, the validated category, and the identifier of the authenticated user. Fourth, the `Report` is persisted to the database. Fifth, a `ReportPic` entity linking the Cloudinary URL to the `Report` is persisted. Sixth, an `AuditLog` entry with `ActionType` of `AuditActionType.ReportCreated` is persisted. All three persistence operations are committed in a single call to `SaveChangesAsync`, ensuring atomicity.

Prior to submission, the user may invoke the AI-assisted category suggestion endpoint, which forwards the uploaded image to the `ICategoryAIClient` (backed by a Hugging Face Space) to receive a suggested category label. Similarly, the AI-assisted description endpoint invokes `IDescriptionAIClient` to receive a generated textual description of the visible infrastructure problem. Both results are returned to the client as suggestion DTOs that the user may accept, modify, or discard before submitting.

Nearby Reports

The Nearby Reports feature retrieves infrastructure reports located within a specified radius of the user's current geographic coordinates. The `IPublicNearbyReportsRepository` interface defines `GetNearbyReportsAsync(double latitude, double longitude, double radiusInKm)`, which accepts the user's current position and a search radius in kilometres. The repository implementation uses the Haversine formula or an equivalent SQL-level spatial calculation to filter reports whose coordinates fall within the specified radius. The returned `NearbyReportDto` objects provide

enough information for the client to render a list or map of problems near the user's location without exposing unnecessary internal fields.

Also Suffer

The Also Suffer feature allows a user to register that they are personally affected by an existing report, thereby increasing its perceived severity and urgency. The `IPublicAlsoSufferRepository` defines three operations: `GetReportStatusAsync` to retrieve the current status of the target report, `AlreadyAffectedAsync` to check whether the requesting user has already registered their affiliation, and `AddAsync` to persist a new `ReportAffectedUser` record. The service layer validates that the report exists and is in an active state before performing the duplication check, ensuring that a user can mark themselves as affected only once per report.

Map and Marker Popup

The Public User Map feature provides the front-end mapping component with filtered report data for geographic visualisation. The `IPublicMapRepository` defines `GetMapReportsAsync(int? categoryId, ReportStatus? status)`, which accepts optional filter parameters allowing the client to display all reports or a filtered subset by category and status. The `MarkerPopup` feature retrieves the detailed information required to render an informational popup when a user selects a specific map marker, including the report description, category, status, and submitted images.

My Reports

The My Reports feature retrieves all reports submitted by the currently authenticated user. It applies server-side pagination to avoid transmitting large result sets, returning a `PaginatedResult` wrapper object that includes the total count, current page number, page size, and the list of items for the requested page.

4.2.7 Core Business Features: Authority

The Authority role represents municipal authorities responsible for reviewing incoming reports and assigning field workers to address them. Five feature screens serve this role.

Incoming Reports

The Incoming Reports screen provides Authority users with a managed feed of newly submitted reports that have not yet been assigned to a worker. The repository queries the `Reports` table for entries with a status of `ReportStatus.Submitted`, ordered by submission date in descending order. The service layer applies pagination and maps the results to response DTOs containing the report identifier, description, category, location coordinates, submission timestamp, and the number of affected users who have registered via the Also Suffer feature.

The Assign Worker feature represents the central operational workflow of the Authority role and is implemented in AuthorityAssignWorkerService. The AssignWorkerAsync method accepts a report identifier, a worker identifier, and the identifier of the currently authenticated authority.

The method enforces a strict validation sequence. The target report is retrieved; if it does not exist, an error is returned. The report's current status is checked; only reports with a status of ReportStatus.Submitted may be assigned. If the status is anything other than Submitted, the operation is rejected with an informative message indicating the current status. The target worker is then retrieved from the identity store; if the worker does not exist, an error is returned. The worker's account status is validated to ensure it is UserStatus.Active. The worker's assigned role is confirmed to be "Worker" by querying ASP.NET Core Identity's role management. Finally, the repository's HasActiveTaskAsync method is called to check whether the worker currently holds any report in an active state (Assigned or InProgress); if the worker is already occupied, the assignment is rejected to prevent overloading a single field worker.

If all validations pass, the report's AssignedWorkerId, AssignedByAuthorityId, Status (changed to ReportStatus.Assigned), and AssignedAt timestamp are updated. An AuditLog entry with ActionType of AuditActionType.ReportAssigned is persisted. Changes are committed to the database. Finally, an assignment notification email is dispatched asynchronously to the worker's email address via the IEmailService. The email contains the report identifier, description, and geographic coordinates. Email dispatch failures are caught and logged rather than propagating to the caller, ensuring that a transient email delivery failure does not roll back a successful assignment.

Workers Management

The Workers Management screen allows Authority users to view the roster of active field workers, their specializations, and their current workload. The repository retrieves all users in the Worker role, joining each worker's record with the count of currently assigned reports to allow the Authority to make informed assignment decisions.

Map and Marker Popup

The Authority Map feature provides the authority with a spatial overview of all active reports within the system. The authority-specific map repository supports the same category and status filtering as the Public User map but additionally exposes worker assignment information in the marker data, allowing the authority to visually identify geographic clusters of unassigned problems.

4.2.8 Core Business Features: Worker

The Worker role represents field technicians who receive task assignments and execute repairs. Three feature screens serve this role.

Tasks Screen

The Tasks Screen presents the authenticated worker with a list of reports that have been assigned to them. The `IWorkerTasksRepository` retrieves all reports where `AssignedWorkerId` matches the current worker's identifier and the status is `Assigned`, ordered by assignment date. The Tasks Screen also provides action endpoints. The `IWorkerTaskActionsRepository` defines `GetByIdAsync` to retrieve a specific assigned task, `HasActiveTaskAsync` to check for an existing in-progress task, and `UpdateAsync` to persist status changes. When a worker accepts a task, its status transitions from `Assigned` to `InProgress`, and an `AuditLog` entry of type `WorkerAcceptedTask` is created. When a worker rejects an assigned task, its status reverts to `Submitted` (making it available for reassignment by the authority), and an `AuditLog` entry of type `WorkerRejectedTask` is created.

Current Task Screen

The Current Task Screen allows a worker to view the full details of the task they are currently executing. The `IWorkerCurrentTaskRepository` defines `GetCurrentTaskAsync(string workerId)`, which retrieves the single report in `InProgress` status assigned to the requesting worker. The `WorkerCurrentTaskService` maps this report to a `CurrentTaskDto` that includes the report description, category, photographs, geographic coordinates, a Google Maps deep link for navigation, the name of the submitting citizen, and the name of the authority who assigned the task.

The `WorkerCurrentTaskActionsService` provides two status-transition operations. The `MarkAsFixedAsync` method transitions the report from `InProgress` to `PendingConfirmation` and creates an `AuditLog` entry of type `WorkerMarkedTaskAsFixed`; it accepts an optional comment from the worker. The `MarkAsBlockedAsync` method transitions the report to `Blocked` and records a mandatory reason in the `RejectionReason` field; it creates an `AuditLog` entry of type `ReportBlocked`. The blocked status signals to the Authority that the worker has encountered an external obstacle preventing task completion.

Tasks History Screen

The Tasks History Screen retrieves the historical record of tasks previously handled by the authenticated worker, specifically those in terminal or closed states such as `Resolved`, `Rejected`, and `FixRejected`. This provides workers with a personal audit trail of their completed work and resolution outcomes.

4.2.9 Core Business Features: Admin

The Admin role provides system-wide management capabilities. Four feature screens serve this role.

Dashboard Screen

The Admin Dashboard delivers a comprehensive overview of the system's operational health through the AdminDashboardService, which aggregates five distinct data sources. GetStatsAsync retrieves four high-level metrics: the total number of registered users, the count of currently active workers, the count of reports in unresolved states, and the count of resolved reports. GetWeeklyTrafficAsync retrieves the daily counts of newly submitted and resolved reports for the current calendar week (Monday to Sunday), enabling the administrator to visualise weekly operational trends. GetCategoryStatsAsync retrieves per-category report counts and resolution rates, identifying which infrastructure categories generate the most complaints. GetLiveIssuesAsync retrieves a list of currently active reports; the service applies a priority classification by comparing each report's UploadedAt timestamp against a seven-day threshold — reports older than seven days are classified as "High" priority, while more recent reports are classified as "Normal". GetRecentActionsAsync retrieves the ten most recent AuditLog entries attributed to worker actions, providing the administrator with a real-time feed of field activity.

User Provisioning

The AdminUserCreationService implements the CreateUserAsync method, which allows the administrator to create Authority and Worker accounts. The method enforces strict input validation: the full name, a valid email address, an eleven-digit Egyptian-format phone number (beginning with 010 or 011), a valid role (either "Worker" or "Authority"), and for Worker accounts, a specialization field are all required. If any validation fails, an error message is returned immediately without attempting any database operation.

Upon passing validation, the service constructs the appropriate entity type (a Worker or Authority instance), assigns a system-generated cryptographically random placeholder password, creates the user account via ASP.NET Core Identity's UserManager, and assigns the appropriate role. If role assignment fails, the previously created user account is deleted to ensure consistency. A password reset token is then generated and embedded in a link pointing to the front-end's password-setting page, which is dispatched to the new user's email address via IEmailService. Finally, an AuditLog entry of type AccountCreated is persisted, recording which administrator performed the action.

Users Management

The Users Management screen provides the administrator with paginated, searchable lists of all system users. It supports administrative actions including account suspension (which sets the user's Status to UserStatus.Suspended), account reactivation, and account deletion. Each

management action is accompanied by a corresponding AuditLog entry to maintain a complete accountability trail.

Audit Logs

The Audit Logs screen presents the administrator with a paginated, filterable feed of all AuditLog records in the system. Filters are supported by ActionType enumeration value, by date range, and by the identifier of the acting user. The full breadth of the AuditActionType enumeration — spanning authentication, account management, report lifecycle, and worker actions — ensures that no significant system event occurs without a corresponding audit record.

4.2.10 Authentication and Authorization System

The authentication system is implemented in AuthService and OtpService and manages the complete identity lifecycle for all users.

Registration

When a new PublicUser registration request is received, the system first checks whether an account with the provided email address already exists. If an unconfirmed account exists, a fresh OTP is generated and dispatched to prompt the user to complete verification, and a response indicating the need for email confirmation is returned. If a confirmed account exists, registration is rejected with an appropriate message. Otherwise, a new User entity is constructed, the password is hashed and stored by ASP.NET Core Identity's UserManager, the account is assigned the PublicUser role, and an OTP is generated for email confirmation. If OTP generation fails (for example, due to a transient email service error), the newly created user account is deleted to preserve consistency, and a failure response is returned.

OTP-Based Email Confirmation

OTP-based email confirmation is implemented through OtpService. The GenerateOtp method creates a cryptographically generated six-digit code, stores it in the OtpVerification table with a fixed expiration window, and dispatches it to the user's registered email address. The VerifyOtp method retrieves the most recent unused, unexpired OTP record for the given user and purpose, validates the submitted code, and marks the record as used upon successful verification.

Confirmed registration transitions the user's Status to UserStatus.Active and sets the EmailConfirmed flag to true on the Identity user record.

Login and Token Issuance

The login flow validates the email and password against the Identity store. A comprehensive set of account-state checks is applied in sequence: whether the account is locked out via the ASP.NET Core Identity lockout mechanism, whether the email has been confirmed, whether the

account is in an Active status, and whether the account has been suspended. Upon successful validation, a JSON Web Token (JWT) access token is generated by the private `GenerateJwtToken` method, which constructs a set of claims including the user's identifier (`ClaimTypes.NameIdentifier`), email (`ClaimTypes.Email`), name (`ClaimTypes.Name`), and all assigned roles (`ClaimTypes.Role`). The token is signed using the HMAC-SHA256 (`HmacSha256`) algorithm with a symmetric key read from configuration and is set to expire after a configurable duration (defaulting to one day).

Concurrently, a refresh token is generated by the `GenerateRefreshToken` method using a 64-byte cryptographically random sequence encoded as a Base64 string, set to expire after a configurable number of days (defaulting to 30 days). The refresh token is persisted in the database, associated with the user. The access token is returned in the response body, while the refresh token is transmitted to the client via an HTTP-only cookie. An HTTP-only cookie is a browser security mechanism that makes the cookie inaccessible to client-side JavaScript, significantly reducing the attack surface for Cross-Site Scripting (XSS) attacks.

Token Refresh

The `RefreshTokenAsync` method allows clients to silently renew their session without requiring re-authentication. The method reads the refresh token from the HTTP-only cookie, retrieves the associated user from the database, validates that the token's `IsActive` computed property returns true (meaning the token is neither expired nor revoked), revokes the existing token by setting its `RevokedOn` timestamp, generates a new refresh token and JWT access token, and returns both to the client.

Password Recovery

Password recovery is implemented as a two-step flow. The `ForgotPasswordAsync` method accepts the user's email address and, without confirming whether the address belongs to an existing account, generates an OTP of purpose `PasswordReset` and dispatches it by email. Treating both found and not-found cases identically prevents email enumeration attacks. The `ResetPasswordAsync` method accepts the email, OTP code, and new password; validates the OTP; generates a standard ASP.NET Core Identity password reset token; and applies the new password. As an additional convenience for Authority and Worker accounts provisioned by the administrator (which are created with an Inactive status), a successful password reset automatically transitions the account status to Active, enabling the account for first use.

Logout

The `LogoutAllAsync` method revokes all refresh tokens associated with a given user by setting the `RevokedOn` timestamp on each record, effectively invalidating all active sessions across all devices.

4.2.11 AI Service Integration

The AI integration connects the IRS back-end to two machine learning models hosted on Hugging Face Spaces, accessed via standard HTTP calls managed by .NET's HttpClient.

Two AI client classes are implemented in the Services/AI directory. CategoryAIClient implements the ICategoryAIClient interface and invokes the /predict endpoint of the category classification Space. DescriptionAIClient implements the IDescriptionAIClient interface and invokes the /analyze endpoint of the description generation Space.

Both clients follow the same communication pattern. The uploaded image is wrapped in a MultipartFormDataContent object, with the image stream set to a content type of image/jpeg. The HTTP POST request is dispatched to the respective endpoint. The response body is parsed as a JSON document. CategoryAIClient extracts the value of the "prediction" property from the JSON response, which contains the predicted infrastructure problem category as a string. DescriptionAIClient extracts the value of the "image_description" property, which contains a natural language description of the visible problem. Both clients throw InvalidOperationException if the expected property is absent from the response, ensuring that unexpected API responses are surfaced as explicit errors rather than silent failures.

These clients are registered in the dependency injection container as typed HttpClient instances, with their base addresses configured from the application settings. The PublicSubmitReportService calls both clients to populate the category suggestion and description suggestion endpoints respectively, which the citizen may consult before manually submitting a report.

4.2.12 External Services: Email and Image Management

Email Service

Transactional email is handled by the EmailService class, which implements IEmailService using the MailKit library. MailKit is an open-source cross-platform MIME message library that provides full SMTP client functionality. The email infrastructure is provided by Brevo (formerly Sendinblue), which supplies an SMTP relay accessible at smtp-relay.brevo.com on port 587 using the StartTLS security protocol.

The SendEmailAsync method constructs a MimeMessage object containing the sender address (configured as the system's dedicated email address with a display name), the recipient address, the subject, and an HTML body. The method establishes an SMTP connection, authenticates using the configured credentials, transmits the message, and disconnects. Specific exception types are caught and logged with structured messages: AuthenticationException indicates invalid credentials, SmtplibCommandException indicates that the SMTP server rejected the command, and

`SmtpProtocolException` indicates a protocol-level communication failure. Each exception is re-thrown as an `InvalidOperationException` with a descriptive message to allow calling services to handle email failures appropriately. Email sending can be disabled entirely via the `EmailSettings:EnableSending` configuration key, which is useful for development and testing environments.

The system includes an `EmailSettings:EnableSending` configuration toggle and reads all SMTP parameters (host, port, username, password, sender address, and display name) from the application configuration, ensuring that credentials are never embedded in the source code.

Image Service

Image storage and management are handled by the `IImageService` abstraction, implemented using the Cloudinary .NET SDK. Cloudinary is a cloud-based media management platform that provides reliable, scalable image hosting with global content delivery.

When a report is submitted, the image stream is passed to the `UploadImageAsync` method, which uploads the image to a designated folder path on Cloudinary (specifically the "reports" folder) and returns the secure HTTPS URL of the hosted image. The returned URL is stored in the `ReportPic` entity's `PicUrl` property. Where applicable, the Cloudinary `PublicId` returned from the upload operation is also stored in the `ReportPic` entity, enabling the system to call the deletion API to remove images from Cloudinary when a report is deleted, preventing orphaned files from accumulating on the storage platform.

4.2.13 Configuration and Secrets Management

The system employs a three-tier configuration strategy to cleanly separate placeholder values from real credentials, ensuring that sensitive data is never committed to the Git repository.

The `appsettings.json` file contains non-sensitive placeholder values for all configuration keys. These include the `JwtSettings` section (Key, Issuer, Audience, `DurationInDays`, `RefreshTokenDurationInDays`), the `EmailSettings` section (Host, Port, Username, Password, FromEmail, DisplayName), the database connection string, and the public-facing application and front-end URLs. This file is committed to the repository and serves as documentation of all required configuration keys.

The `appsettings.Development.json` file, which is listed in the `.gitignore` file and therefore never committed to the repository, contains the real credentials used in the local development environment. Developers clone the repository and populate this file with their own local credentials to run the system.

The production environment on MonsterASP.NET is configured through the hosting panel's environment variable management interface. Each configuration key is set as an environment variable whose name follows the ASP.NET Core convention of using double underscores as hierarchy separators (for example, `JwtSettings__Key`). The ASP.NET Core configuration system automatically reads these environment variables and merges them with the values from `appsettings.json` at startup, with environment variables taking precedence. This mechanism ensures that production credentials are stored exclusively in the hosting environment and are never exposed through the source code or deployment artefacts.

4.2.14 API Documentation with Swagger

API documentation is implemented using Swashbuckle, the standard OpenAPI (Swagger) tooling for ASP.NET Core. The Swagger configuration is defined in the Swagger folder of the API project and registered via a service extension method in the Presentation project.

The Swagger UI is configured to support JWT Bearer token authentication directly from the browser-based testing interface. A security definition is added to the Swagger document specifying the Bearer token scheme, and a corresponding security requirement is applied globally to all endpoints. This configuration adds an "Authorize" button to the Swagger UI, allowing developers to enter a JWT access token once and have it automatically included as an Authorization: Bearer header in all subsequent test requests.

Each controller and action is annotated with XML documentation comments that Swashbuckle reads to populate endpoint descriptions and parameter documentation within the Swagger UI. Swagger is enabled in the development environment and on the production host to facilitate both local development and post-deployment verification.

4.2.15 CI/CD Pipeline

Continuous Integration is implemented using GitHub Actions, with workflow definitions stored in the `.github/workflows` directory of the repository. The workflow is configured to trigger on push and pull request events targeting the Dev branch, which serves as the primary integration branch for the project.

The workflow performs a build validation step: it checks out the repository, installs the .NET 9 SDK on the runner, restores all NuGet package dependencies using `dotnet restore`, and executes a full solution build using `dotnet build`. This automated build step ensures that every code change integrated into the Dev branch compiles successfully before it can be merged, preventing the introduction of compilation errors into the shared codebase.

4.2.16 API Testing with Postman

API endpoints are tested and documented using Postman, a widely adopted platform for API development and testing. The Postman collection and environment files are stored in the .postman directory at the root of the repository, making them accessible to all team members.

The collection is organized to mirror the controller hierarchy of the application, grouping requests by role and feature screen. Requests are organized under folders corresponding to Auth, Admin, Authority, PublicUser, and Worker, with sub-folders matching the screen-based feature organization (for example, Auth/Register, Auth/Login, Authority/AssignWorker).

Two environments are configured: a Local environment with a base URL of <http://localhost:{port}> for development testing, and a Production environment with the deployed API base URL. Each request uses the `{{baseUrl}}` Postman variable for its root address, enabling developers to switch between local and production testing by changing the active environment without modifying individual requests. Authentication-dependent requests use a shared Postman environment variable for the JWT access token, which is automatically populated by the login request's test script.

4.2.17 Deployment

The Infrastructure Reporting System is hosted on MonsterASP.NET, a shared hosting provider that serves the API through Internet Information Services (IIS) as the web server. The SQL Server database is hosted on DatabaseASP.NET, a cloud-hosted SQL Server service that provides the production database endpoint referenced by the application's connection string.

The deployment process uses Visual Studio's Publish profile targeting FTP (File Transfer Protocol) transport. The publish profile is configured with the FTP host, credentials, and the target IIS virtual directory path provided by MonsterASP.NET. To deploy, the developer runs the publish operation from Visual Studio, which compiles the application in Release configuration, produces a self-contained deployment bundle, and transfers the output files to the hosting server via FTP.

The database schema is deployed separately using a SQL migration script generated by the EF Core Script-Migration command. This script is executed against the production SQL Server instance via a database management tool prior to or alongside the application deployment.

The branching strategy follows a simplified Git Flow model. The Dev branch serves as the primary integration branch, into which feature branches are merged via Pull Requests. The

CI/CD pipeline validates each pull request before it is merged. A stable, deployable state on Dev is promoted to the main branch for release.

Chapter 5

Testing & Evaluation

- 1.1 Testing strategies (unit testing, integration testing, user testing).
- 1.2 Performance metrics (accuracy, speed, scalability, etc.).
- 1.3 Comparison with existing solutions (if applicable).

5.1 Introduction

This chapter presents the testing and evaluation activities conducted on the front-end implementation of the Smart Infrastructure Reporting System. The purpose of the testing process was to ensure that all implemented interfaces, navigation mechanisms, authentication functions, and user interactions operate correctly and satisfy the system requirements. Various testing approaches were applied to verify functionality, usability, security, and overall user experience.

5.2 Testing Strategy

The testing process focused on validating the functionality of the Angular-based front-end application. Testing activities were performed throughout the development process to ensure that each module operates correctly and integrates successfully with the back-end services.

The testing strategy included verification of user authentication processes, role-based navigation, report management interfaces, profile management functionalities, and communication with RESTful APIs. In addition, user interface testing was conducted to evaluate responsiveness, usability, and consistency across different pages of the application.

5.3 Front-End Functional Testing

Functional testing was conducted to verify that all implemented features perform according to the specified requirements. The Authentication Module was tested to ensure successful user registration, login, password recovery, and session management operations. The Public User Module was evaluated to verify report submission, report tracking, and profile management functionalities.

The Authority Module was tested to ensure that reports could be reviewed, monitored, and assigned correctly. Similarly, the Worker Module was evaluated to verify task viewing, status updates, and task completion processes. Administrative functionalities including user management and system supervision were also successfully tested.

The results confirmed that all major front-end functionalities operated correctly and communicated successfully with the back-end services.

5.4 User Interface and Usability Evaluation

User interface testing was performed to evaluate the overall usability and accessibility of the system. The implemented interfaces were reviewed to ensure consistency in design, navigation, and user interaction patterns.

Special attention was given to form validation, navigation menus, dashboard layouts, and feedback messages. The evaluation demonstrated that users can navigate between pages efficiently and perform system operations with minimal effort. Validation messages and error notifications also contributed to improving the overall user experience.

5.5 Security and Access Control Testing

Security testing focused on validating authentication and authorization mechanisms implemented within the front-end application. Route Guards were tested to ensure that unauthorized users could not access protected pages.

Role-based access control mechanisms were evaluated to verify that Public Users, Authorities, Workers, and Administrators could only access functionalities associated with their assigned roles. JWT token handling and secure API communication were also tested successfully.

The results confirmed that the implemented security mechanisms effectively protect sensitive system resources and user data.

5.6 Performance Evaluation

The front-end application demonstrated satisfactory performance during testing. Angular routing enabled smooth navigation between pages without requiring full page reloads, resulting in a responsive user experience.

API communication through Angular services was efficient and allowed data to be retrieved and displayed with minimal delay. The modular architecture and reusable components also contribute to maintainability and future scalability of the system.

Additionally, responsive design techniques ensured that the application remained usable across different screen sizes and device types.

5.7 Testing Screenshots

This section presents screenshots captured during the testing process. The screenshots demonstrate successful execution of the implemented functionalities and verify that the system interfaces operate as expected.

5.1 – Login Page:

Welcome Back

Please enter your details to access your reports.

Email

Password

[Forgot Password?](#)

Log In

Don't have an account? [Sign Up](#)

If forget password:



Forgot Password?

Enter your email and we'll send you an OTP code to reset your password.

Email Address

an1703@fayoum.edu.eg

Send OTP Code

[Back to Login](#)



Verify OTP

We sent a verification code to your email. Please enter the 6-digit code below.

7 0 2 9 3 4

Verify Code

[Resend](#)

[← Change Email](#)

Create New Password

Enter your email, OTP code, and new password

Email

an1703@fayoum.edu.eg

OTP Code

702934

New Password

.....

Strength: Good

- ✓ At least 8 characters
- ✓ Contains a number
- ✓ Contains a special character

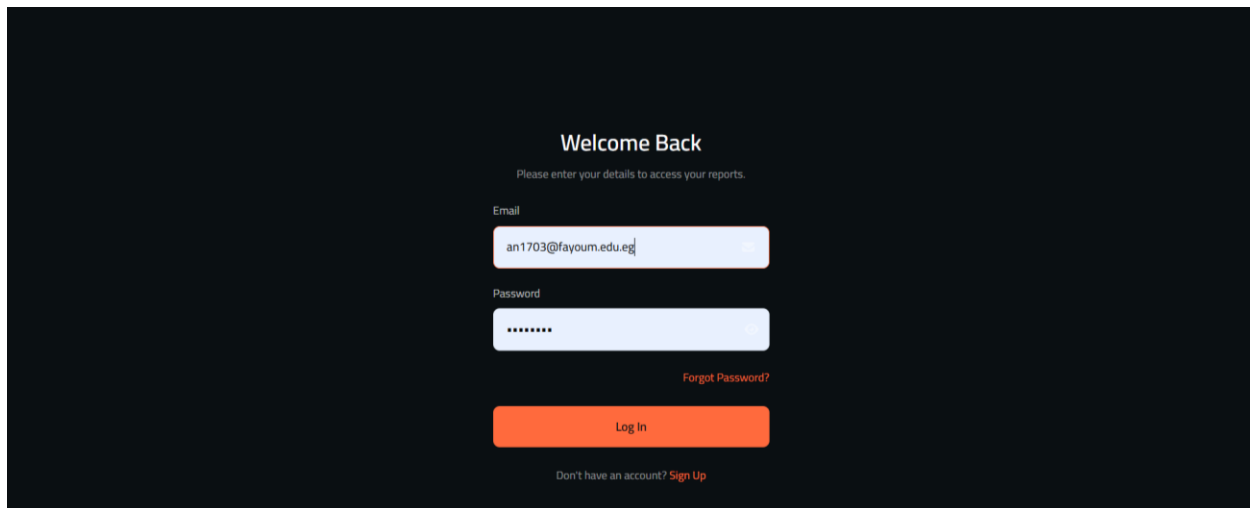
Confirm Password

.....

Reset Password

[← Back to login](#)

This sequence demonstrates the complete password recovery process. The user initiates a password reset by entering their registered email, receives a One-Time Password (OTP) for verification, and finally creates a new password after successful validation. The process was tested successfully and confirmed secure communication with the authentication system.



Welcome Back
Please enter your details to access your reports.

Email
an1703@fayoum.edu.eg

Password

[Forgot Password?](#)

Log In

Don't have an account? [Sign Up](#)

Figure 5.1: Login interface tested to verify user authentication and secure access to the system.

5.3 – Public User Dashboard:

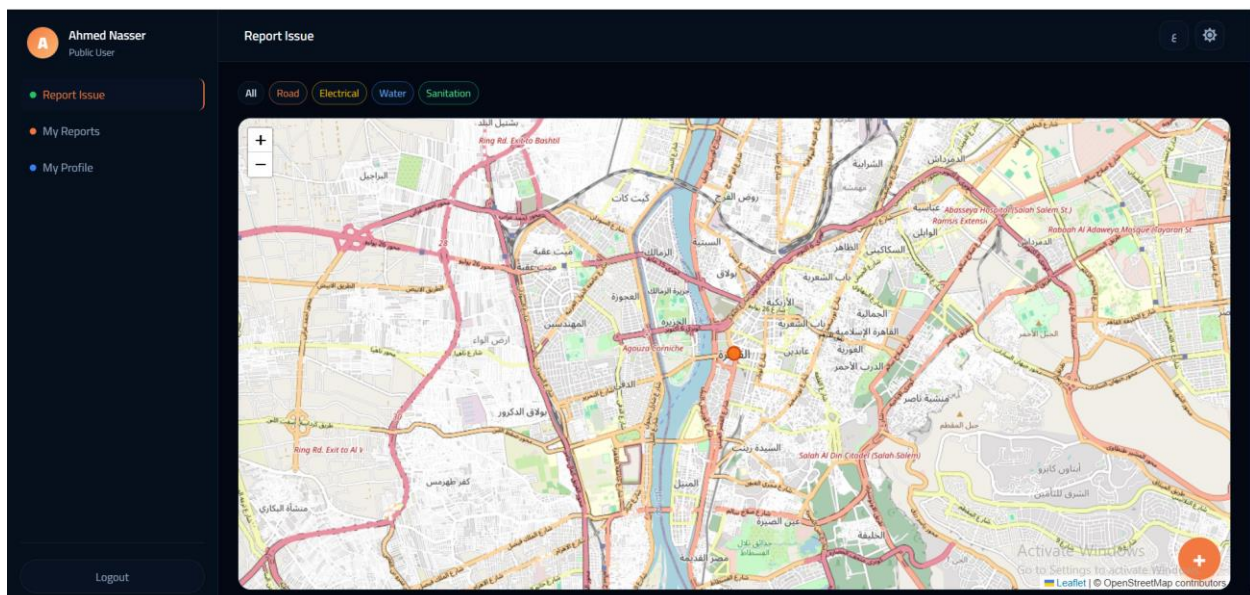


Figure 5.3: Public user dashboard displaying reporting and report-tracking functionalities.

5.4 – Submit Report Page:

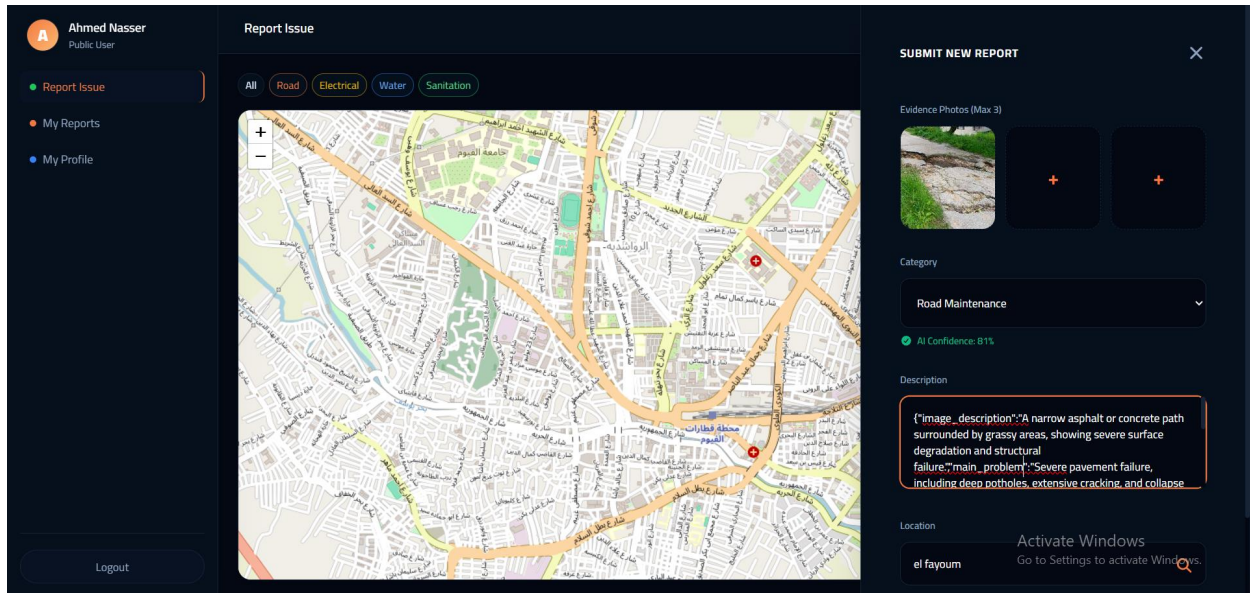
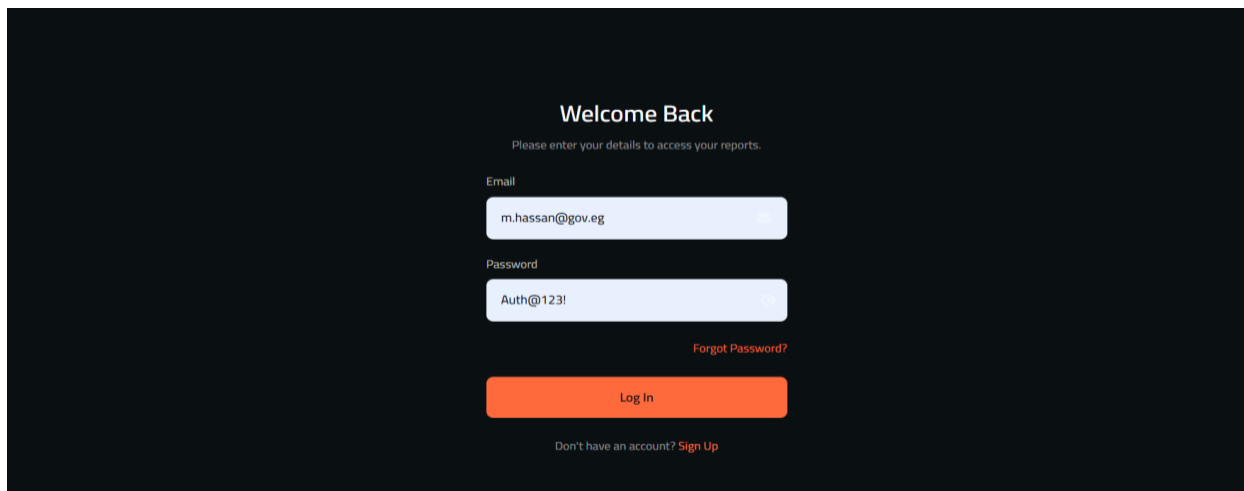


Figure 5.4: Report submission interface tested to verify issue reporting and data entry processes.

5.5 – Authority Dashboard:



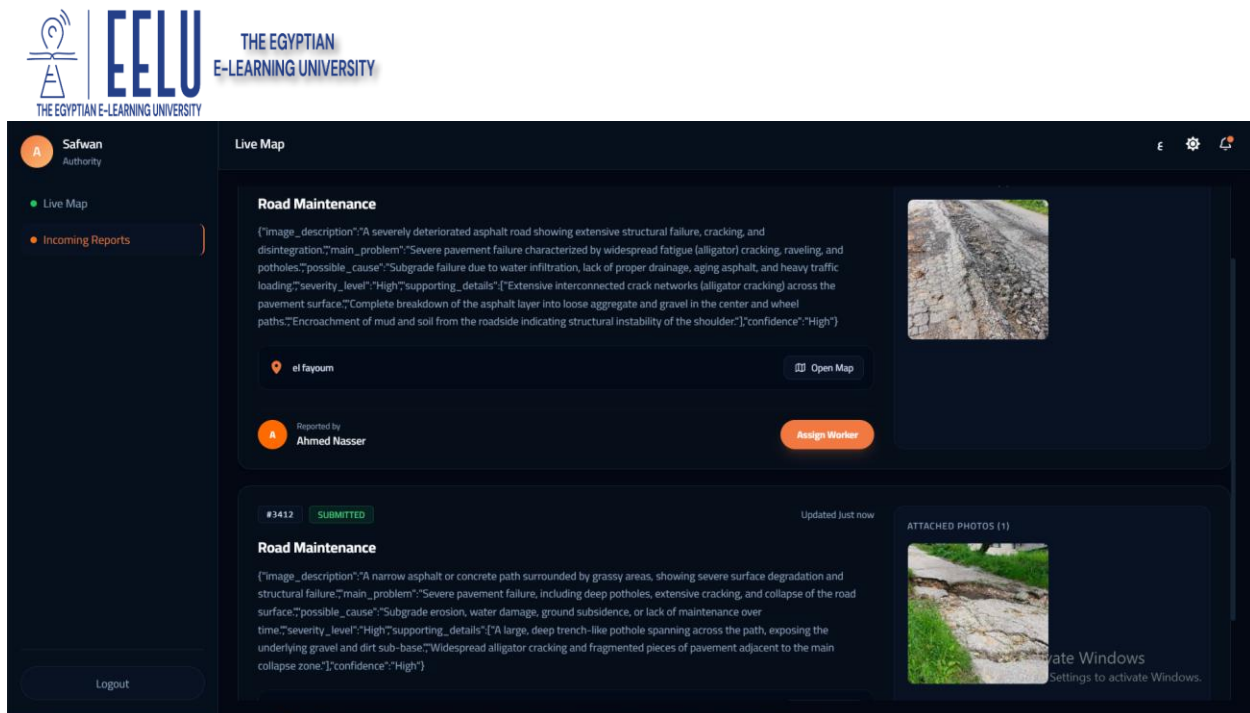
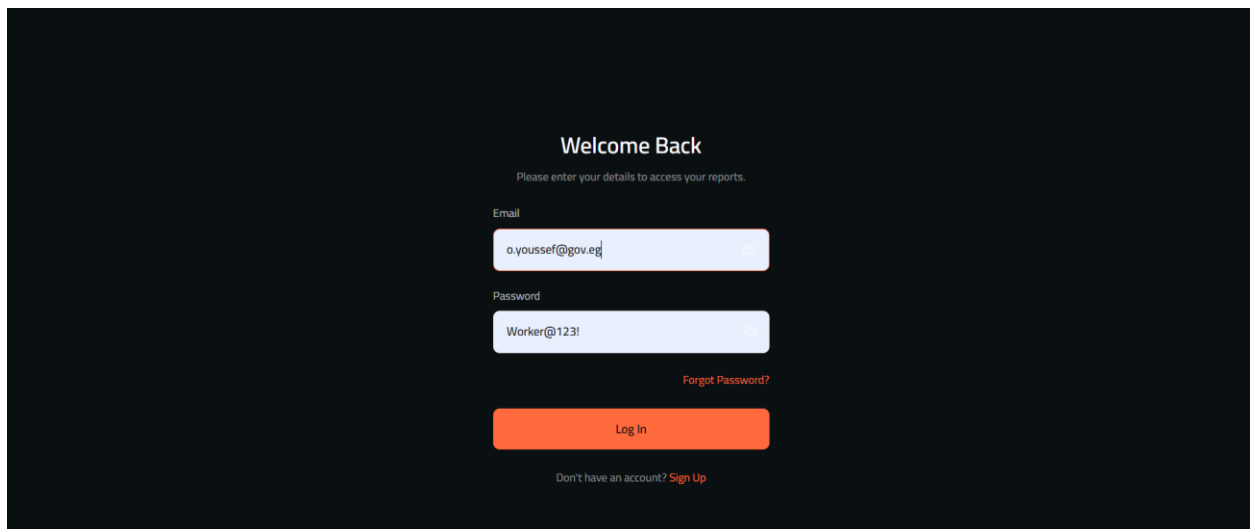


Figure 5.5: Authority dashboard demonstrating report monitoring and task assignment functionalities.

5.6 – Worker Dashboard:



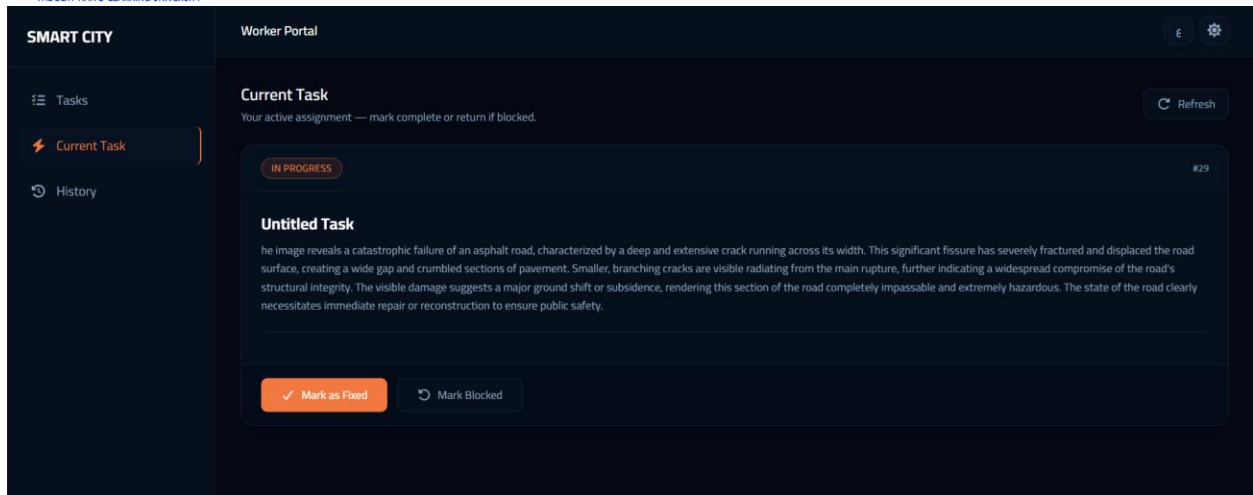
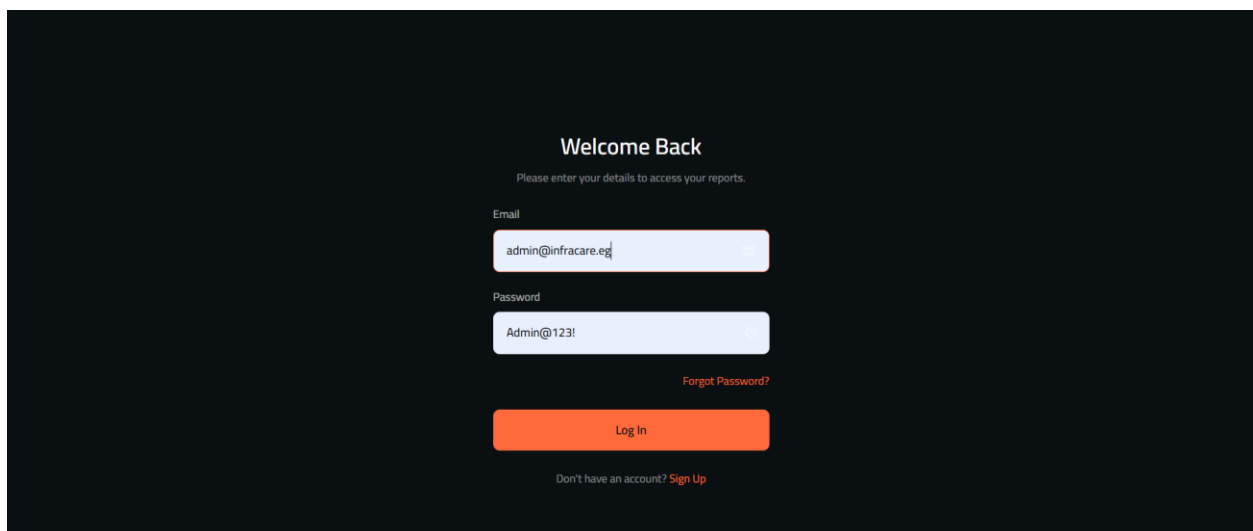
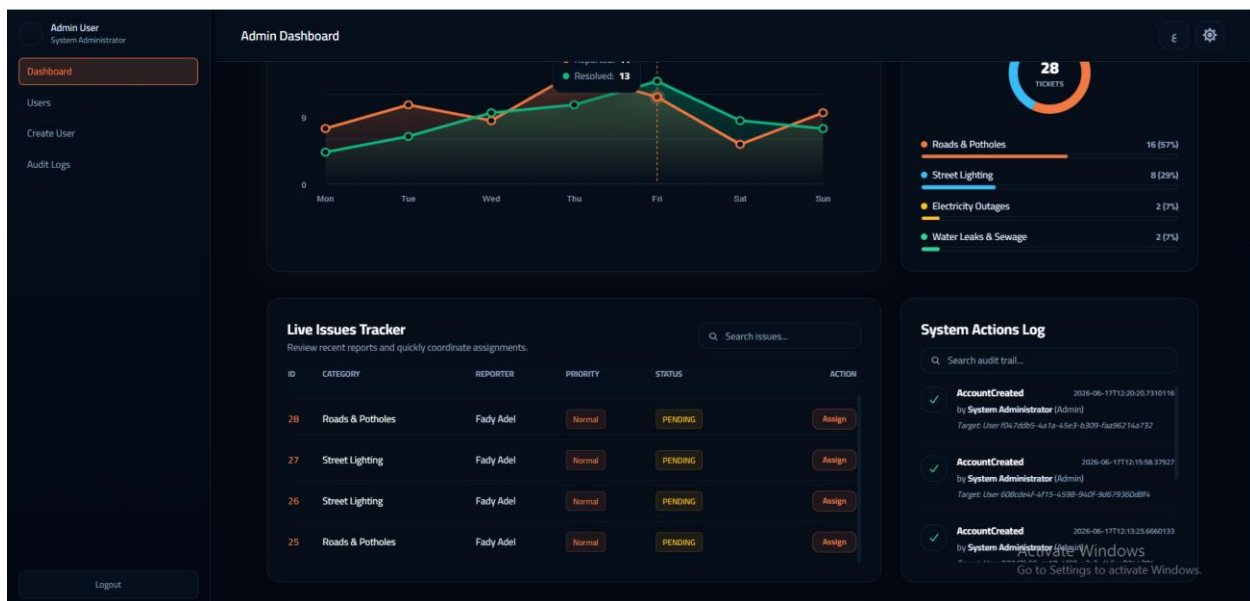
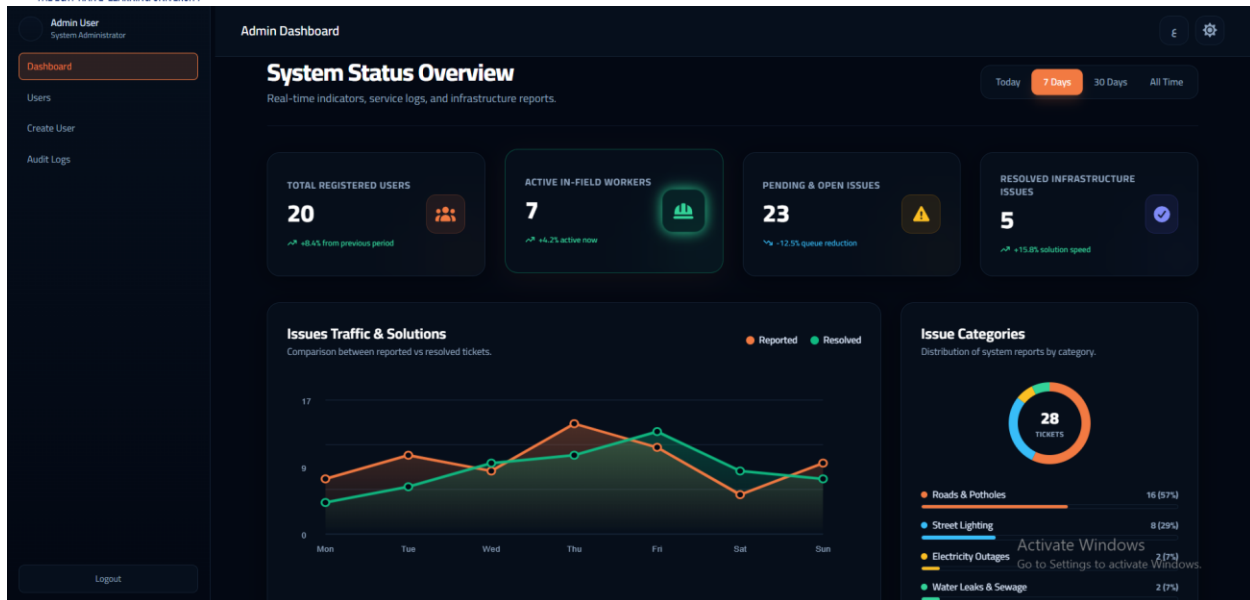
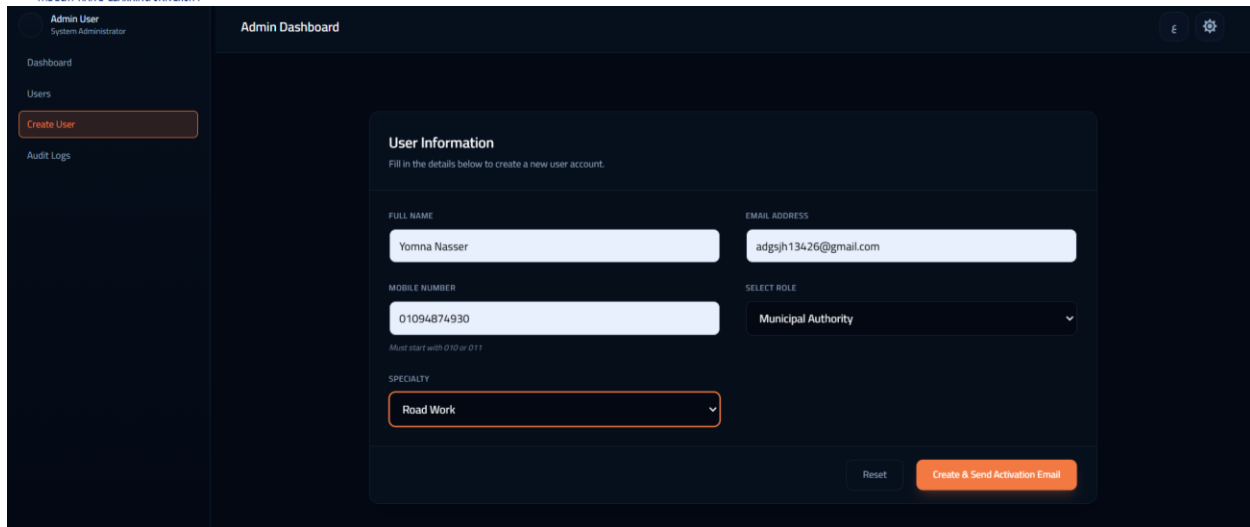


Figure 5.6: Worker interface displaying assigned tasks and progress management features.

5.7 – Administrator Dashboard:







Admin User
System Administrator

Admin Dashboard

Dashboard
Users
Create User
Audit Logs

User Information
Fill in the details below to create a new user account.

FULL NAME: Yomna Nasser

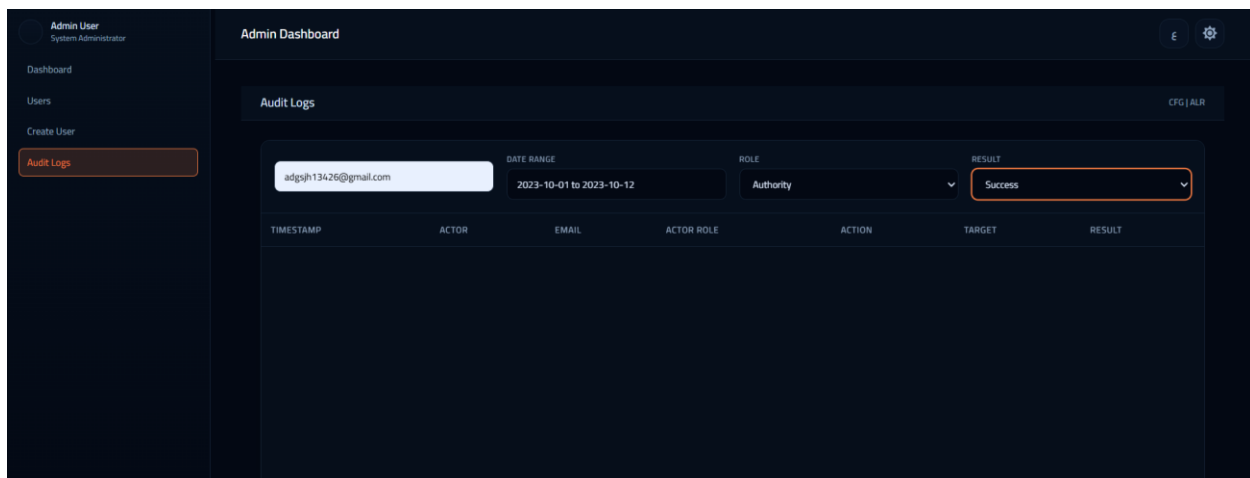
EMAIL ADDRESS: adgsjh13426@gmail.com

MOBILE NUMBER: 01094874930
Must start with 010 or 011

SELECT ROLE: Municipal Authority

SPECIALTY: Road Work

Reset Create & Send Activation Email



Admin User
System Administrator

Admin Dashboard

Dashboard
Users
Create User
Audit Logs

Audit Logs CFG | ALR

adgsjh13426@gmail.com DATE RANGE: 2023-10-01 to 2023-10-12 ROLE: Authority RESULT: Success

TIMESTAMP	ACTOR	EMAIL	ACTOR ROLE	ACTION	TARGET	RESULT

5.7: Administrative dashboard tested to verify user management and system supervision capabilities.

5.8 Unit Testing

5.9.1 Introduction

Unit testing was conducted throughout the development of the Infrastructure Reporting System to verify the correctness of individual software components in isolation. The primary objective of unit testing was to ensure that the system's business logic operates as expected under both normal and exceptional conditions. By isolating dependencies such as databases, external services, and APIs, the tests provide reliable validation of the service layer while reducing the risk of defects and regressions during future development.

The unit testing process focused on validating authentication workflows, report management operations, authority and worker functionalities, profile management, image processing, and AI-assisted report analysis. This approach contributed significantly to improving the reliability, maintainability, and overall quality of the system.

5.9.2 Testing Tools and Frameworks

The unit testing infrastructure was implemented using several industry-standard tools and frameworks.

Table 5.11: Testing Tools and Frameworks

Tool	Purpose
xUnit	Unit testing framework used to create and execute test cases
Moq	Mocking framework used to simulate external dependencies
FluentAssertions	Assertion library used to validate test outcomes
Microsoft.NET.Test.Sdk	Test execution and reporting platform
Coverlet	Code coverage analysis and reporting tool

The testing environment targeted the .NET 9 platform and followed the principles of Clean Architecture to ensure proper separation of concerns during testing.

5.9.3 Testing Methodology

The unit test suite followed the Arrange-Act-Assert (AAA) testing pattern. Each test case was organized into three phases: preparing test data and dependencies, executing the target functionality, and validating the expected results.

Mock objects were used to isolate the system under test from external dependencies such as repositories, email services, image services, and AI services. This ensured that each test evaluated only the behavior of the component being tested.

The testing methodology included:

- Service-layer testing to verify business logic.
- Validation testing to confirm correct handling of invalid inputs.

- Business-rule testing to ensure compliance with system requirements.
- Exception testing to verify proper handling of unexpected errors.
- Dependency isolation through mocking to ensure deterministic and repeatable test execution.

This methodology enabled comprehensive verification of the system's functionality while maintaining fast and reliable test execution.

5.9.4 Tested Components

The following table summarizes the main system modules covered by unit tests.

Table 5.12: Tested Components

Module	Services
Authentication	AuthService, OtpService
Administration	AdminDashboardService, AdminUsersService, AdminUserCreationService, AdminAuditLogsService
Authority	AuthorityWorkersService, AuthorityAssignWorkerService, AuthorityIncomingReportsService, AuthorityMapService, AuthorityMarkerPopupService
Public User	PublicSubmitReportService, PublicNearbyReportsService, PublicMapService, PublicUserReportsService, PublicAlsoSufferService, PublicMarkerPopupService
Worker	WorkerTasksService, WorkerTaskActionsService, WorkerCurrentTaskService, WorkerCurrentTaskActionsService, WorkerHistoryService
Shared Services	EmailService, ImageService, ProfileService, ProfileManagementService
AI Services	AnalyzeImageAIClient

Utility Components	EmailTemplates
--------------------	----------------

The tests covered authentication workflows, user management operations, report submission and processing, worker task management, authority assignment workflows, profile management, image handling, email notifications, and AI-assisted report analysis.

5.9.5 Test Coverage Summary

The overall composition of the unit test suite is summarized below.

Table 5.13: Unit Test Suite Summary

Metric	Value
Test Projects	1
Test Classes	28
Test Methods	222
Production Service Classes Tested	24
Test Framework	xUnit
Mocking Framework	Moq
Assertion Library	FluentAssertions
Coverage Tool	Coverlet

All service-layer classes within the application have dedicated unit tests, ensuring comprehensive validation of the project's business logic.

5.9.6 Test Coverage Results

Code coverage analysis was performed using Visual Studio Code Coverage together with Coverlet. The results indicate extensive validation of the production code.

Table 5.14: Overall Project Coverage

Project	Coverage
Infrastructure Reporting System (Overall)	97.46%
InfraReportingSystem.Domain	100.00%
InfraReportingSystem.Services	92.15%
InfraReportingSystem.Shared	97.68%

Table 5.15: Coverage by Module

Module	Coverage
AI Services	100.00%
Users Module	95.19%
Profile Services	100.00%
Authentication Services	83.83%
Image Services	70.73%
Email Services	69.64%

The results demonstrate that the majority of the application's business logic is exercised through automated unit tests. Critical modules such as Domain, AI Services, and Profile Services achieved complete coverage, while the remaining modules maintained high coverage levels.

5.9.7 Coverage Gaps

Although the overall solution achieved 97.46% code coverage, a small portion of the codebase remains uncovered.

Table 5.16: Coverage Gaps

Component	Coverage	Reason
-----------	----------	--------

EmailService	69.64%	Certain SMTP communication paths require integration testing
ImageService	70.73%	Cloudinary SDK interactions are difficult to fully simulate using unit tests
Authentication Services	83.83%	Some Identity framework edge cases remain uncovered
Configuration Classes	0.00%	Configuration classes contain properties only and no business logic

In addition, API controllers and repository implementations are not directly unit tested. These components primarily act as infrastructure layers and would be more appropriately validated through integration testing.

5.9.8 Results

The unit testing effort resulted in a comprehensive automated test suite consisting of 28 test classes and 222 test methods implemented within a dedicated test project. The test suite successfully validated the core business logic of the Infrastructure Reporting System, including authentication, report management, authority operations, worker workflows, profile management, image processing, email functionality, and AI-assisted analysis.

The project achieved an overall code coverage of 97.46%, demonstrating that the vast majority of the application's functionality is verified through automated testing. Critical modules such as the Domain layer, AI services, and Profile services achieved complete coverage, while the Services layer maintained a coverage level of 92.15%.

The testing results confirm that the system's core functionality operates correctly and that key business rules are enforced consistently. The implemented unit tests provide a strong foundation for future maintenance, enhancements, and regression prevention, contributing to the overall reliability and quality of the Infrastructure Reporting System.

Summary:

The testing and evaluation activities confirmed that the implementation satisfies the functional, usability, security, and performance requirements of the Smart Infrastructure Reporting System. All major modules operated successfully, communication with the back-end services was verified, and the application provided a responsive and user-friendly experience. The obtained

results indicate that the front-end application is stable, reliable, and ready for deployment and future enhancement.

Chapter 6

Results & Discussion

1.1 Introduction

1.2 Summary of findings.

1.3 Interpretation of results (Did the project meet its objectives?).

1.4 Limitations of the proposed solution.

6.1 Introduction:

This chapter presents the results achieved by the Smart Infrastructure Reporting System and discusses how the implemented solution addresses the challenges identified in earlier chapters. The chapter evaluates the effectiveness of the system in improving infrastructure issue reporting, communication between stakeholders, and overall management efficiency. It also analyzes the extent to which the project objectives were achieved and highlights the limitations encountered during development.

The evaluation presented in this chapter is based on the implemented functionalities, user role interactions, security mechanisms, and workflow management processes. The discussion focuses on how the developed platform contributes to improving the efficiency of infrastructure issue reporting and supports better coordination between citizens, authorities, workers, and administrators.

6.2 Summary of Findings:

The developed Smart Infrastructure Reporting System successfully provides a centralized platform that connects citizens, government authorities, field workers, and administrators within a single environment.

The implementation demonstrated the following key findings:

- Citizens can submit infrastructure reports easily using a web-based interface.
- Reports can be tracked throughout their lifecycle, improving transparency and user engagement.
- Authorities can review, manage, and assign reports efficiently through dedicated dashboards.
- Workers can receive tasks, update progress, and provide completion information directly through the system.
- The authentication and authorization mechanisms ensure secure access control for all user roles.
- Cloud-based image storage and management simplify report documentation and verification.
- The system architecture supports scalability and maintainability through the adoption of Clean Architecture principles.

Furthermore, the system demonstrates the practical application of modern software engineering practices, including role-based access control, cloud integration, and modular architecture, which contribute to improved system reliability and maintainability.

The integration of these features within a single platform reduces dependency on traditional reporting methods and enhances communication among all stakeholders. As a result, the system provides a more organized, transparent, and efficient approach to managing infrastructure-related issues.

Overall, the system successfully integrates reporting, task management, communication, and monitoring processes into one unified platform, providing a practical solution for smart infrastructure management.

6.3 Interpretation of Results:

The results indicate that the project successfully achieved its primary objectives.

Objective 1: Providing a Multi-Role Infrastructure Reporting Platform

The first objective was to provide a multi-role infrastructure reporting platform. This objective was achieved through the implementation of separate functionalities for Public Users, Authorities, Workers, and Administrators.

The implementation of multiple user roles ensures that each stakeholder can interact with the system according to their responsibilities. Citizens focus on reporting issues, authorities manage and assign reports, workers handle field operations, and administrators oversee the entire platform. This separation of responsibilities improves operational efficiency and reduces the possibility of unauthorized access or misuse of system resources.

Objective 2: Improving Issue Management and Coordination

The second objective was to improve issue management and coordination. The centralized dashboard and assignment workflow significantly enhance communication between stakeholders and reduce the complexity of traditional reporting procedures.

The centralized workflow introduced by the system reduces delays commonly associated with traditional reporting methods. By providing a unified platform for communication and task assignment, the system minimizes information loss and enables faster coordination between authorities and field workers. This contributes to more efficient issue resolution and better service delivery.

Objective 3: Enhancing Security and Reliability

The third objective focused on security and reliability. Through JWT authentication, role-based authorization, OTP verification, and refresh token mechanisms, the system provides a secure environment for managing user accounts and sensitive data.

Security mechanisms play a critical role in ensuring trust within the platform. The combination of authentication, authorization, and verification processes protects sensitive user information and prevents unauthorized access. As a result, the system provides a reliable environment for both citizens and government entities to exchange information securely.

The project also achieved its goal of improving transparency by allowing citizens to track report status updates and receive notifications regarding issue progress. This functionality increases user confidence in the reporting process and encourages greater citizen participation in identifying and reporting infrastructure issues.

Based on these outcomes, it can be concluded that the proposed system effectively addresses the problem statement presented in Chapter 1 and offers a practical solution for smart infrastructure management. The achieved results demonstrate the feasibility of adopting digital transformation approaches in infrastructure management and highlight the benefits of integrating modern technologies into public service operations.

6.4 Limitations of the Proposed Solution:

Although the system achieves its intended objectives, several limitations remain.

Large-Scale Deployment Evaluation

System performance has not yet been evaluated under large-scale real-world deployment involving thousands of simultaneous users. Additional performance testing is required to assess scalability and system responsiveness under heavy workloads.

Lack of Mobile Applications

Mobile applications for Android and iOS have not yet been developed. The current solution relies on a responsive web interface, which provides accessibility across devices but does not offer the full functionality and convenience of dedicated mobile applications.

External System Integration

Integration with government information systems and external municipal databases is currently not implemented. Future integration would allow better data sharing, improved coordination, and more efficient infrastructure management processes.

These limitations are common in early-stage smart city solutions and do not reduce the practical value of the proposed system. Instead, they highlight opportunities for future development and integration with emerging technologies

Most of these limitations are related to deployment scope and future integrations rather than core system functionality, indicating that the current implementation provides a stable and extensible foundation for future development.

Despite these limitations, the developed system provides a strong foundation for future enhancements and real-world deployment. The modular architecture adopted in the system ensures that additional features and integrations can be incorporated in future versions without requiring major modifications to the existing platform.

Chapter 7

Conclusion & Future Work

7.1 Summary of Contributions

This project presented the design and implementation of a Smart Infrastructure Reporting System aimed at improving the reporting, management, and resolution of urban infrastructure issues.

The system contributes to the field of smart city solutions by providing:

- A centralized platform that connects all stakeholders.
- Secure authentication and role-based access control.
- Efficient report submission and tracking mechanisms.
- Task assignment and management capabilities for authorities and workers.
- Cloud-based image management and documentation.
- A scalable architecture based on modern software engineering principles.

The project demonstrates how digital technologies can enhance urban infrastructure management, improve citizen participation, and support more efficient governmental operations.

In conclusion, the developed system successfully addresses the identified problem and provides a practical framework for smarter and more responsive infrastructure management.

7.2 Strengths of the Proposed System:

The developed Smart Infrastructure Reporting System offers several strengths that contribute to its effectiveness and practical applicability.

First, the system provides a centralized platform that enables seamless communication and coordination among citizens, government authorities, field workers, and administrators. This integration reduces fragmentation and improves workflow efficiency.

Second, the implementation of secure authentication and authorization mechanisms, including JWT authentication, OTP verification, refresh tokens, and role-based access control, enhances system security and protects sensitive user data.

Third, the adoption of Clean Architecture principles improves maintainability, scalability, and flexibility, allowing future enhancements to be integrated with minimal impact on existing components.

Finally, the integration of cloud-based image storage and report tracking features increases transparency and enables users to monitor the progress of reported infrastructure issues efficiently.

7.3 Challenges and Limitations:

Despite the successful implementation of the proposed solution, several challenges and limitations remain.

In addition, the AI-based classification component is limited by the availability of training datasets and requires further improvement to achieve higher accuracy across different infrastructure categories.

Furthermore, the system has not yet been evaluated in large-scale real-world deployments involving thousands of concurrent users. Additional performance and stress testing are required before full-scale adoption.

These limitations provide valuable opportunities for future improvements and technological integration.

7.4 Future Work

Several enhancements can be considered in future versions of the system:

1. **AI-Based Severity Assessment:** The AI module can be extended to automatically determine the severity and priority level of infrastructure issues based on image analysis, helping authorities prioritize urgent cases more effectively.
2. **Damage Localization and Assessment:** The AI module can be enhanced to identify the exact location and extent of damage within uploaded images using advanced computer vision techniques.

3. **Smart Worker Recommendation:** The system can automatically recommend the most suitable worker based on specialization, location, and workload using intelligent assignment algorithms.

Summary:

This chapter summarized the major contributions of the Smart Infrastructure Reporting System and discussed potential directions for future development. The proposed solution provides a practical foundation for improving infrastructure issue reporting and management through digital transformation technologies. Future enhancements, including mobile applications, IoT integration, advanced AI capabilities, and cloud-native deployment, can further increase the effectiveness and impact of the system in smart city environments.

References

- [1] Almulhim, A. I. (2025). Integrating Artificial Intelligence into Smart Infrastructure Management for Sustainable Urban Planning. *Technologies*, 13(11), 481.
- [2] Ejaz, N. (2025). Generative AI-driven edge-cloud system for intelligent road infrastructure inspection. *ScienceDirect*.
- [3] Project Document. Smart Infrastructure Reporting System (pasted_file_aewYYV_Smart_Infrastructure_Reporting_System.pptx).
- [4] Hexagon. Citizen reporting smartphone app.
- [5] GovPilot. Modern Citizen Engagement: How Local Governments Use Digital Platforms.
- [6] Streamwide. Team Signaling for Smart Cities.
- [7] Kumar, D. (2025). Advancing Bridge Infrastructure Management through LLM-based Systems. *International Journal of Business and Engineering Management Research*.

APPENDICES (OPTIONAL)

Datasets

[8] Hyman, A. (2023). Urban Issues Dataset. Kaggle.

<https://www.kaggle.com/datasets/akinduhiman/urban-issues-dataset>

- Survey questionnaires (if applicable).